

✓ Building Damage Classification

```
import zipfile
import os
import numpy as np
import pickle
from skimage import io
from skimage.transform import resize
import matplotlib.pyplot as plt
from tqdm import tqdm
```

✓ Loading the data

```
# Data Load 1 (this is when done on Google Colab)
# Google drive set up
from google.colab import drive
drive.mount('/content/drive')

# Extract Tier_1 (training) images
tier1_base = '/content/drive/MyDrive/Building Damage Assessment/Tier_1'
tier1_images_zip = os.path.join(tier1_base, 'images.zip')

if os.path.exists(tier1_images_zip):
    with zipfile.ZipFile(tier1_images_zip, 'r') as zip_ref:
        zip_ref.extractall('/content/tier1')
    print("Tier1 images extracted")

# Extract Holdout (validation) images
holdout_base = '/content/drive/MyDrive/Building Damage Assessment/Holdout'
holdout_images_zip = os.path.join(holdout_base, 'images.zip')

if os.path.exists(holdout_images_zip):
    with zipfile.ZipFile(holdout_images_zip, 'r') as zip_ref:
        zip_ref.extractall('/content/holdout')
    print("Holdout images extracted")

# Metadata
metadata_path = '/content/drive/MyDrive/Building Damage Assessment/Building_Damage_Metadata/building_metadata.pkl'

with open(metadata_path, 'rb') as f:
    metadata = pickle.load(f)

train_metadata = metadata['train']
val_metadata = metadata['val']

print(f"Training samples: {len(train_metadata):,}")
print(f"Validation samples: {len(val_metadata):,}")
print(f"Total samples: {len(train_metadata) + len(val_metadata):,}")
```

```

↳ Mounted at /content/drive
Tier1 images extracted
Holdout images extracted
Training samples: 119,310
Validation samples: 40,054
Total samples: 159,364

# Data Load 2 (this is when done on local machine)
# Metadata
metadata_path = r'Building_Damage_Metadata\building_metadata.pkl'

with open(metadata_path, 'rb') as f:
    metadata = pickle.load(f)

train_metadata = metadata['train']
val_metadata = metadata['val']

print(f"Training samples: {len(train_metadata):,}")
print(f"Validation samples: {len(val_metadata):,}")
print(f"Total samples: {len(train_metadata) + len(val_metadata):,}")

for entry in train_metadata:
    entry['image_path'] = entry['image_path'].replace('/content/tier1/images/', 'geotiffs/tier1/images/')

for entry in val_metadata:
    entry['image_path'] = entry['image_path'].replace('/content/hold/images/', 'geotiffs/hold/images/')

print(train_metadata[0]['image_path'])

# Folder where actual images are stored
tier1_folder = r'geotiffs/tier1/images'
tier1_files = os.listdir(tier1_folder)
metadata_filenames = [os.path.basename(entry['image_path']) for entry in train_metadata]

# Files that the metadata expects but are not in the folder
missing_files = [fname for fname in metadata_filenames if fname not in tier1_files]

print(f"Files listed in metadata but missing in folder: {len(missing_files)}")
if missing_files:
    print("Example missing:", missing_files[:5])
else:
    print("All metadata files are present in the folder!")

# Class distribution Analysis
def analyze_class_distribution(metadata_list):
    from collections import Counter

    labels = [meta['damage_label'] for meta in metadata_list]
    distribution = Counter(labels)

    print("Class Distribution:")
    total = len(labels)
    for label, count in distribution.items():
        percentage = (count / total) * 100

```

```

    print(f"    {label}: {count:}, ({percentage:.1f}%)"

# Check for imbalance
max_count = max(distribution.values())
min_count = min(distribution.values())
imbalance_ratio = max_count / min_count

print(f"\nImbalance ratio: {imbalance_ratio:.1f}:1")

if imbalance_ratio > 5:
    print("Class imbalance detected")

return distribution

# Analyze both splits
print("TRAINING SET:")
train_dist = analyze_class_distribution(train_metadata)
print("\nVALIDATION SET:")
val_dist = analyze_class_distribution(val_metadata)

TRAINING SET:
Class Distribution:
  no-damage: 88,961 (74.6%)
  destroyed: 7,507 (6.3%)
  major-damage: 11,802 (9.9%)
  minor-damage: 11,040 (9.3%)

Imbalance ratio: 11.9:1
Class imbalance detected

VALIDATION SET:
Class Distribution:
  no-damage: 28,852 (72.0%)
  minor-damage: 4,715 (11.8%)
  major-damage: 3,840 (9.6%)
  destroyed: 2,647 (6.6%)

Imbalance ratio: 10.9:1
Class imbalance detected

```

✓ Modelling

```

import copy
import time
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from PIL import Image
import torchvision.models as models
import torchvision.transforms as transforms
from sklearn.metrics import classification_report, confusion_matrix
import torch.nn.functional as F

```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(device)
```

↗ cuda

▼ Class Weights

```
# Calculate class weights for imbalanced dataset
def calculate_class_weights(metadata_list):

    from sklearn.utils.class_weight import compute_class_weight

    labels = [meta['damage_label'] for meta in metadata_list]
    label_map = {'no-damage': 0, 'minor-damage': 1, 'major-damage': 2, 'destroyed': 3}
    numeric_labels = [label_map[label] for label in labels]

    class_weights = compute_class_weight('balanced',
                                         classes=np.unique(numeric_labels),
                                         y=numeric_labels)

    weight_dict = {i: weight for i, weight in enumerate(class_weights)}
    print("Class weights:")
    label_names = ['no-damage', 'minor-damage', 'major-damage', 'destroyed']
    for i, weight in weight_dict.items():
        print(f"    {label_names[i]}: {weight:.3f}")

    return torch.FloatTensor(class_weights)

# Calculate weights for training set
class_weights = calculate_class_weights(train_metadata)
```

↗ Class weights:

```
no-damage: 0.335
minor-damage: 2.702
major-damage: 2.527
destroyed: 3.973
```

▼ Dataset and Data Processing

```
class BuildingDamageDataset(Dataset):
    def __init__(self, metadata_list, target_size=(224, 224), transform=None):
        self.metadata = metadata_list
        self.target_size = target_size # Kept for compatibility, though resizing is now in transforms
        self.transform = transform

    # Label mapping
    self.label_map = {
        'no-damage': 0,
        'minor-damage': 1,
        'major-damage': 2,
        'destroyed': 3
    }
```

```

def __len__(self):
    return len(self.metadata)

def __getitem__(self, idx):
    try:
        meta = self.metadata[idx]

        # Load satellite image
        image = io.imread(meta['image_path'])

        # Handle different image formats
        if len(image.shape) == 2:
            image = np.stack([image, image, image], axis=-1) # Convert grayscale to RGB
        elif len(image.shape) == 3 and image.shape[2] > 3:
            image = image[:, :, :3] # Take first 3 channels if multi-channel

        # Extract crop using stored bbox
        min_x, min_y, max_x, max_y = meta['bbox']
        crop = image[min_y:max_y, min_x:max_x]

        # Convert to float32 and ensure range [0, 255] for transforms
        crop = crop.astype(np.float32)
        if crop.max() > 1.0: # If image is in [0, 255] range, keep it; otherwise, scale
            crop = crop
        else:
            crop = crop * 255.0 # Scale [0, 1] to [0, 255] if needed

        # Apply transforms (resizing and normalization now on GPU)
        if self.transform:
            crop = self.transform(crop)

        # Get label
        label = self.label_map[meta['damage_label']]

        return crop, label

    except Exception as e:
        print(f"Error loading sample {idx}: {e}")
        # Return a dummy sample in case of error
        dummy_crop = torch.zeros(3, self.target_size[0], self.target_size[1])
        return dummy_crop, 0

train_transforms = transforms.Compose([
    transforms.ToPILImage(), # Convert numpy array to PIL Image (required for some transforms)
    transforms.Resize((224, 224), interpolation=transforms.InterpolationMode.BICUBIC), # Resize to 224x224
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(10),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(), # Convert to tensor (GPU-compatible)
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) # ImageNet normalization
])

# Validation transforms with resizing and normalization on GPU
val_transforms = transforms.Compose([
    transforms.ToPILImage(), # Convert numpy array to PIL Image
    transforms.Resize((224, 224), interpolation=transforms.InterpolationMode.BICUBIC), # Resize to 224x224

```

```

    transforms.ToTensor(), # Convert to tensor
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) # ImageNet normalization
])

# Datasets
train_dataset = BuildingDamageDataset(train_metadata, target_size=(224, 224), transform=train_transforms)
val_dataset = BuildingDamageDataset(val_metadata, target_size=(224, 224), transform=val_transforms)

# DataLoaders
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True, num_workers=2, pin_memory=True if torch.cuda.is_available() else False)
val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False, num_workers=2, pin_memory=True if torch.cuda.is_available() else False)

```

✓ Training Functions

```

# Main Training Function
def train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15, model_name="Model"):

    model = model.to(device)

    # Track best model
    best_model_wts = copy.deepcopy(model.state_dict())
    best_acc = 0.0

    # Time tracking
    since = time.time()

    # Training metrics history
    history = {
        'train_loss': [],
        'val_loss': [],
        'train_acc': [],
        'val_acc': [],
        'best_epoch': 0
    }

    print(f"Training {model_name}")
    print("=" * 50)

    # Main training loop
    for epoch in range(num_epochs):
        print(f"Epoch {epoch+1}/{num_epochs}")
        print('-' * 20)

        # Each epoch has training and validation phase
        for phase in ['train', 'val']:
            if phase == 'train':
                model.train()
                dataloader = train_loader
            else:
                model.eval()
                dataloader = val_loader

            running_loss = 0.0
            running_corrects = 0

```

```

all_preds = []
all_labels = []

# Progress bar for batches
pbar = tqdm(dataloader, desc=f'{phase}')

for inputs, labels in pbar:
    inputs, labels = inputs.to(device), labels.to(device)

    optimizer.zero_grad()

    # Forward pass
    with torch.set_grad_enabled(phase == 'train'):
        outputs = model(inputs)
        _, preds = torch.max(outputs, 1)
        loss = criterion(outputs, labels)

        if phase == 'train':
            loss.backward()
            optimizer.step()

    # Statistics
    running_loss += loss.item() * inputs.size(0)
    running_corrects += torch.sum(preds == labels.data)

    # Update progress bar
    pbar.set_postfix({
        'loss': f'{loss.item():.4f}',
        'acc': f'{running_corrects.double() / ((pbar.n + 1) * inputs.size(0)):.4f}'
    })

    # Collect for metrics
    all_preds.extend(preds.cpu().numpy())
    all_labels.extend(labels.cpu().numpy())

# Calculate epoch statistics
dataset_size = len(dataloader.dataset)
epoch_loss = running_loss / dataset_size
epoch_acc = running_corrects.double() / dataset_size

print(f"{phase} Loss: {epoch_loss:.4f} Acc: {epoch_acc:.4f}")

# Store history
if phase == 'train':
    history['train_loss'].append(epoch_loss)
    history['train_acc'].append(epoch_acc.item())
else:
    history['val_loss'].append(epoch_loss)
    history['val_acc'].append(epoch_acc.item())

# Check if best model
if epoch_acc > best_acc:
    best_acc = epoch_acc
    best_model_wts = copy.deepcopy(model.state_dict())
    history['best_epoch'] = epoch

```

```

    # Print classification report every 5 epochs or at the end
    if (epoch + 1) % 5 == 0 or epoch == num_epochs - 1:
        print("\nClassification Report:")
        class_names = ['no-damage', 'minor-damage', 'major-damage', 'destroyed']
        print(classification_report(all_labels, all_preds,
                                   target_names=class_names,
                                   zero_division=0))

    # Update scheduler
    if scheduler is not None:
        scheduler.step(history['val_loss'][-1]) # Step on validation loss

    print()

# Training complete
time_elapsed = time.time() - since
print(f"Training complete in {time_elapsed // 60:.0f}m {time_elapsed % 60:.0f}s")
print(f"Best val acc: {best_acc:.4f} (epoch {history['best_epoch'] + 1})")

# Load best model weights
model.load_state_dict(best_model_wts)
return model, history

# Train function (adjusted for Ordinal Loss)
def train_model1(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15, model_name="Model", use_ordinal=False):

    model = model.to(device)

    # Track best model
    best_model_wts = copy.deepcopy(model.state_dict())
    best_acc = 0.0

    # Time tracking
    since = time.time()

    # Training metrics history
    history = {
        'train_loss': [], 'val_loss': [],
        'train_acc': [], 'val_acc': [],
        'best_epoch': 0
    }

    print(f"Training {model_name}")
    print("=" * 50)

    for epoch in range(num_epochs):
        print(f"Epoch {epoch+1}/{num_epochs}")
        print('-' * 20)

        for phase in ['train', 'val']:
            if phase == 'train':
                model.train()
                dataloader = train_loader
            else:
                model.eval()
                dataloader = val_loader

```



```

running_loss = 0.0
running_corrects = 0
all_preds = []
all_labels = []

pbar = tqdm(dataloader, desc=f'{phase}')

for inputs, labels in pbar:
    inputs, labels = inputs.to(device), labels.to(device)

    optimizer.zero_grad()

    with torch.set_grad_enabled(phase == 'train'):
        outputs = model(inputs)

        if use_ordinal:
            # Pass the model itself to access its cutpoints (thresholds)
            preds = ordinal_predict(outputs)
        else:
            _, preds = torch.max(outputs, 1)

        # Loss
        loss = criterion(outputs, labels)

        if phase == 'train':
            loss.backward()
            optimizer.step()

    # Stats
    running_loss += loss.item() * inputs.size(0)
    # Ensure preds and labels are on the same device and data type for comparison
    running_corrects += torch.sum(preds.to(labels.device) == labels.data)

    pbar.set_postfix({
        'loss': f'{loss.item():.4f}',
        'acc': f'{running_corrects.double() / ((pbar.n + 1) * inputs.size(0)):.4f}'
    })

    all_preds.extend(preds.cpu().numpy())
    all_labels.extend(labels.cpu().numpy())

dataset_size = len(dataloader.dataset)
epoch_loss = running_loss / dataset_size
epoch_acc = running_corrects.double() / dataset_size

print(f'{phase} Loss: {epoch_loss:.4f} Acc: {epoch_acc:.4f}')

if phase == 'train':
    history['train_loss'].append(epoch_loss)
    history['train_acc'].append(epoch_acc.item())
else:
    history['val_loss'].append(epoch_loss)

```

```

        history['val_acc'].append(epoch_acc.item())

    if epoch_acc > best_acc:
        best_acc = epoch_acc
        best_model_wts = copy.deepcopy(model.state_dict())
        history['best_epoch'] = epoch

    if (epoch + 1) % 5 == 0 or epoch == num_epochs - 1:
        print("\nClassification Report:")
        class_names = ['no-damage', 'minor-damage', 'major-damage', 'destroyed']
        print(classification_report(all_labels, all_preds,
                                   target_names=class_names, zero_division=0))

    if scheduler is not None:
        scheduler.step(history['val_loss'][-1])

    print()

    time_elapsed = time.time() - since
    print(f"Training complete in {time_elapsed // 60:.0f}m {time_elapsed % 60:.0f}s")
    print(f"Best val acc: {best_acc:.4f} (epoch {history['best_epoch'] + 1})")

    model.load_state_dict(best_model_wts)
    return model, history

def ordinal_predict(logits):
    # logits: [batch_size, K-1]
    # Predict by comparing logits to 0 (thresholds are implicitly at 0 in the loss)
    # The number of thresholds crossed indicates the class
    return torch.sum(logits > 0, dim=1)

```

✓ Focal Loss

```

class FocalLoss(nn.Module):
    def __init__(self, alpha=None, gamma=2, reduction='mean'):
        super(FocalLoss, self).__init__()
        self.alpha = alpha # controls class imbalance, tensor or None
        self.gamma = gamma # focuses on hard examples
        self.reduction = reduction
        if self.alpha is not None and not isinstance(self.alpha, torch.Tensor):
            raise ValueError("alpha must be a tensor or None")

    def forward(self, inputs, targets):
        # Use Cross entropy, pass weight only if alpha is provided
        CE_loss = F.cross_entropy(inputs, targets, reduction='none', weight=self.alpha if self.alpha is not None else None)
        pt = torch.exp(-CE_loss)

        # Apply the focal adjustment
        focal_loss = (1 - pt) ** self.gamma * CE_loss

        # Apply reduction (mean, sum, or no reduction)
        if self.reduction == 'mean':
            return focal_loss.mean()
        elif self.reduction == 'sum':

```

```

        return focal_loss.sum()
    else:
        return focal_loss

```

✓ Ordinal Loss

```

class OrdinalLoss(nn.Module):
    def __init__(self, weight=None, reduction='mean'): # optional parameters and using mean to agg the loss
        super(OrdinalLoss, self).__init__()
        self.weight = weight
        self.reduction = reduction

    def forward(self, logits, targets): # forward pass (logits: predictions, targets: groundtruths)
        # logits: [batch_size, K-1]
        # targets: [batch_size]
        batch_size, num_thresholds = logits.size()
        thresholds = torch.arange(num_thresholds).unsqueeze(0).to(logits.device) # [1, K-1]
        cumulative_targets = (targets.unsqueeze(1) > thresholds).float() # [batch_size, K-1]
        bce = F.binary_cross_entropy_with_logits(logits, cumulative_targets, reduction='none')
        if self.weight is not None:
            weights = self.weight[targets].unsqueeze(1).to(logits.device)
            bce = bce * weights
        if self.reduction == 'mean':
            return bce.mean()
        elif self.reduction == 'sum':
            return bce.sum()
        else:
            return bce

```

✓ Ablation Study Of Loss Functions

✓ ResNET

```

from torchvision.models import resnet50, ResNet50_Weights

# ResNET Model with Weighted CE
model = models.resnet50(weights=ResNet50_Weights.DEFAULT)
for param in model.parameters():
    param.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, 4)
for param in model.layer4.parameters():
    param.requires_grad = True

model = model.to(device)
# Weighted Loss, Optimizer, Scheduler
criterion = nn.CrossEntropyLoss(weight=class_weights.to(device))
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

```

```
# Train
model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10,
                             model_name="ResNet-50 Weighted Cross Entropy")
```

↗ Training ResNet-50 Weighted Cross Entropy

=====

Epoch 1/10

train: 100%|██████████| 3729/3729 [22:01<00:00, 2.82it/s, loss=0.5206, acc=1.7014]
train Loss: 0.7107 Acc: 0.7445
val: 100%|██████████| 1252/1252 [04:59<00:00, 4.18it/s, loss=0.5084, acc=1.1948]
val Loss: 0.5443 Acc: 0.8210

Epoch 2/10

train: 100%|██████████| 3729/3729 [23:37<00:00, 2.63it/s, loss=0.3992, acc=1.8052]
train Loss: 0.6019 Acc: 0.7899
val: 100%|██████████| 1252/1252 [05:04<00:00, 4.11it/s, loss=0.7557, acc=1.1808]
val Loss: 0.5109 Acc: 0.8114

Epoch 3/10

train: 100%|██████████| 3729/3729 [24:21<00:00, 2.55it/s, loss=0.6662, acc=1.8420]
train Loss: 0.5543 Acc: 0.8060
val: 100%|██████████| 1252/1252 [04:57<00:00, 4.21it/s, loss=0.6527, acc=1.1835]
val Loss: 0.4992 Acc: 0.8139

Epoch 4/10

train: 100%|██████████| 3729/3729 [24:44<00:00, 2.51it/s, loss=0.5780, acc=1.8632]
train Loss: 0.5207 Acc: 0.8153
val: 100%|██████████| 1252/1252 [05:03<00:00, 4.12it/s, loss=0.5005, acc=1.1704]
val Loss: 0.5108 Acc: 0.8042

Epoch 5/10

train: 100%|██████████| 3729/3729 [24:51<00:00, 2.50it/s, loss=0.3546, acc=1.8814]
train Loss: 0.4956 Acc: 0.8232
val: 100%|██████████| 1252/1252 [05:07<00:00, 4.07it/s, loss=0.4645, acc=1.1920]
val Loss: 0.4948 Acc: 0.8191

Classification Report:

	precision	recall	f1-score	support
no-damage	0.97	0.84	0.90	28852
minor-damage	0.49	0.74	0.59	4715
major-damage	0.56	0.74	0.64	3840
destroyed	0.77	0.84	0.80	2647
accuracy			0.82	40054
macro avg	0.70	0.79	0.73	40054
weighted avg	0.86	0.82	0.83	40054

Epoch 6/10

train: 100%|██████████| 3729/3729 [25:30<00:00, 2.44it/s, loss=0.1797, acc=1.8979]
train Loss: 0.4702 Acc: 0.8305
val: 100%|██████████| 1252/1252 [05:05<00:00, 4.10it/s, loss=0.5663, acc=1.2084]
val Loss: 0.5029 Acc: 0.8310

Epoch 7/10

```

torch.save(model.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification Models/resnet50_ce1.pth')
with open('/content/drive/MyDrive/Building Damage Assessment/Classification Models/history_resnetce1.pkl', 'wb') as f:
    pickle.dump(history, f)

# ResNET Model with Focal Loss
model = resnet50(weights=ResNet50_Weights.DEFAULT)
for param in model.parameters():
    param.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, 4)
for param in model.layer4.parameters():
    param.requires_grad = True

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

# Weighted Focal Loss
criterion = FocalLoss(alpha=class_weights.to(device), gamma=2)
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

# Train
model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10, model_name="ResNet-50 Weighted Focal Loss")

```



Training ResNet-50 Weighted Focal Loss

=====

Epoch 1/10

```

train: 100%|██████████| 3729/3729 [29:30<00:00, 2.11it/s, loss=0.2728, acc=1.1627]
train Loss: 0.4569 Acc: 0.5087
val: 100%|██████████| 1252/1252 [05:59<00:00, 3.49it/s, loss=0.2001, acc=0.9553]
val Loss: 0.4332 Acc: 0.6564

```

Epoch 2/10

```

train: 100%|██████████| 3729/3729 [29:58<00:00, 2.07it/s, loss=0.5504, acc=1.3345]
train Loss: 0.3768 Acc: 0.5839
val: 100%|██████████| 1252/1252 [06:05<00:00, 3.42it/s, loss=0.2362, acc=0.9829]
val Loss: 0.3840 Acc: 0.6759

```

Epoch 3/10

```

train: 100%|██████████| 3729/3729 [30:43<00:00, 2.02it/s, loss=0.0815, acc=1.3927]
train Loss: 0.3441 Acc: 0.6094
val: 100%|██████████| 1252/1252 [05:47<00:00, 3.60it/s, loss=0.2241, acc=0.9464]
val Loss: 0.3723 Acc: 0.6508

```

Epoch 4/10

```

train: 100%|██████████| 3729/3729 [30:04<00:00, 2.07it/s, loss=0.1687, acc=1.4287]
train Loss: 0.3210 Acc: 0.6252
val: 100%|██████████| 1252/1252 [05:43<00:00, 3.64it/s, loss=0.1691, acc=0.9392]
val Loss: 0.3789 Acc: 0.6459

```

Epoch 5/10

```

train: 100%|██████████| 3729/3729 [29:42<00:00, 2.09it/s, loss=0.1931, acc=1.4590]
train Loss: 0.3012 Acc: 0.6384

```

```
val: 100%|██████████| 1252/1252 [05:42<00:00, 3.65it/s, loss=0.2957, acc=1.0040]
val Loss: 0.3827 Acc: 0.6904
```

```
Classification Report:
              precision    recall  f1-score   support

no-damage      0.99        0.65       0.78       28852
minor-damage    0.33        0.83       0.47        4715
major-damage    0.46        0.73       0.56        3840
destroyed       0.69        0.85       0.76        2647

 accuracy              0.69       40054
 macro avg              0.62       40054
 weighted avg           0.84       40054
```

Epoch 6/10

```
-----
train: 100%|██████████| 3729/3729 [28:37<00:00, 2.17it/s, loss=0.1464, acc=1.4929]
train Loss: 0.2834 Acc: 0.6532
val: 100%|██████████| 1252/1252 [05:39<00:00, 3.69it/s, loss=0.2591, acc=0.9782]
val Loss: 0.3762 Acc: 0.6721
```

Epoch 7/10

```
torch.save(model.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification Models/resnet50_fl1.pth')
with open('/content/drive/MyDrive/Building Damage Assessment/Classification Models/history_resnet50fl1.pkl', 'wb') as f:
    import pickle
    pickle.dump(history, f)
```

```
# ResNET with Ordinal Loss
model = models.resnet50(weights=models.ResNet50_Weights.DEFAULT)
for param in model.parameters():
    param.requires_grad = False
# Modify the last layer for CORAL (K-1 outputs)
model.fc = nn.Linear(model.fc.in_features, 3) # 4 classes - 1 = 3 outputs

for param in model.layer4.parameters():
    param.requires_grad = True

model = model.to(device)

# Use Ordinal loss Class weighted
criterion = OrdinalLoss(weight=class_weights.to(device))
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

# Train with ordinal loss
model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10,
                             model_name="ResNet-50 OL", use_ordinal=True)
```



Training ResNet-50 CORAL Loss

```
=====
Epoch 1/10
-----
train: 100%|██████████| 3729/3729 [28:49<00:00, 2.16it/s, loss=0.2974, acc=1.5172]
train Loss: 0.2940 Acc: 0.6639
val: 100%|██████████| 1252/1252 [05:12<00:00, 4.01it/s, loss=0.1722, acc=1.1204]
val Loss: 0.2601 Acc: 0.7704
```

Epoch 2/10

```

-----
train: 100%|██████████| 3729/3729 [27:45<00:00, 2.24it/s, loss=0.0537, acc=1.6899]
train Loss: 0.2420 Acc: 0.7394
val: 100%|██████████| 1252/1252 [05:06<00:00, 4.09it/s, loss=0.2154, acc=1.0929]
val Loss: 0.2504 Acc: 0.7510

```

Epoch 3/10

```

-----
train: 100%|██████████| 3729/3729 [28:58<00:00, 2.15it/s, loss=0.3166, acc=1.7384]
train Loss: 0.2238 Acc: 0.7606
val: 100%|██████████| 1252/1252 [05:30<00:00, 3.79it/s, loss=0.2089, acc=1.1172]
val Loss: 0.2403 Acc: 0.7682

```

Epoch 4/10

```

-----
train: 100%|██████████| 3729/3729 [28:42<00:00, 2.17it/s, loss=0.4269, acc=1.7688]
train Loss: 0.2094 Acc: 0.7740
val: 100%|██████████| 1252/1252 [05:21<00:00, 3.89it/s, loss=0.2007, acc=1.1414]
val Loss: 0.2370 Acc: 0.7849

```

Epoch 5/10

```

-----
train: 100%|██████████| 3729/3729 [28:25<00:00, 2.19it/s, loss=0.3456, acc=1.7980]
train Loss: 0.1967 Acc: 0.7867
val: 100%|██████████| 1252/1252 [05:25<00:00, 3.84it/s, loss=0.2103, acc=1.1469]
val Loss: 0.2549 Acc: 0.7887

```

Classification Report:

	precision	recall	f1-score	support
no-damage	0.98	0.80	0.88	28852
minor-damage	0.41	0.72	0.52	4715
major-damage	0.54	0.75	0.63	3840
destroyed	0.79	0.82	0.80	2647
accuracy			0.79	40054
macro avg	0.68	0.77	0.71	40054
weighted avg	0.86	0.79	0.81	40054

Epoch 6/10

```

-----
train: 100%|██████████| 3729/3729 [28:15<00:00, 2.20it/s, loss=0.0947, acc=1.8186]
train Loss: 0.1876 Acc: 0.7957
val: 100%|██████████| 1252/1252 [05:32<00:00, 3.77it/s, loss=0.2121, acc=1.1445]
val Loss: 0.2433 Acc: 0.7864

```

Epoch 7/10

```

torch.save(model.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification Models/resnet50_c1l1.pth')
with open('/content/drive/MyDrive/Building Damage Assessment/Classification Models/history_resnet50c1l1.pkl', 'wb') as f:
    import pickle
    pickle.dump(history, f)

```

```

# ResNet Model with Cross Entropy (No weights)
model = resnet50(weights=ResNet50_Weights.DEFAULT)
for param in model.parameters():
    param.requires_grad = False

```

```

model.fc = nn.Linear(model.fc.in_features, 4)
for param in model.layer4.parameters():
    param.requires_grad = True

model = model.to(device)
# Unweighted Loss, Optimizer, Scheduler
criterion = nn.CrossEntropyLoss() # Removed weight parameter
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

# Train
model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10,
                             model_name="ResNet-50 Unweighted Cross Entropy")

# Save
torch.save(model.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification Models/resnet50_unweighted_ce1.pth')
with open('/content/drive/MyDrive/Building Damage Assessment/Classification Models/history_resnet50unweightedce1.pkl', 'wb') as f:
    import pickle
    pickle.dump(history, f)

```

 Downloading: "<https://download.pytorch.org/models/resnet50-11ad3fa6.pth>" to /root/.cache/torch/hub/checkpoints/resnet50-11ad3fa6.pth
 100%|██████████| 97.8M/97.8M [00:00<00:00, 185MB/s]
 Training ResNet-50 Unweighted Cross Entropy
 =====

Epoch 1/10

```

-----
train: 100%|██████████| 3729/3729 [23:11<00:00, 2.68it/s, loss=0.2044, acc=1.9273]
train Loss: 0.4308 Acc: 0.8433
val: 100%|██████████| 1252/1252 [04:57<00:00, 4.21it/s, loss=0.3019, acc=1.2582]
val Loss: 0.3905 Acc: 0.8645

```

Epoch 2/10

```

-----
train: 100%|██████████| 3729/3729 [25:51<00:00, 2.40it/s, loss=0.2313, acc=1.9854]
train Loss: 0.3572 Acc: 0.8687
val: 100%|██████████| 1252/1252 [04:57<00:00, 4.21it/s, loss=0.3003, acc=1.2654]
val Loss: 0.3661 Acc: 0.8702

```

Epoch 3/10

```

-----
train: 100%|██████████| 3729/3729 [25:58<00:00, 2.39it/s, loss=0.5101, acc=2.0045]
train Loss: 0.3312 Acc: 0.8771
val: 100%|██████████| 1252/1252 [05:02<00:00, 4.14it/s, loss=0.2777, acc=1.2687]
val Loss: 0.3708 Acc: 0.8717

```

Epoch 4/10

```

-----
train: 100%|██████████| 3729/3729 [25:50<00:00, 2.41it/s, loss=0.3240, acc=2.0208]
train Loss: 0.3103 Acc: 0.8842
val: 100%|██████████| 1252/1252 [05:01<00:00, 4.15it/s, loss=0.3422, acc=1.2686]
val Loss: 0.3583 Acc: 0.8724

```

Epoch 5/10

```

-----
train: 100%|██████████| 3729/3729 [26:12<00:00, 2.37it/s, loss=0.1293, acc=2.0316]
train Loss: 0.2950 Acc: 0.8890
val: 100%|██████████| 1252/1252 [05:15<00:00, 3.97it/s, loss=0.3452, acc=1.2635]
val Loss: 0.3705 Acc: 0.8689

```

Classification Report:

	precision	recall	f1-score	support
no-damage	0.93	0.94	0.94	28852
minor-damage	0.68	0.58	0.62	4715
major-damage	0.63	0.71	0.67	3840
destroyed	0.85	0.81	0.83	2647
accuracy			0.87	40054
macro avg	0.77	0.76	0.76	40054
weighted avg	0.87	0.87	0.87	40054

Epoch 6/10

```

-----
train: 100%|██████████| 3729/3729 [26:56<00:00, 2.31it/s, loss=0.5009, acc=2.0445]
train Loss: 0.2800 Acc: 0.8946
val: 100%|██████████| 1252/1252 [05:17<00:00, 3.95it/s, loss=0.3562, acc=1.2569]
val Loss: 0.3757 Acc: 0.8643

```

ResNet Model with Focal Loss (No Weights)

model = resnet50(weights=ResNet50_Weights.DEFAULT)

for param in model.parameters():

param.requires_grad = False

model.fc = nn.Linear(model.fc.in_features, 4)

for param in model.layer4.parameters():

param.requires_grad = True

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = model.to(device)

Unweighted Focal Loss

criterion = FocalLoss(gamma=2) # Removed alpha parameter

optimizer = optim.Adam(model.parameters(), lr=0.001)

scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

Train

```

model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10,
                              model_name="ResNet-50 Unweighted Focal Loss")

```

 Training ResNet-50 Unweighted Focal Loss

```

=====
Epoch 1/10
-----
train: 100%|██████████| 3729/3729 [32:06<00:00, 1.94it/s, loss=0.1375, acc=1.9213]
train Loss: 0.1963 Acc: 0.8407
val: 100%|██████████| 1252/1252 [08:37<00:00, 2.42it/s, loss=0.1873, acc=1.2537]
val Loss: 0.1740 Acc: 0.8622

```

Epoch 2/10

```

-----
train: 100%|██████████| 3729/3729 [31:31<00:00, 1.97it/s, loss=0.1633, acc=1.9761]
train Loss: 0.1594 Acc: 0.8647
val: 100%|██████████| 1252/1252 [08:36<00:00, 2.42it/s, loss=0.1639, acc=1.2519]
val Loss: 0.1696 Acc: 0.8609

```

Epoch 3/10

```

-----
train: 100%|██████████| 3729/3729 [33:25<00:00, 1.86it/s, loss=0.1830, acc=1.9980]
train Loss: 0.1449 Acc: 0.8743
val: 100%|██████████| 1252/1252 [08:45<00:00, 2.38it/s, loss=0.1892, acc=1.2490]

```

```
val Loss: 0.1667 Acc: 0.8589
```

```
Epoch 4/10
```

```
-----
train: 100%|██████████| 3729/3729 [31:46<00:00, 1.96it/s, loss=0.0388, acc=2.0106]
train Loss: 0.1354 Acc: 0.8798
val: 100%|██████████| 1252/1252 [08:27<00:00, 2.47it/s, loss=0.1921, acc=1.2546]
val Loss: 0.1612 Acc: 0.8628
```

```
Epoch 5/10
```

```
-----
train: 100%|██████████| 3729/3729 [31:57<00:00, 1.94it/s, loss=0.1696, acc=2.0234]
train Loss: 0.1259 Acc: 0.8854
val: 100%|██████████| 1252/1252 [08:37<00:00, 2.42it/s, loss=0.1390, acc=1.2578]
val Loss: 0.1642 Acc: 0.8650
```

```
Classification Report:
```

	precision	recall	f1-score	support
no-damage	0.91	0.96	0.93	28852
minor-damage	0.69	0.53	0.60	4715
major-damage	0.70	0.60	0.65	3840
destroyed	0.82	0.82	0.82	2647
accuracy			0.86	40054
macro avg	0.78	0.73	0.75	40054
weighted avg	0.86	0.86	0.86	40054

```
Epoch 6/10
```

```
-----
train: 100%|██████████| 3729/3729 [31:53<00:00, 1.95it/s, loss=0.0282, acc=2.0349]
train Loss: 0.1185 Acc: 0.8904
val: 100%|██████████| 1252/1252 [08:36<00:00, 2.42it/s, loss=0.1553, acc=1.2505]
val Loss: 0.1776 Acc: 0.8600
```

```
Epoch 7/10
```

```
# Save
```

```
save_dir = r"C:\Users\sajja\Desktop\Dissertation\Classification Models"
os.makedirs(save_dir, exist_ok=True)
model_path = os.path.join(save_dir, "resnet50_unweighted_f11.pth")
history_path = os.path.join(save_dir, "history_resnet50unweightedf11.pkl")
```

```
# Save the model weights
```

```
torch.save(model.state_dict(), model_path)
```

```
# Save the training history
```

```
with open(history_path, "wb") as f:
    pickle.dump(history, f)
```

```
# ResNet Model with Unweighted Focal Loss (High gamma value)
```

```
model = resnet50(weights=ResNet50_Weights.DEFAULT)
for param in model.parameters():
    param.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, 4)
for param in model.layer4.parameters():
    param.requires_grad = True
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
model = model.to(device)
```

```
# Unweighted Focal Loss
```

```
criterion = FocalLoss(gamma=4) # Removed alpha parameter
```

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)
```

```
# Train
```

```
model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10, model_name="ResNet-50 Unweighted Focal Loss")
```



```
Training ResNet-50 Unweighted Focal Loss
```

```
=====
```

```
Epoch 1/10
```

```
-----
```

```
train: 100%|██████████| 3729/3729 [31:49<00:00, 1.95it/s, loss=0.0317, acc=1.9130]
```

```
train Loss: 0.1031 Acc: 0.8371
```

```
val: 100%|██████████| 1252/1252 [08:39<00:00, 2.41it/s, loss=0.0584, acc=1.2313]
```

```
val Loss: 0.0991 Acc: 0.8467
```

```
Epoch 2/10
```

```
-----
```

```
train: 100%|██████████| 3729/3729 [32:34<00:00, 1.91it/s, loss=0.0212, acc=1.9635]
```

```
train Loss: 0.0821 Acc: 0.8592
```

```
val: 100%|██████████| 1252/1252 [08:46<00:00, 2.38it/s, loss=0.0533, acc=1.2588]
```

```
val Loss: 0.0853 Acc: 0.8657
```

```
Epoch 3/10
```

```
-----
```

```
train: 100%|██████████| 3729/3729 [3:13:49<00:00, 3.12s/it, loss=0.1031, acc=1.9842]
```

```
train Loss: 0.0736 Acc: 0.8682
```

```
val: 100%|██████████| 1252/1252 [08:54<00:00, 2.34it/s, loss=0.0764, acc=1.2548]
```

```
val Loss: 0.0820 Acc: 0.8629
```

```
Epoch 4/10
```

```
-----
```

```
train: 100%|██████████| 3729/3729 [32:15<00:00, 1.93it/s, loss=0.0513, acc=1.9998]
```

```
train Loss: 0.0677 Acc: 0.8750
```

```
val: 100%|██████████| 1252/1252 [08:28<00:00, 2.46it/s, loss=0.0984, acc=1.2417]
```

```
val Loss: 0.0873 Acc: 0.8539
```

```
Epoch 5/10
```

```
-----
```

```
train: 100%|██████████| 3729/3729 [33:47<00:00, 1.84it/s, loss=0.0237, acc=2.0080]
```

```
train Loss: 0.0625 Acc: 0.8787
```

```
val: 100%|██████████| 1252/1252 [09:08<00:00, 2.28it/s, loss=0.1188, acc=1.2555]
```

```
val Loss: 0.0845 Acc: 0.8634
```

```
Classification Report:
```

	precision	recall	f1-score	support
no-damage	0.92	0.95	0.93	28852
minor-damage	0.65	0.58	0.62	4715
major-damage	0.68	0.60	0.64	3840
destroyed	0.81	0.82	0.82	2647
accuracy			0.86	40054
macro avg	0.77	0.74	0.75	40054
weighted avg	0.86	0.86	0.86	40054

```
Epoch 6/10
```

```

-----
train: 100%|██████████| 3729/3729 [34:42<00:00, 1.79it/s, loss=0.0355, acc=2.0170]
train Loss: 0.0590 Acc: 0.8826
val: 100%|██████████| 1252/1252 [10:51<00:00, 1.92it/s, loss=0.1046, acc=1.2500]
val Loss: 0.0854 Acc: 0.8596

Epoch 7/10

# Save
save_dir = r"C:\Users\sajja\Desktop\Dissertation\Classification Models"
os.makedirs(save_dir, exist_ok=True)
model_path = os.path.join(save_dir, "resnet50_unweighted_flhg.pth")
history_path = os.path.join(save_dir, "history_resnet50unweightedflhg.pkl")

# Save the model weights
torch.save(model.state_dict(), model_path)
# Save the training history
with open(history_path, "wb") as f:
    pickle.dump(history, f)

# ResNET Model with Ordinal Loss (No Weights)
model = models.resnet50(weights=models.ResNet50_Weights.DEFAULT)
for param in model.parameters():
    param.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, 3) # Change to scalar output
for param in model.layer4.parameters():
    param.requires_grad = True

model = model.to(device)
criterion = OrdinalLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

# Train
model, history = train_modelol(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10,
                                model_name="ResNet-50 Ordinal Loss (No Weights)", use_ordinal=True)

```



Training ResNet-50 Ordinal Loss (No Weights)

=====

Epoch 1/10

```

train: 100%|██████████| 3729/3729 [31:58<00:00, 1.94it/s, loss=0.1962, acc=1.7213]
train Loss: 0.2894 Acc: 0.7532
val: 100%|██████████| 1252/1252 [08:39<00:00, 2.41it/s, loss=0.3590, acc=1.0828]
val Loss: 0.2836 Acc: 0.7446

```

Epoch 2/10

```

train: 100%|██████████| 3729/3729 [31:38<00:00, 1.96it/s, loss=0.2042, acc=1.7502]
train Loss: 0.2353 Acc: 0.7658
val: 100%|██████████| 1252/1252 [08:25<00:00, 2.48it/s, loss=0.2292, acc=1.1019]
val Loss: 0.2471 Acc: 0.7577

```

Epoch 3/10

```

train: 100%|██████████| 3729/3729 [31:39<00:00, 1.96it/s, loss=0.1775, acc=1.7619]
train Loss: 0.2171 Acc: 0.7709
val: 100%|██████████| 1252/1252 [08:34<00:00, 2.43it/s, loss=0.2314, acc=1.1067]

```

```
val Loss: 0.2330 Acc: 0.7610
```

```
Epoch 4/10
```

```
-----
train: 100%|██████████| 3729/3729 [31:20<00:00, 1.98it/s, loss=0.1658, acc=1.7668]
train Loss: 0.2052 Acc: 0.7731
val: 100%|██████████| 1252/1252 [08:32<00:00, 2.44it/s, loss=0.4799, acc=1.1088]
val Loss: 0.2326 Acc: 0.7625
```

```
Epoch 5/10
```

```
-----
train: 100%|██████████| 3729/3729 [31:15<00:00, 1.99it/s, loss=0.4591, acc=1.7757]
train Loss: 0.1944 Acc: 0.7770
val: 100%|██████████| 1252/1252 [08:38<00:00, 2.42it/s, loss=0.3480, acc=1.1141]
val Loss: 0.2304 Acc: 0.7662
```

```
Classification Report:
```

	precision	recall	f1-score	support
no-damage	0.92	0.95	0.94	28852
minor-damage	0.31	0.69	0.43	4715
major-damage	0.00	0.00	0.00	3840
destroyed	0.00	0.00	0.00	2647
accuracy			0.77	40054
macro avg	0.31	0.41	0.34	40054
weighted avg	0.70	0.77	0.73	40054

```
Epoch 6/10
```

```
-----
train: 100%|██████████| 3729/3729 [31:39<00:00, 1.96it/s, loss=0.1349, acc=1.7817]
train Loss: 0.1845 Acc: 0.7796
val: 100%|██████████| 1252/1252 [08:37<00:00, 2.42it/s, loss=0.2643, acc=1.1039]
val Loss: 0.2367 Acc: 0.7591
```

```
Epoch 7/10
```

```
# Save
```

```
save_dir = r"C:\Users\sajja\Desktop\Dissertation\Classification Models"
os.makedirs(save_dir, exist_ok=True)
model_path = os.path.join(save_dir, "resnet50_unweighted_ordinal.pth")
history_path = os.path.join(save_dir, "history_resnet50unweightedordinal.pkl")
```

```
# Save the model weights
```

```
torch.save(model.state_dict(), model_path)
```

```
# Save the training history
```

```
with open(history_path, "wb") as f:
    pickle.dump(history, f)
```

✓ EfficientNET

```
from torchvision.models import efficientnet_b3, EfficientNet_B3_Weights
```

```
# EfficientNet (B3) Model with Weighted CE
```

```
model = models.efficientnet_b3(weights=models.EfficientNet_B3_Weights.DEFAULT)
```

```
for param in model.parameters():
    param.requires_grad = False
model.classifier[1] = nn.Linear(model.classifier[1].in_features, 4)
for param in model.features[7].parameters():
    param.requires_grad = True

# Weighted Loss, Optimizer, Scheduler
criterion = nn.CrossEntropyLoss(weight=class_weights.to(device))
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

# Train
model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10, model_name="EfficientNet-B3 Weighted Cross Entropy")
```

➡ Training EfficientNet-B3 Weighted Cross Entropy

=====

Epoch 1/10

train: 100%|██████████| 3729/3729 [22:59<00:00, 2.70it/s, loss=0.2201, acc=1.5948]
train Loss: 0.7890 Acc: 0.6978
val: 100%|██████████| 1252/1252 [04:58<00:00, 4.20it/s, loss=0.8150, acc=1.0359]
val Loss: 0.6769 Acc: 0.7118

Epoch 2/10

train: 100%|██████████| 3729/3729 [24:38<00:00, 2.52it/s, loss=0.8963, acc=1.7149]
train Loss: 0.6742 Acc: 0.7504
val: 100%|██████████| 1252/1252 [04:56<00:00, 4.22it/s, loss=0.6717, acc=1.1066]
val Loss: 0.6069 Acc: 0.7610

Epoch 3/10

train: 100%|██████████| 3729/3729 [25:53<00:00, 2.40it/s, loss=0.9277, acc=1.7521]
train Loss: 0.6326 Acc: 0.7667
val: 100%|██████████| 1252/1252 [05:00<00:00, 4.17it/s, loss=0.5004, acc=1.1549]
val Loss: 0.5651 Acc: 0.7935

Epoch 4/10

train: 100%|██████████| 3729/3729 [26:11<00:00, 2.37it/s, loss=0.4291, acc=1.7768]
train Loss: 0.6015 Acc: 0.7775
val: 100%|██████████| 1252/1252 [04:54<00:00, 4.25it/s, loss=0.5632, acc=1.1583]
val Loss: 0.5501 Acc: 0.7965

Epoch 5/10

train: 100%|██████████| 3729/3729 [26:43<00:00, 2.33it/s, loss=0.2677, acc=1.7953]
train Loss: 0.5822 Acc: 0.7856
val: 100%|██████████| 1252/1252 [04:55<00:00, 4.23it/s, loss=0.8053, acc=1.1184]
val Loss: 0.5965 Acc: 0.7691

Classification Report:

	precision	recall	f1-score	support
no-damage	0.98	0.77	0.86	28852
minor-damage	0.42	0.71	0.53	4715
major-damage	0.48	0.78	0.59	3840
destroyed	0.72	0.82	0.77	2647
accuracy			0.77	40054

```

    macro avg      0.65      0.77      0.69      40054
    weighted avg    0.85      0.77      0.79      40054

```

```
Epoch 6/10
```

```

-----
train: 100%|██████████| 3729/3729 [26:32<00:00, 2.34it/s, loss=0.4295, acc=1.8082]
train Loss: 0.5640 Acc: 0.7912
val: 100%|██████████| 1252/1252 [04:53<00:00, 4.26it/s, loss=0.9032, acc=1.1566]
val Loss: 0.5450 Acc: 0.7948

```

```
Epoch 7/10
```

```

# Save the EfficientNet-B3 model
torch.save(model.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification Models/efficientnet_b3_ce1.pth')
with open('/content/drive/MyDrive/Building Damage Assessment/Classification Models/history_efficientnetb3ce1.pkl', 'wb') as f:
    import pickle
    pickle.dump(history, f)

```

```

# EfficientNet-B3 Model with Weighted Focal Loss
model = efficientnet_b3(weights=EfficientNet_B3_Weights.DEFAULT)
for param in model.parameters():
    param.requires_grad = False
model.classifier[1] = nn.Linear(model.classifier[1].in_features, 4)
for param in model.features[7].parameters():
    param.requires_grad = True

```

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

```

```

# Weighted Focal Loss
criterion = FocalLoss(alpha=class_weights.to(device), gamma=2)
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

```

```
model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10, model_name="EfficientNet-B3 Weighted Focal Loss")
```

📄 Downloading: "https://download.pytorch.org/models/efficientnet_b3_rwightman-b3899882.pth" to /root/.cache/torch/hub/checkpoints/efficientnet_b3_rwightman-b3899882.pth

```
100%|██████████| 47.2M/47.2M [00:00<00:00, 134MB/s]
```

```
Training EfficientNet-B3 Weighted Focal Loss
```

```
=====
```

```
Epoch 1/10
```

```

-----
train: 100%|██████████| 3729/3729 [29:51<00:00, 2.08it/s, loss=0.0778, acc=1.0222]
train Loss: 0.5061 Acc: 0.4473
val: 100%|██████████| 1252/1252 [05:37<00:00, 3.71it/s, loss=0.2548, acc=0.8567]
val Loss: 0.4808 Acc: 0.5886

```

```
Epoch 2/10
```

```

-----
train: 100%|██████████| 3729/3729 [30:00<00:00, 2.07it/s, loss=0.2450, acc=1.2247]
train Loss: 0.4293 Acc: 0.5359
val: 100%|██████████| 1252/1252 [05:44<00:00, 3.63it/s, loss=0.2180, acc=0.7775]
val Loss: 0.4313 Acc: 0.5342

```

```
Epoch 3/10
```

```

-----
train: 100%|██████████| 3729/3729 [30:48<00:00, 2.02it/s, loss=0.6515, acc=1.2801]
train Loss: 0.3982 Acc: 0.5601

```

```
val: 100%|██████████| 1252/1252 [05:37<00:00, 3.71it/s, loss=0.1907, acc=0.8789]
val Loss: 0.4499 Acc: 0.6044
```

Epoch 4/10

```
-----
train: 100%|██████████| 3729/3729 [31:03<00:00, 2.00it/s, loss=0.0848, acc=1.3101]
train Loss: 0.3774 Acc: 0.5733
val: 100%|██████████| 1252/1252 [05:40<00:00, 3.68it/s, loss=0.1897, acc=0.9636]
val Loss: 0.4074 Acc: 0.6621
```

Epoch 5/10

```
-----
train: 100%|██████████| 3729/3729 [30:46<00:00, 2.02it/s, loss=0.3482, acc=1.3541]
train Loss: 0.3613 Acc: 0.5925
val: 100%|██████████| 1252/1252 [05:46<00:00, 3.61it/s, loss=0.2774, acc=0.8744]
val Loss: 0.4314 Acc: 0.6008
```

Classification Report:

	precision	recall	f1-score	support
no-damage	0.99	0.54	0.70	28852
minor-damage	0.28	0.74	0.40	4715
major-damage	0.43	0.72	0.54	3840
destroyed	0.44	0.89	0.59	2647
accuracy			0.60	40054
macro avg	0.53	0.72	0.56	40054
weighted avg	0.82	0.60	0.64	40054

Epoch 6/10

```
-----
train: 100%|██████████| 3729/3729 [30:50<00:00, 2.02it/s, loss=0.6459, acc=1.3684]
train Loss: 0.3481 Acc: 0.5988
val: 100%|██████████| 1252/1252 [05:38<00:00, 3.70it/s, loss=0.2282, acc=0.9124]
val Loss: 0.4398 Acc: 0.6269
```

```
torch.save(model.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification Models/efficientnet_b3_fl1.pth')
with open('/content/drive/MyDrive/Building Damage Assessment/Classification Models/history_efficientnetb3fl1.pkl', 'wb') as f:
    import pickle
    pickle.dump(history, f)
```

EfficientNet-B3 Model with Weighted Ordinal Loss

```
model = efficientnet_b3(weights=EfficientNet_B3_Weights.DEFAULT)
```

```
for param in model.parameters():
```

```
    param.requires_grad = False
```

```
model.classifier[1] = nn.Linear(model.classifier[1].in_features, 3)
```

```
for param in model.features[7].parameters():
```

```
    param.requires_grad = True
```

```
criterion = OrdinalLoss(weight=class_weights.to(device))
```

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)
```

```
model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10, model_name="EfficientNet-B3 OL", use_ordinal=True)
```



Downloading: "https://download.pytorch.org/models/efficientnet_b3_rwightman-b3899882.pth" to /root/.cache/torch/hub/checkpoints/efficientnet_b3_rwightman-b3899882.pth

```
100%|██████████| 47.2M/47.2M [00:00<00:00, 171MB/s]
```

```
Training EfficientNet-B3 CORAL
```



```
=====
Epoch 1/10
-----
train: 100%|██████████| 3729/3729 [23:20<00:00, 2.66it/s, loss=0.1600, acc=1.3906]
train Loss: 0.3299 Acc: 0.6085
val: 100%|██████████| 1252/1252 [05:00<00:00, 4.17it/s, loss=0.1702, acc=1.0411]
val Loss: 0.3044 Acc: 0.7159
```

```
Epoch 2/10
-----
train: 100%|██████████| 3729/3729 [24:40<00:00, 2.52it/s, loss=0.3162, acc=1.5860]
train Loss: 0.2727 Acc: 0.6940
val: 100%|██████████| 1252/1252 [05:03<00:00, 4.13it/s, loss=0.2153, acc=1.0059]
val Loss: 0.2647 Acc: 0.6917
```

```
Epoch 3/10
-----
train: 100%|██████████| 3729/3729 [26:40<00:00, 2.33it/s, loss=0.4115, acc=1.6385]
train Loss: 0.2552 Acc: 0.7169
val: 100%|██████████| 1252/1252 [05:13<00:00, 3.99it/s, loss=0.1302, acc=1.0940]
val Loss: 0.2516 Acc: 0.7517
```

```
Epoch 4/10
-----
train: 100%|██████████| 3729/3729 [26:16<00:00, 2.37it/s, loss=0.2021, acc=1.6688]
train Loss: 0.2429 Acc: 0.7302
val: 100%|██████████| 1252/1252 [05:04<00:00, 4.11it/s, loss=0.1535, acc=1.0977]
val Loss: 0.2736 Acc: 0.7543
```

```
Epoch 5/10
-----
train: 100%|██████████| 3729/3729 [26:15<00:00, 2.37it/s, loss=0.1297, acc=1.6964]
train Loss: 0.2325 Acc: 0.7423
val: 100%|██████████| 1252/1252 [05:10<00:00, 4.03it/s, loss=0.1566, acc=1.1166]
val Loss: 0.2487 Acc: 0.7678
```

Classification Report:

	precision	recall	f1-score	support
no-damage	0.98	0.77	0.86	28852
minor-damage	0.41	0.77	0.53	4715
major-damage	0.52	0.70	0.60	3840
destroyed	0.69	0.85	0.76	2647
accuracy			0.77	40054
macro avg	0.65	0.77	0.69	40054
weighted avg	0.85	0.77	0.79	40054

```
Epoch 6/10
-----
train: 100%|██████████| 3729/3729 [26:05<00:00, 2.38it/s, loss=0.1508, acc=1.7187]
train Loss: 0.2255 Acc: 0.7520
val: 100%|██████████| 1252/1252 [05:09<00:00, 4.05it/s, loss=0.1174, acc=1.1056]
val Loss: 0.2669 Acc: 0.7506
```

```
torch.save(model.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification Models/efficientnet_b3_coral.pth')
with open('/content/drive/MyDrive/Building Damage Assessment/Classification Models/history_efficientnetb3coral.pkl', 'wb') as f:
    import pickle
    pickle.dump(history, f)
```

```
# Efficient Net with unweighted Cross Entropy
model = efficientnet_b3(weights=EfficientNet_B3_Weights.DEFAULT)
for param in model.parameters():
    param.requires_grad = False
model.classifier[1] = nn.Linear(model.classifier[1].in_features, 4)
for param in model.features[7].parameters():
    param.requires_grad = True

# Unweighted Loss, Optimizer, Scheduler
criterion = nn.CrossEntropyLoss() # Removed weight parameter
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

# Train
model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10,
                             model_name="EfficientNet-B3 Cross Entropy (No Weights)")

# Save
save_dir = r"C:\Users\sajja\Desktop\Dissertation\Classification Models"
os.makedirs(save_dir, exist_ok=True)
model_path = os.path.join(save_dir, "efficientnet_unweighted_ce.pth")
history_path = os.path.join(save_dir, "history_efficientnetunweightedce.pkl")

# Save the model weights
torch.save(model.state_dict(), model_path)
# Save the training history
with open(history_path, "wb") as f:
    pickle.dump(history, f)
```



Training EfficientNet-B3 Cross Entropy (No Weights)

=====

Epoch 1/10

train: 100%|██████████| 3729/3729 [30:00<00:00, 2.07it/s, loss=0.5171, acc=1.8819]

train Loss: 0.4854 Acc: 0.8235

val: 100%|██████████| 1252/1252 [08:29<00:00, 2.46it/s, loss=0.2804, acc=1.2243]

val Loss: 0.4742 Acc: 0.8419

Epoch 2/10

train: 100%|██████████| 3729/3729 [30:46<00:00, 2.02it/s, loss=0.4527, acc=1.9420]

train Loss: 0.4074 Acc: 0.8497

val: 100%|██████████| 1252/1252 [08:25<00:00, 2.48it/s, loss=0.1914, acc=1.2466]

val Loss: 0.4086 Acc: 0.8572

Epoch 3/10

train: 100%|██████████| 3729/3729 [30:37<00:00, 2.03it/s, loss=0.5086, acc=1.9596]

train Loss: 0.3804 Acc: 0.8574

val: 100%|██████████| 1252/1252 [08:19<00:00, 2.51it/s, loss=0.2757, acc=1.2467]

val Loss: 0.3914 Acc: 0.8573

Epoch 4/10

train: 100%|██████████| 3729/3729 [30:56<00:00, 2.01it/s, loss=0.1766, acc=1.9734]

train Loss: 0.3634 Acc: 0.8635

val: 100%|██████████| 1252/1252 [08:30<00:00, 2.45it/s, loss=0.2206, acc=1.2541]

val Loss: 0.3744 Acc: 0.8624

Epoch 5/10

```

train: 100%|██████████| 3729/3729 [30:16<00:00, 2.05it/s, loss=0.3262, acc=1.9851]
train Loss: 0.3497 Acc: 0.8686
val: 100%|██████████| 1252/1252 [08:26<00:00, 2.47it/s, loss=0.2764, acc=1.2512]
val Loss: 0.3858 Acc: 0.8604

```

Classification Report:

	precision	recall	f1-score	support
no-damage	0.93	0.94	0.93	28852
minor-damage	0.66	0.55	0.60	4715
major-damage	0.60	0.72	0.66	3840
destroyed	0.86	0.79	0.83	2647
accuracy			0.86	40054
macro avg	0.76	0.75	0.76	40054
weighted avg	0.86	0.86	0.86	40054

Epoch 6/10

```

train: 100%|██████████| 3729/3729 [30:01<00:00, 2.07it/s, loss=0.5470, acc=1.9927]
train Loss: 0.3400 Acc: 0.8719
val: 100%|██████████| 1252/1252 [08:19<00:00, 2.50it/s, loss=0.3071, acc=1.2484]
val Loss: 0.3839 Acc: 0.8585

```

Epoch 7/10

EfficientNet-B3 Model with UnWeighted Focal Loss

model = efficientnet_b3(weights=EfficientNet_B3_Weights.DEFAULT)

for param in model.parameters():

param.requires_grad = False

model.classifier[1] = nn.Linear(model.classifier[1].in_features, 4)

for param in model.features[7].parameters():

param.requires_grad = True

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = model.to(device)

UnWeighted Focal Loss

criterion = FocalLoss(gamma=2)

optimizer = optim.Adam(model.parameters(), lr=0.001)

scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10, model_name="EfficientNet-B3 UnWeighted Focal Loss")

torch.save(model.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification Models/efficientnet_b3_unweightedf11.pth')

with open('/content/drive/MyDrive/Building Damage Assessment/Classification Models/history_efficientnetb3unweightedf11.pkl', 'wb') as f:

import pickle

pickle.dump(history, f)

Downloading: "https://download.pytorch.org/models/efficientnet_b3_rwightman-b3899882.pth" to /root/.cache/torch/hub/checkpoints/efficientnet_b3_rwightman-b3899882.pth

100%|██████████| 47.2M/47.2M [00:00<00:00, 116MB/s]

Training EfficientNet-B3 UnWeighted Focal Loss

=====

Epoch 1/10

```

train: 100%|██████████| 3729/3729 [24:28<00:00, 2.54it/s, loss=0.0490, acc=1.8693]

```

```

train Loss: 0.2284 Acc: 0.8180
val: 100%|██████████| 1252/1252 [05:28<00:00, 3.81it/s, loss=0.1348, acc=1.2227]
val Loss: 0.2011 Acc: 0.8402

```

Epoch 2/10

```

-----
train: 100%|██████████| 3729/3729 [25:39<00:00, 2.42it/s, loss=0.2303, acc=1.9245]
train Loss: 0.1894 Acc: 0.8421
val: 100%|██████████| 1252/1252 [05:37<00:00, 3.71it/s, loss=0.1779, acc=1.2305]
val Loss: 0.2017 Acc: 0.8455

```

Epoch 3/10

```

-----
train: 100%|██████████| 3729/3729 [26:38<00:00, 2.33it/s, loss=0.2681, acc=1.9437]
train Loss: 0.1756 Acc: 0.8505
val: 100%|██████████| 1252/1252 [05:35<00:00, 3.73it/s, loss=0.1324, acc=1.2346]
val Loss: 0.1832 Acc: 0.8490

```

Epoch 4/10

```

-----
train: 100%|██████████| 3729/3729 [26:11<00:00, 2.37it/s, loss=0.0660, acc=1.9573]
train Loss: 0.1657 Acc: 0.8564
val: 100%|██████████| 1252/1252 [05:28<00:00, 3.81it/s, loss=0.1846, acc=1.2349]
val Loss: 0.1799 Acc: 0.8492

```

Epoch 5/10

```

-----
train: 100%|██████████| 3729/3729 [26:24<00:00, 2.35it/s, loss=0.1114, acc=1.9702]
train Loss: 0.1580 Acc: 0.8621
val: 100%|██████████| 1252/1252 [05:32<00:00, 3.76it/s, loss=0.2054, acc=1.2428]
val Loss: 0.1684 Acc: 0.8539

```

Classification Report:

	precision	recall	f1-score	support
no-damage	0.90	0.96	0.93	28852
minor-damage	0.70	0.44	0.54	4715
major-damage	0.63	0.63	0.63	3840
destroyed	0.88	0.77	0.82	2647
accuracy			0.85	40054
macro avg	0.78	0.70	0.73	40054
weighted avg	0.85	0.85	0.85	40054

Epoch 6/10

```

-----
train: 100%|██████████| 3729/3729 [26:26<00:00, 2.35it/s, loss=0.3110, acc=1.9767]
train Loss: 0.1525 Acc: 0.8649
val: 100%|██████████| 1252/1252 [05:46<00:00, 3.62it/s, loss=0.1112, acc=1.2379]

```

```
# EfficientNet-B3 Model with UnWeighted Focal Loss and High Gamma
```

```
model = efficientnet_b3(weights=EfficientNet_B3_Weights.DEFAULT)
```

```
for param in model.parameters():
```

```
    param.requires_grad = False
```

```
model.classifier[1] = nn.Linear(model.classifier[1].in_features, 4)
```

```
for param in model.features[7].parameters():
```

```
    param.requires_grad = True
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
model = model.to(device)

# UnWeighted Focal Loss
criterion = FocalLoss(gamma=4)
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10, model_name="EfficientNet-B3 UnWeighted Focal Loss (High Gamma)")
```

↩ Training EfficientNet-B3 UnWeighted Focal Loss (High Gamma)

```
=====
Epoch 1/10
-----
train: 100%|██████████| 3729/3729 [26:06<00:00, 2.38it/s, loss=0.0796, acc=1.8522]
train Loss: 0.1229 Acc: 0.8105
val: 100%|██████████| 1252/1252 [05:27<00:00, 3.82it/s, loss=0.0793, acc=1.1982]
val Loss: 0.1123 Acc: 0.8233

Epoch 2/10
-----
train: 100%|██████████| 3729/3729 [25:49<00:00, 2.41it/s, loss=0.0561, acc=1.9072]
train Loss: 0.0999 Acc: 0.8345
val: 100%|██████████| 1252/1252 [05:28<00:00, 3.82it/s, loss=0.0524, acc=1.2277]
val Loss: 0.0920 Acc: 0.8442

Epoch 3/10
-----
train: 100%|██████████| 3729/3729 [26:48<00:00, 2.32it/s, loss=0.0339, acc=1.9264]
train Loss: 0.0909 Acc: 0.8429
val: 100%|██████████| 1252/1252 [05:42<00:00, 3.65it/s, loss=0.0754, acc=1.2396]
val Loss: 0.0909 Acc: 0.8517

Epoch 4/10
-----
train: 100%|██████████| 3729/3729 [27:22<00:00, 2.27it/s, loss=0.0981, acc=1.9424]
train Loss: 0.0851 Acc: 0.8499
val: 100%|██████████| 1252/1252 [05:32<00:00, 3.77it/s, loss=0.0724, acc=1.2251]
val Loss: 0.1032 Acc: 0.8418

Epoch 5/10
-----
train: 100%|██████████| 3729/3729 [27:37<00:00, 2.25it/s, loss=0.1007, acc=1.9516]
train Loss: 0.0812 Acc: 0.8540
val: 100%|██████████| 1252/1252 [05:36<00:00, 3.72it/s, loss=0.0671, acc=1.2386]
val Loss: 0.1334 Acc: 0.8511
```

Classification Report:

	precision	recall	f1-score	support
no-damage	0.89	0.96	0.92	28852
minor-damage	0.70	0.43	0.53	4715
major-damage	0.64	0.59	0.62	3840
destroyed	0.89	0.75	0.81	2647
accuracy			0.85	40054
macro avg	0.78	0.68	0.72	40054
weighted avg	0.84	0.85	0.84	40054

Epoch 6/10

```

train: 100%|██████████| 3729/3729 [27:00<00:00, 2.30it/s, loss=0.0789, acc=1.9615]
train Loss: 0.0768 Acc: 0.8583
val: 100%|██████████| 1252/1252 [05:30<00:00, 3.79it/s, loss=0.0943, acc=1.2286]
val Loss: 0.0910 Acc: 0.8442

```

Epoch 7/10

```

torch.save(model.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification Models/efficientnet_b3_unweightedfl1_hg1.pth')
with open('/content/drive/MyDrive/Building Damage Assessment/Classification Models/history_efficientnetb3unweightedfl1_hg1.pkl', 'wb') as f:
    import pickle
    pickle.dump(history, f)

```

```

# EfficientNet-B3 Model with Ordinal Loss (No Weights)
model = efficientnet_b3(weights=EfficientNet_B3_Weights.DEFAULT)
for param in model.parameters():
    param.requires_grad = False
model.classifier[1] = nn.Linear(model.classifier[1].in_features, 3)
for param in model.features[7].parameters():
    param.requires_grad = True

```

```

criterion = OrdinalLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

```

```

model, history = train_modelol(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10,
                                model_name="EfficientNet-B3 OL No Weights", use_ordinal=True)

```



Training EfficientNet-B3 OL No Weights

=====

Epoch 1/10

```

train: 100%|██████████| 3729/3729 [31:10<00:00, 1.99it/s, loss=0.1702, acc=1.8358]
train Loss: 0.2249 Acc: 0.8033
val: 100%|██████████| 1252/1252 [08:38<00:00, 2.41it/s, loss=0.1466, acc=1.2054]
val Loss: 0.2319 Acc: 0.8289

```

Epoch 2/10

```

train: 100%|██████████| 3729/3729 [31:12<00:00, 1.99it/s, loss=0.3791, acc=1.9095]
train Loss: 0.1823 Acc: 0.8355
val: 100%|██████████| 1252/1252 [08:40<00:00, 2.40it/s, loss=0.1555, acc=1.2261]
val Loss: 0.1727 Acc: 0.8431

```

Epoch 3/10

```

train: 100%|██████████| 3729/3729 [31:23<00:00, 1.98it/s, loss=0.3445, acc=1.9332]
train Loss: 0.1693 Acc: 0.8459
val: 100%|██████████| 1252/1252 [08:38<00:00, 2.42it/s, loss=0.0928, acc=1.2337]
val Loss: 0.1655 Acc: 0.8484

```

Epoch 4/10

```

train: 100%|██████████| 3729/3729 [31:28<00:00, 1.97it/s, loss=0.0876, acc=1.9495]
train Loss: 0.1592 Acc: 0.8530
val: 100%|██████████| 1252/1252 [08:36<00:00, 2.42it/s, loss=0.1764, acc=1.2330]
val Loss: 0.1720 Acc: 0.8479

```

Epoch 5/10

```

train: 100%|██████████| 3729/3729 [31:34<00:00, 1.97it/s, loss=0.1032, acc=1.9574]
train Loss: 0.1538 Acc: 0.8565
val: 100%|██████████| 1252/1252 [08:42<00:00, 2.39it/s, loss=0.1072, acc=1.2388]
val Loss: 0.1843 Acc: 0.8519

```

Classification Report:

	precision	recall	f1-score	support
no-damage	0.92	0.93	0.93	28852
minor-damage	0.59	0.55	0.57	4715
major-damage	0.61	0.66	0.64	3840
destroyed	0.87	0.78	0.82	2647
accuracy			0.85	40054
macro avg	0.75	0.73	0.74	40054
weighted avg	0.85	0.85	0.85	40054

Epoch 6/10

```

-----
train: 100%|██████████| 3729/3729 [31:20<00:00, 1.98it/s, loss=0.0141, acc=1.9661]
train Loss: 0.1483 Acc: 0.8603
val: 100%|██████████| 1252/1252 [08:38<00:00, 2.42it/s, loss=0.1202, acc=1.2375]
val Loss: 0.1638 Acc: 0.8510

```

Epoch 7/10

Save

```

save_dir = r"C:\Users\sajja\Desktop\Dissertation\Classification Models"
os.makedirs(save_dir, exist_ok=True)
model_path = os.path.join(save_dir, "efficientnet_unweighted_01.pth")
history_path = os.path.join(save_dir, "history_efficientnetunweighted01.pkl")

```

Save the model weights

```
torch.save(model.state_dict(), model_path)
```

Save the training history

```

with open(history_path, "wb") as f:
    pickle.dump(history, f)

```

✓ ViT-B/16

```
from torchvision.models import vit_b_16, ViT_B_16_Weights
```

ViT-B/16 Model with Weighted CE

```

model = vit_b_16(weights=ViT_B_16_Weights.IMAGENET1K_V1)
for param in model.parameters():
    param.requires_grad = False
model.heads.head = nn.Linear(model.heads.head.in_features, 4) # Replace classification head for 4 classes
for param in model.encoder.layers[-1].parameters(): # Unfreeze last transformer layer
    param.requires_grad = True

```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
model = model.to(device)
```

Weighted Loss, Optimizer, Scheduler

```

criterion = nn.CrossEntropyLoss(weight=class_weights.to(device))
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

# Train
model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10, model_name="ViT-B/16 Weighted Cross Entropy")

```

Downloading: "https://download.pytorch.org/models/vit_b_16-c867db91.pth" to /root/.cache/torch/hub/checkpoints/vit_b_16-c867db91.pth

100%|██████████| 330M/330M [00:08<00:00, 40.7MB/s]
Training ViT-B/16 Weighted Cross Entropy
=====

Epoch 1/10				

train: 100%	██████████	3729/3729	[31:43<00:00, 1.96it/s, loss=0.6523, acc=1.6798]	
train Loss: 0.7058 Acc: 0.7350				
val: 100%	██████████	1252/1252	[08:18<00:00, 2.51it/s, loss=0.9170, acc=1.1650]	
val Loss: 0.5391 Acc: 0.8011				

Epoch 2/10

train: 100%	██████████	3729/3729	[32:12<00:00, 1.93it/s, loss=0.5937, acc=1.7834]	
train Loss: 0.5951 Acc: 0.7804				
val: 100%	██████████	1252/1252	[08:34<00:00, 2.43it/s, loss=0.9838, acc=1.1037]	
val Loss: 0.5817 Acc: 0.7590				

Epoch 3/10

train: 100%	██████████	3729/3729	[32:22<00:00, 1.92it/s, loss=0.6153, acc=1.8126]	
train Loss: 0.5550 Acc: 0.7931				
val: 100%	██████████	1252/1252	[08:34<00:00, 2.43it/s, loss=0.9343, acc=1.1632]	
val Loss: 0.5345 Acc: 0.7999				

Epoch 4/10

train: 100%	██████████	3729/3729	[32:08<00:00, 1.93it/s, loss=0.7211, acc=1.8388]	
train Loss: 0.5185 Acc: 0.8046				
val: 100%	██████████	1252/1252	[08:16<00:00, 2.52it/s, loss=0.8591, acc=1.1310]	
val Loss: 0.5756 Acc: 0.7778				

Epoch 5/10

train: 100%	██████████	3729/3729	[31:27<00:00, 1.98it/s, loss=0.4514, acc=1.8564]	
train Loss: 0.4895 Acc: 0.8123				
val: 100%	██████████	1252/1252	[08:15<00:00, 2.53it/s, loss=0.9637, acc=1.1231]	
val Loss: 0.5928 Acc: 0.7723				

Classification Report:

	precision	recall	f1-score	support
no-damage	0.97	0.78	0.87	28852
minor-damage	0.44	0.75	0.55	4715
major-damage	0.45	0.76	0.56	3840
destroyed	0.83	0.77	0.80	2647
accuracy			0.77	40054
macro avg	0.67	0.76	0.70	40054
weighted avg	0.85	0.77	0.80	40054

Epoch 6/10

```

train: 100%|██████████| 3729/3729 [31:53<00:00, 1.95it/s, loss=0.1686, acc=1.8733]
train Loss: 0.4613 Acc: 0.8197
val: 100%|██████████| 1252/1252 [08:29<00:00, 2.46it/s, loss=1.2242, acc=1.1827]
val Loss: 0.5487 Acc: 0.8133

# Save the ViT-B/16 model
torch.save(model.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification Models/vit_b16_ce1.pth')
with open('/content/drive/MyDrive/Building Damage Assessment/Classification Models/history_vitb16ce1.pkl', 'wb') as f:
    import pickle
    pickle.dump(history, f)

# ViT-B/16 Model with Focal Loss
from torchvision.models import vit_b_16, ViT_B_16_Weights
model = vit_b_16(weights=ViT_B_16_Weights.IMAGENET1K_V1)
for param in model.parameters():
    param.requires_grad = False
model.heads.head = nn.Linear(model.heads.head.in_features, 4)
for param in model.encoder.layers[-1].parameters():
    param.requires_grad = True

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

# Weighted Focal Loss
criterion = FocalLoss(alpha=class_weights.to(device), gamma=2)
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10, model_name="ViT-B/16 Weighted Focal Loss")

📄 Downloading: "https://download.pytorch.org/models/vit_b_16-c867db91.pth" to /root/.cache/torch/hub/checkpoints/vit_b_16-c867db91.pth
100%|██████████| 330M/330M [00:02<00:00, 140MB/s]
Training ViT-B/16 Weighted Focal Loss
=====
Epoch 1/10
-----
train: 100%|██████████| 3729/3729 [36:38<00:00, 1.70it/s, loss=0.0383, acc=1.1829]
train Loss: 0.4505 Acc: 0.5176
val: 100%|██████████| 1252/1252 [09:16<00:00, 2.25it/s, loss=0.2979, acc=0.8606]
val Loss: 0.4203 Acc: 0.5918

Epoch 2/10
-----
train: 100%|██████████| 3729/3729 [36:59<00:00, 1.68it/s, loss=0.1105, acc=1.3271]
train Loss: 0.3691 Acc: 0.5807
val: 100%|██████████| 1252/1252 [09:15<00:00, 2.25it/s, loss=0.3154, acc=0.8717]
val Loss: 0.4222 Acc: 0.5994

Epoch 3/10
-----
train: 100%|██████████| 3729/3729 [36:33<00:00, 1.70it/s, loss=1.1181, acc=1.3808]
train Loss: 0.3373 Acc: 0.6042
val: 100%|██████████| 1252/1252 [09:17<00:00, 2.25it/s, loss=0.1040, acc=0.9726]
val Loss: 0.4200 Acc: 0.6688

Epoch 4/10
-----
train: 100%|██████████| 3729/3729 [36:54<00:00, 1.68it/s, loss=0.6052, acc=1.4153]
train Loss: 0.3121 Acc: 0.6193

```

```
val: 100%|██████████| 1252/1252 [09:18<00:00, 2.24it/s, loss=0.2696, acc=0.8077]
val Loss: 0.4090 Acc: 0.5554
```

Epoch 5/10

```
-----
train: 100%|██████████| 3729/3729 [36:48<00:00, 1.69it/s, loss=0.6422, acc=1.4288]
train Loss: 0.2897 Acc: 0.6252
val: 100%|██████████| 1252/1252 [09:17<00:00, 2.25it/s, loss=0.2887, acc=0.9455]
val Loss: 0.4169 Acc: 0.6502
```

Classification Report:

	precision	recall	f1-score	support
no-damage	0.99	0.60	0.75	28852
minor-damage	0.29	0.81	0.43	4715
major-damage	0.46	0.73	0.56	3840
destroyed	0.67	0.81	0.74	2647
accuracy			0.65	40054
macro avg	0.60	0.74	0.62	40054
weighted avg	0.83	0.65	0.69	40054

Epoch 6/10

```
-----
train: 100%|██████████| 3729/3729 [37:17<00:00, 1.67it/s, loss=0.1826, acc=1.4347]
train Loss: 0.2740 Acc: 0.6278
val: 100%|██████████| 1252/1252 [09:20<00:00, 2.23it/s, loss=0.3648, acc=0.9213]
val Loss: 0.4363 Acc: 0.6335
```

```
torch.save(model.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification Models/vit_b16_f11.pth')
with open('/content/drive/MyDrive/Building Damage Assessment/Classification Models/history_vitb16f11.pkl', 'wb') as f:
    import pickle
    pickle.dump(history, f)
```

ViT with Ordinal Loss

```
model = vit_b_16(weights=ViT_B_16_Weights.IMAGENET1K_V1)
```

```
for param in model.parameters():
    param.requires_grad = False
model.heads.head = nn.Linear(model.heads.head.in_features, 3)
for param in model.encoder.layers[-1].parameters():
    param.requires_grad = True
model = model.to(device)
```

```
criterion = OrdinalLoss(weight=class_weights.to(device))
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)
```

```
model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10,
                             model_name="ViT-B/16 OL", use_ordinal=True)
```



Training ViT-B/16 OL

```
=====
Epoch 1/10
```

```
-----
train: 100%|██████████| 3729/3729 [32:00<00:00, 1.94it/s, loss=0.1217, acc=1.5282]
train Loss: 0.2855 Acc: 0.6687
```

```
val: 100%|██████████| 1252/1252 [08:47<00:00, 2.37it/s, loss=0.2061, acc=1.0500]
val Loss: 0.2636 Acc: 0.7220
```

Epoch 2/10

```
-----
train: 100%|██████████| 3729/3729 [32:46<00:00, 1.90it/s, loss=0.1213, acc=1.6979]
train Loss: 0.2359 Acc: 0.7430
val: 100%|██████████| 1252/1252 [08:43<00:00, 2.39it/s, loss=0.1626, acc=1.0313]
val Loss: 0.2517 Acc: 0.7092
```

Epoch 3/10

```
-----
train: 100%|██████████| 3729/3729 [33:25<00:00, 1.86it/s, loss=0.3256, acc=1.7439]
train Loss: 0.2163 Acc: 0.7631
val: 100%|██████████| 1252/1252 [08:53<00:00, 2.35it/s, loss=0.2872, acc=1.1565]
val Loss: 0.2568 Acc: 0.7953
```

Epoch 4/10

```
-----
train: 100%|██████████| 3729/3729 [32:58<00:00, 1.88it/s, loss=0.6350, acc=1.7803]
train Loss: 0.2017 Acc: 0.7790
val: 100%|██████████| 1252/1252 [08:51<00:00, 2.35it/s, loss=0.2505, acc=1.0725]
val Loss: 0.2582 Acc: 0.7375
```

Epoch 5/10

```
-----
train: 100%|██████████| 3729/3729 [33:06<00:00, 1.88it/s, loss=0.1257, acc=1.7959]
train Loss: 0.1918 Acc: 0.7858
val: 100%|██████████| 1252/1252 [08:56<00:00, 2.33it/s, loss=0.1841, acc=1.1311]
val Loss: 0.2555 Acc: 0.7778
```

Classification Report:

	precision	recall	f1-score	support
no-damage	0.97	0.79	0.87	28852
minor-damage	0.40	0.75	0.52	4715
major-damage	0.53	0.70	0.60	3840
destroyed	0.75	0.81	0.78	2647
accuracy			0.78	40054
macro avg	0.66	0.76	0.69	40054
weighted avg	0.85	0.78	0.80	40054

Epoch 6/10

```
-----
train: 100%|██████████| 3729/3729 [33:40<00:00, 1.85it/s, loss=0.1574, acc=1.8626]
train Loss: 0.1614 Acc: 0.8150
val: 100%|██████████| 1252/1252 [08:50<00:00, 2.36it/s, loss=0.1906, acc=1.1364]
val Loss: 0.2576 Acc: 0.7815
```

```
torch.save(model.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification Models/vit_b16_ordinal.pth')
with open('/content/drive/MyDrive/Building Damage Assessment/Classification Models/history_vitb16ordinal.pkl', 'wb') as f:
    import pickle
    pickle.dump(history, f)
```

```
# ViT-B/16 Model with UnWeighted CE
model = vit_b_16(weights=ViT_B_16_Weights.IMAGENET1K_V1)
for param in model.parameters():
```

```

    param.requires_grad = False
model.heads.head = nn.Linear(model.heads.head.in_features, 4) # Replace classification head for 4 classes
for param in model.encoder.layers[-1].parameters(): # Unfreeze last transformer layer
    param.requires_grad = True

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

# Weighted Loss, Optimizer, Scheduler
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

# Train
model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10, model_name="ViT-B/16 UnWeighted Cross Entropy")

# Save the ViT-B/16 model
torch.save(model.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification Models/vit_b16_unweightedce1.pth')
with open('/content/drive/MyDrive/Building Damage Assessment/Classification Models/history_vitb16unweightedce1.pkl', 'wb') as f:
    import pickle
    pickle.dump(history, f)

```

 Downloading: "https://download.pytorch.org/models/vit_b_16-c867db91.pth" to /root/.cache/torch/hub/checkpoints/vit_b_16-c867db91.pth
 100%|██████████| 330M/330M [00:02<00:00, 137MB/s]
 Training ViT-B/16 UnWeighted Cross Entropy
 =====
 Epoch 1/10

 train: 100%|██████████| 3729/3729 [32:38<00:00, 1.90it/s, loss=0.3975, acc=1.9336]
 train Loss: 0.4195 Acc: 0.8461
 val: 100%|██████████| 1252/1252 [09:03<00:00, 2.30it/s, loss=0.2805, acc=1.2327]
 val Loss: 0.4028 Acc: 0.8477

 Epoch 2/10

 train: 100%|██████████| 3729/3729 [33:49<00:00, 1.84it/s, loss=0.1210, acc=1.9889]
 train Loss: 0.3490 Acc: 0.8703
 val: 100%|██████████| 1252/1252 [09:00<00:00, 2.32it/s, loss=0.2064, acc=1.2384]
 val Loss: 0.3962 Acc: 0.8516

 Epoch 3/10

 train: 100%|██████████| 3729/3729 [33:45<00:00, 1.84it/s, loss=0.2039, acc=2.0080]
 train Loss: 0.3211 Acc: 0.8786
 val: 100%|██████████| 1252/1252 [08:57<00:00, 2.33it/s, loss=0.2299, acc=1.2472]
 val Loss: 0.3940 Acc: 0.8576

 Epoch 4/10

 train: 100%|██████████| 3729/3729 [33:48<00:00, 1.84it/s, loss=0.2612, acc=2.0240]
 train Loss: 0.3003 Acc: 0.8856
 val: 100%|██████████| 1252/1252 [08:54<00:00, 2.34it/s, loss=0.1741, acc=1.2520]
 val Loss: 0.3793 Acc: 0.8610

 Epoch 5/10

 train: 100%|██████████| 3729/3729 [33:59<00:00, 1.83it/s, loss=0.3166, acc=2.0375]
 train Loss: 0.2835 Acc: 0.8915
 val: 100%|██████████| 1252/1252 [08:58<00:00, 2.32it/s, loss=0.1641, acc=1.2418]
 val Loss: 0.3995 Acc: 0.8539

```

Classification Report:
      precision    recall  f1-score   support

no-damage      0.92      0.93      0.93     28852
minor-damage    0.62      0.54      0.58      4715
major-damage    0.62      0.68      0.65      3840
destroyed       0.83      0.81      0.82      2647

 accuracy      0.85      0.85      0.85     40054
 macro avg      0.75      0.74      0.74     40054
 weighted avg   0.85      0.85      0.85     40054

```

Epoch 6/10

```

-----
train: 100%|██████████| 3729/3729 [34:16<00:00, 1.81it/s, loss=0.4027, acc=2.0520]
train Loss: 0.2655 Acc: 0.8979
val: 100%|██████████| 1252/1252 [09:11<00:00, 2.27it/s, loss=0.1591, acc=1.2485]
val Loss: 0.4011 Acc: 0.8586

```

```

# ViT-B/16 Model with UnWeighted Focal Loss
from torchvision.models import vit_b_16, ViT_B_16_Weights
model = vit_b_16(weights=ViT_B_16_Weights.IMAGENET1K_V1)
for param in model.parameters():
    param.requires_grad = False
model.heads.head = nn.Linear(model.heads.head.in_features, 4)
for param in model.encoder.layers[-1].parameters():
    param.requires_grad = True

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

# UnWeighted Focal Loss
criterion = FocalLoss(gamma=2)
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10, model_name="ViT-B/16 UnWeighted Focal Loss")

torch.save(model.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification Models/vit_b16_unweightedf11.pth')
with open('/content/drive/MyDrive/Building Damage Assessment/Classification Models/history_vitb16unweightedf11.pkl', 'wb') as f:
    import pickle
    pickle.dump(history, f)

📄 Downloading: "https://download.pytorch.org/models/vit_b_16-c867db91.pth" to /root/.cache/torch/hub/checkpoints/vit_b_16-c867db91.pth
100%|██████████| 330M/330M [00:05<00:00, 65.7MB/s]
Training ViT-B/16 UnWeighted Focal Loss
=====
Epoch 1/10
-----
train: 100%|██████████| 3729/3729 [30:59<00:00, 2.00it/s, loss=0.4200, acc=1.9245]
train Loss: 0.1969 Acc: 0.8421
val: 100%|██████████| 1252/1252 [08:21<00:00, 2.50it/s, loss=0.1018, acc=1.2199]
val Loss: 0.1884 Acc: 0.8389

Epoch 2/10
-----
train: 100%|██████████| 3729/3729 [31:41<00:00, 1.96it/s, loss=0.2445, acc=1.9762]

```

```
train Loss: 0.1560 Acc: 0.8647
val: 100%|██████████| 1252/1252 [08:26<00:00, 2.47it/s, loss=0.1021, acc=1.2444]
val Loss: 0.1755 Acc: 0.8558

Epoch 3/10
-----
train: 100%|██████████| 3729/3729 [32:28<00:00, 1.91it/s, loss=0.0745, acc=1.9999]
train Loss: 0.1404 Acc: 0.8751
val: 100%|██████████| 1252/1252 [08:22<00:00, 2.49it/s, loss=0.0962, acc=1.2499]
val Loss: 0.1729 Acc: 0.8595

Epoch 4/10
-----
train: 100%|██████████| 3729/3729 [32:14<00:00, 1.93it/s, loss=0.0592, acc=2.0119]
train Loss: 0.1305 Acc: 0.8804
val: 100%|██████████| 1252/1252 [08:20<00:00, 2.50it/s, loss=0.1338, acc=1.2516]
val Loss: 0.1715 Acc: 0.8607

Epoch 5/10
-----
train: 100%|██████████| 3729/3729 [32:56<00:00, 1.89it/s, loss=0.1834, acc=2.0249]
train Loss: 0.1205 Acc: 0.8860
val: 100%|██████████| 1252/1252 [08:28<00:00, 2.46it/s, loss=0.1510, acc=1.2497]
val Loss: 0.1748 Acc: 0.8594

Classification Report:
      precision    recall  f1-score   support

no-damage      0.89      0.97      0.93     28852
minor-damage    0.75      0.41      0.53      4715
major-damage    0.64      0.68      0.66      3840
destroyed       0.90      0.75      0.82      2647

   accuracy
macro avg    0.80      0.70      0.73     40054
weighted avg  0.85      0.86      0.85     40054

Epoch 6/10
-----
train: 100%|██████████| 3729/3729 [32:30<00:00, 1.91it/s, loss=0.1195, acc=2.0327]
train Loss: 0.1135 Acc: 0.8894
val: 100%|██████████| 1252/1252 [08:23<00:00, 2.49it/s, loss=0.1823, acc=1.2485]
val Loss: 0.1754 Acc: 0.8586

# ViT-B/16 Model with UnWeighted Focal Loss + High Gamma
from torchvision.models import vit_b_16, ViT_B_16_Weights
model = vit_b_16(weights=ViT_B_16_Weights.IMAGENET1K_V1)
for param in model.parameters():
    param.requires_grad = False
model.heads.head = nn.Linear(model.heads.head.in_features, 4)
for param in model.encoder.layers[-1].parameters():
    param.requires_grad = True

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

# UnWeighted Focal Loss
criterion = FocalLoss(gamma=4)
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10, model_name="ViT-B/16 UnWeighted Focal Loss High Gamma")

torch.save(model.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification Models/vit_b16_unweightedf1lhighg.pth')
with open('/content/drive/MyDrive/Building Damage Assessment/Classification Models/history_vitb16unweightedf1lhighg.pkl', 'wb') as f:
    import pickle
    pickle.dump(history, f)
```

↗ Training ViT-B/16 UnWeighted Focal Loss High Gamma

=====

Epoch 1/10

train: 100%|██████████| 3729/3729 [34:02<00:00, 1.83it/s, loss=0.1966, acc=1.9084]
train Loss: 0.1038 Acc: 0.8350
val: 100%|██████████| 1252/1252 [08:31<00:00, 2.45it/s, loss=0.0899, acc=1.2282]
val Loss: 0.0978 Acc: 0.8446

Epoch 2/10

train: 100%|██████████| 3729/3729 [32:52<00:00, 1.89it/s, loss=0.0120, acc=1.9662]
train Loss: 0.0792 Acc: 0.8603
val: 100%|██████████| 1252/1252 [08:25<00:00, 2.47it/s, loss=0.1052, acc=1.2501]
val Loss: 0.0889 Acc: 0.8597

Epoch 3/10

train: 100%|██████████| 3729/3729 [32:31<00:00, 1.91it/s, loss=0.2098, acc=1.9838]
train Loss: 0.0703 Acc: 0.8680
val: 100%|██████████| 1252/1252 [08:25<00:00, 2.48it/s, loss=0.1056, acc=1.2439]
val Loss: 0.0877 Acc: 0.8554

Epoch 4/10

train: 100%|██████████| 3729/3729 [32:52<00:00, 1.89it/s, loss=0.0631, acc=1.9938]
train Loss: 0.0642 Acc: 0.8724
val: 100%|██████████| 1252/1252 [08:26<00:00, 2.47it/s, loss=0.0809, acc=1.2481]
val Loss: 0.0905 Acc: 0.8583

Epoch 5/10

train: 100%|██████████| 3729/3729 [32:59<00:00, 1.88it/s, loss=0.0164, acc=2.0037]
train Loss: 0.0583 Acc: 0.8768
val: 100%|██████████| 1252/1252 [08:25<00:00, 2.48it/s, loss=0.0790, acc=1.2464]
val Loss: 0.0915 Acc: 0.8571

Classification Report:

	precision	recall	f1-score	support
no-damage	0.89	0.96	0.93	28852
minor-damage	0.68	0.48	0.56	4715
major-damage	0.68	0.60	0.64	3840
destroyed	0.87	0.76	0.81	2647
accuracy			0.86	40054
macro avg	0.78	0.70	0.73	40054
weighted avg	0.85	0.86	0.85	40054

Epoch 6/10

```

train: 100%|██████████| 3729/3729 [32:28<00:00, 1.91it/s, loss=0.0194, acc=2.0130]
train Loss: 0.0546 Acc: 0.8808
val: 100%|██████████| 1252/1252 [08:22<00:00, 2.49it/s, loss=0.1531, acc=1.2440]
val Loss: 0.0959 Acc: 0.8555

```

Epoch 7/10

```

# ViT with Ordinal Loss (No Weights)
model = vit_b_16(weights=ViT_B_16_Weights.IMAGENET1K_V1)

for param in model.parameters():
    param.requires_grad = False
model.heads.head = nn.Linear(model.heads.head.in_features, 3)
for param in model.encoder.layers[-1].parameters():
    param.requires_grad = True
model = model.to(device)

criterion = OrdinalLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

model, history = train_modelol(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10,
                                model_name="ViT-B/16 OL No weights", use_ordinal=True)

```



Training ViT-B/16 OL No weights

=====

Epoch 1/10

```

train: 100%|██████████| 3729/3729 [1:02:17<00:00, 1.00s/it, loss=0.1835, acc=1.8950]
train Loss: 0.1872 Acc: 0.8292
val: 100%|██████████| 1252/1252 [14:17<00:00, 1.46it/s, loss=0.1492, acc=1.2130]
val Loss: 0.1794 Acc: 0.8341

```

Epoch 2/10

```

train: 100%|██████████| 3729/3729 [1:02:13<00:00, 1.00s/it, loss=0.1349, acc=1.9607]
train Loss: 0.1508 Acc: 0.8579
val: 100%|██████████| 1252/1252 [14:14<00:00, 1.47it/s, loss=0.1389, acc=1.2218]
val Loss: 0.1722 Acc: 0.8402

```

Epoch 3/10

```

train: 100%|██████████| 3729/3729 [1:02:09<00:00, 1.00s/it, loss=0.0765, acc=1.9836]
train Loss: 0.1379 Acc: 0.8679
val: 100%|██████████| 1252/1252 [14:13<00:00, 1.47it/s, loss=0.2090, acc=1.2210]
val Loss: 0.1721 Acc: 0.8396

```

Epoch 4/10

```

train: 100%|██████████| 3729/3729 [1:02:14<00:00, 1.00s/it, loss=0.3306, acc=1.9987]
train Loss: 0.1290 Acc: 0.8746
val: 100%|██████████| 1252/1252 [14:13<00:00, 1.47it/s, loss=0.1960, acc=1.2454]
val Loss: 0.1627 Acc: 0.8564

```

Epoch 5/10

```

train: 100%|██████████| 3729/3729 [1:02:19<00:00, 1.00s/it, loss=0.0280, acc=2.0141]
train Loss: 0.1214 Acc: 0.8813
val: 100%|██████████| 1252/1252 [14:16<00:00, 1.46it/s, loss=0.2718, acc=1.2403]
val Loss: 0.1612 Acc: 0.8529

```



```

Classification Report:
              precision    recall  f1-score   support

no-damage      0.93        0.93        0.93     28852
minor-damage    0.55        0.60        0.58       4715
major-damage    0.65        0.62        0.64       3840
destroyed       0.89        0.78        0.83       2647

 accuracy      0.85        0.85        0.85     40054
 macro avg      0.76        0.73        0.74     40054
 weighted avg    0.86        0.85        0.85     40054

```

Epoch 6/10

```

-----
train: 100%|██████████| 3729/3729 [1:01:57<00:00, 1.00it/s, loss=0.0706, acc=2.0230]
train Loss: 0.1148 Acc: 0.8852
val: 100%|██████████| 1252/1252 [14:05<00:00, 1.48it/s, loss=0.1710, acc=1.2455]
val Loss: 0.1627 Acc: 0.8565

```

Epoch 7/10

```

# Save
save_dir = r"C:\Users\sajja\Desktop\Dissertation\Classification Models"
os.makedirs(save_dir, exist_ok=True)
model_path = os.path.join(save_dir, "vitb16olnoweights.pth")
history_path = os.path.join(save_dir, "history_vitb16unweightedol.pkl")

```

```

# Save the model weights
torch.save(model.state_dict(), model_path)
# Save the training history
with open(history_path, "wb") as f:
    pickle.dump(history, f)

```

✓ DeiT

```
import timm
```

 c:\Users\sajja\anaconda3\envs\dissertation_env\lib\site-packages\tqdm\auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See <https://ipywidgets.readthedocs.io/en/>:
from .autonotebook import tqdm as notebook_tqdm

```

# DeiT Model with Weighted CE
model = timm.create_model('deit_base_patch16_224', pretrained=True)
for param in model.parameters():
    param.requires_grad = False
model.head = nn.Linear(model.head.in_features, 4) # Replace classification head for 4 classes
for param in model.blocks[-1].parameters(): # Unfreeze last transformer block
    param.requires_grad = True

```

```

# Move model to device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

```

```
# Weighted Loss, Optimizer, Scheduler
```

```

criterion = nn.CrossEntropyLoss(weight=class_weights.to(device))
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

# Train
model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10,
                             model_name="DeiT-B Weighted Cross Entropy")

```

"""

Accidentally re run the cell, so the model is already trained and saved but the output is not shown
 Final output from previously saved session:

Training DeiT-B Weighted Cross Entropy
 =====

Epoch 1/10
 train Loss: 0.7531 Acc: 0.7116
 val Loss: 0.6417 Acc: 0.7604

Epoch 2/10
 train Loss: 0.6695 Acc: 0.7460
 val Loss: 0.6761 Acc: 0.7137

Epoch 3/10
 train Loss: 0.6456 Acc: 0.7561
 val Loss: 0.6939 Acc: 0.7245

Epoch 4/10
 train Loss: 0.6302 Acc: 0.7622
 val Loss: 0.5862 Acc: 0.7791

Epoch 5/10
 train Loss: 0.6191 Acc: 0.7661
 val Loss: 0.6245 Acc: 0.7489

Epoch 6/10
 train Loss: 0.6049 Acc: 0.7709
 val Loss: 0.5717 Acc: 0.7957

Epoch 7/10
 train Loss: 0.6003 Acc: 0.7759
 val Loss: 0.6599 Acc: 0.7295

Epoch 8/10
 train Loss: 0.5930 Acc: 0.7754
 val Loss: 0.6140 Acc: 0.7523

Epoch 9/10
 train Loss: 0.5834 Acc: 0.7798
 val Loss: 0.6047 Acc: 0.7581

Epoch 10/10
 train Loss: 0.5543 Acc: 0.7887
 val Loss: 0.5870 Acc: 0.7556

Classification Report:
 precision recall f1-score support

no-damage	0.97	0.76	0.85	28852
minor-damage	0.41	0.74	0.53	4715
major-damage	0.49	0.70	0.57	3840
destroyed	0.62	0.85	0.72	2647
accuracy			0.76	40054
macro avg	0.62	0.76	0.67	40054
weighted avg	0.84	0.76	0.78	40054

Training complete in 397m 7s
 Best val acc: 0.7957 (epoch 6)

"""

```
# Save the DeiT-B model
torch.save(model.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification Models/deit_b_ce1.pth')
with open('/content/drive/MyDrive/Building Damage Assessment/Classification Models/history_deitbce1.pkl', 'wb') as f:
    import pickle
    pickle.dump(history, f)
```

```
# DeiT Model with Focal Loss
model = timm.create_model('deit_base_patch16_224', pretrained=True)
for param in model.parameters():
    param.requires_grad = False
model.head = nn.Linear(model.head.in_features, 4)
for param in model.blocks[-1].parameters():
    param.requires_grad = True
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
```

```
# Weighted Focal Loss
criterion = FocalLoss(alpha=class_weights.to(device), gamma=2)
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)
```

```
model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10,
                             model_name="DeiT-B Weighted Focal Loss")
```



/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:

The secret `HF\_TOKEN` does not exist in your Colab secrets.

To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it as secret in your Google Colab and restart your session.

You will be able to reuse this secret in all of your notebooks.

Please note that authentication is recommended but still optional to access public models or datasets.

warnings.warn()

Training DeiT-B Weighted Focal Loss

=====

Epoch 1/10

train: 100%|██████████| 3729/3729 [29:26<00:00, 2.11it/s, loss=0.1166, acc=1.1309]

train Loss: 0.4836 Acc: 0.4948

val: 100%|██████████| 1252/1252 [08:50<00:00, 2.36it/s, loss=0.3625, acc=0.7470]

val Loss: 0.4951 Acc: 0.5137

Epoch 2/10

```

-----
train: 100%|██████████| 3729/3729 [31:00<00:00, 2.00it/s, loss=0.6645, acc=1.2273]
train Loss: 0.4277 Acc: 0.5370
val: 100%|██████████| 1252/1252 [08:50<00:00, 2.36it/s, loss=0.4633, acc=0.7138]
val Loss: 0.4584 Acc: 0.4909

```

Epoch 3/10

```

-----
train: 100%|██████████| 3729/3729 [30:54<00:00, 2.01it/s, loss=0.2581, acc=1.2659]
train Loss: 0.4070 Acc: 0.5539
val: 100%|██████████| 1252/1252 [08:41<00:00, 2.40it/s, loss=0.3347, acc=0.7489]
val Loss: 0.4448 Acc: 0.5150

```

Epoch 4/10

```

-----
train: 100%|██████████| 3729/3729 [31:14<00:00, 1.99it/s, loss=0.3713, acc=1.2905]
train Loss: 0.3954 Acc: 0.5647
val: 100%|██████████| 1252/1252 [08:42<00:00, 2.39it/s, loss=0.4113, acc=0.8704]
val Loss: 0.4548 Acc: 0.5985

```

Epoch 5/10

```

-----
train: 100%|██████████| 3729/3729 [32:36<00:00, 1.91it/s, loss=0.1505, acc=1.3228]
train Loss: 0.3784 Acc: 0.5788
val: 100%|██████████| 1252/1252 [08:47<00:00, 2.38it/s, loss=0.3124, acc=0.6782]
val Loss: 0.4363 Acc: 0.4663

```

Classification Report:

	precision	recall	f1-score	support
no-damage	0.99	0.34	0.51	28852
minor-damage	0.24	0.83	0.37	4715
major-damage	0.37	0.65	0.48	3840
destroyed	0.32	0.87	0.47	2647
accuracy			0.47	40054
macro avg	0.48	0.67	0.46	40054
weighted avg	0.80	0.47	0.49	40054

Epoch 6/10

```

torch.save(model.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification Models/deit_b_f11.pth')
with open('/content/drive/MyDrive/Building Damage Assessment/Classification Models/history_deitbf11.pkl', 'wb') as f:
    import pickle
    pickle.dump(history, f)

```

```

# DeiT Model with Ordinal Loss
model = timm.create_model('deit_base_patch16_224', pretrained=True)
for param in model.parameters():
    param.requires_grad = False
model.head = nn.Linear(model.head.in_features, 3) # K-1 thresholds
for param in model.blocks[-1].parameters():
    param.requires_grad = True

```

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

```

```

criterion = OrdinalLoss(weight=class_weights.to(device))

```

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10, model_name="DeiT-B OL", use_ordinal=True)
```

```
Training DeiT-B OL
=====
Epoch 1/10
-----
train: 100%|██████████| 3729/3729 [31:29<00:00, 1.97it/s, loss=0.5437, acc=1.4470]
train Loss: 0.3080 Acc: 0.6332
val: 100%|██████████| 1252/1252 [08:38<00:00, 2.42it/s, loss=0.4107, acc=0.7976]
val Loss: 0.3140 Acc: 0.5485

Epoch 2/10
-----
train: 100%|██████████| 3729/3729 [31:51<00:00, 1.95it/s, loss=0.0876, acc=1.5720]
train Loss: 0.2727 Acc: 0.6879
val: 100%|██████████| 1252/1252 [08:17<00:00, 2.52it/s, loss=0.2660, acc=0.9853]
val Loss: 0.3150 Acc: 0.6776

Epoch 3/10
-----
train: 100%|██████████| 3729/3729 [32:31<00:00, 1.91it/s, loss=0.2616, acc=1.6142]
train Loss: 0.2589 Acc: 0.7063
val: 100%|██████████| 1252/1252 [08:20<00:00, 2.50it/s, loss=0.3397, acc=0.9604]
val Loss: 0.2846 Acc: 0.6604

Epoch 4/10
-----
train: 100%|██████████| 3729/3729 [32:13<00:00, 1.93it/s, loss=0.1689, acc=1.6370]
train Loss: 0.2508 Acc: 0.7163
val: 100%|██████████| 1252/1252 [08:26<00:00, 2.47it/s, loss=0.2770, acc=1.0055]
val Loss: 0.2805 Acc: 0.6914

Epoch 5/10
-----
train: 100%|██████████| 3729/3729 [32:23<00:00, 1.92it/s, loss=0.1055, acc=1.6500]
train Loss: 0.2441 Acc: 0.7220
val: 100%|██████████| 1252/1252 [08:30<00:00, 2.45it/s, loss=0.3608, acc=1.0672]
val Loss: 0.2810 Acc: 0.7339

Classification Report:
      precision    recall  f1-score   support

no-damage      0.97      0.73      0.84      28852
minor-damage    0.34      0.78      0.47       4715
major-damage    0.50      0.64      0.56       3840
destroyed       0.85      0.79      0.82       2647

    accuracy      0.66      0.74      0.67      40054
   macro avg      0.66      0.74      0.67      40054
  weighted avg      0.84      0.73      0.77      40054

Epoch 6/10
-----
train: 100%|██████████| 3729/3729 [32:27<00:00, 1.91it/s, loss=0.4124, acc=1.6694]
train Loss: 0.2390 Acc: 0.7305
val: 100%|██████████| 1252/1252 [08:35<00:00, 2.43it/s, loss=0.3070, acc=0.9816]
val Loss: 0.2803 Acc: 0.6750
```

Epoch 7/10

```
torch.save(model.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification Models/deit_b_ordinal.pth')
with open('/content/drive/MyDrive/Building Damage Assessment/Classification Models/history_deitb_ordinal.pkl', 'wb') as f:
    import pickle
    pickle.dump(history, f)

# DeiT Model with Unweighted Cross entropy Loss
model = timm.create_model('deit_base_patch16_224', pretrained=True)
for param in model.parameters():
    param.requires_grad = False
model.head = nn.Linear(model.head.in_features, 4)
for param in model.blocks[-1].parameters():
    param.requires_grad = True

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

# Unweighted Cross Entropy
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10, model_name="DeiT-B UnWeighted Focal Loss")

torch.save(model.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification Models/deit_b_unweightedce1.pth')
with open('/content/drive/MyDrive/Building Damage Assessment/Classification Models/history_deitb_unweightedce1.pkl', 'wb') as f:
    import pickle
    pickle.dump(history, f)
```

```

/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(

model.safetensors: 100%                               346M/346M [00:01<00:00, 259MB/s]

Training DeiT-B UnWeighted Focal Loss
=====
Epoch 1/10
-----
train: 100%|██████████| 3729/3729 [30:51<00:00, 2.01it/s, loss=0.2510, acc=1.8987]
train Loss: 0.4559 Acc: 0.8308
val: 100%|██████████| 1252/1252 [09:05<00:00, 2.29it/s, loss=0.3955, acc=1.2201]
val Loss: 0.4404 Acc: 0.8390

Epoch 2/10
-----
train: 100%|██████████| 3729/3729 [31:54<00:00, 1.95it/s, loss=0.0901, acc=1.9466]
train Loss: 0.3998 Acc: 0.8518
val: 100%|██████████| 1252/1252 [09:21<00:00, 2.23it/s, loss=0.2172, acc=1.2140]
val Loss: 0.4449 Acc: 0.8348

Epoch 3/10
-----
train: 100%|██████████| 3729/3729 [33:01<00:00, 1.88it/s, loss=0.1208, acc=1.9603]
train Loss: 0.3824 Acc: 0.8577
val: 100%|██████████| 1252/1252 [09:15<00:00, 2.25it/s, loss=0.4328, acc=1.2296]
val Loss: 0.4192 Acc: 0.8456

Epoch 4/10
-----
train: 100%|██████████| 3729/3729 [32:37<00:00, 1.91it/s, loss=0.8491, acc=1.9661]
train Loss: 0.3707 Acc: 0.8603
val: 100%|██████████| 1252/1252 [09:09<00:00, 2.28it/s, loss=0.3707, acc=1.2338]
val Loss: 0.4122 Acc: 0.8485

Epoch 5/10
-----
train: 100%|██████████| 3729/3729 [32:13<00:00, 1.93it/s, loss=0.0422, acc=1.9732]
train Loss: 0.3625 Acc: 0.8634
val: 100%|██████████| 1252/1252 [09:00<00:00, 2.32it/s, loss=0.3540, acc=1.2313]
val Loss: 0.4191 Acc: 0.8468

Classification Report:
      precision    recall  f1-score   support

no-damage         0.91      0.93      0.92      28852
minor-damage       0.65      0.52      0.58       4715
major-damage       0.57      0.67      0.61       3840
destroyed          0.87      0.76      0.81       2647

accuracy                   0.85      40054
macro avg              0.75      0.72      0.73      40054
weighted avg           0.85      0.85      0.84      40054

# DeiT Model with Unweighted Focal Loss
model = timm.create_model('deit_base_patch16_224', pretrained=True)
for param in model.parameters():
```


no-damage	0.89	0.96	0.92	28852
minor-damage	0.73	0.43	0.54	4715
major-damage	0.62	0.61	0.61	3840
destroyed	0.84	0.78	0.81	2647
accuracy			0.85	40054
macro avg	0.77	0.69	0.72	40054
weighted avg	0.84	0.85	0.84	40054

Epoch 6/10

```

-----
train: 100%|██████████| 3729/3729 [31:38<00:00, 1.96it/s, loss=0.1558, acc=1.9692]
train Loss: 0.1585 Acc: 0.8617
val: 100%|██████████| 1252/1252 [08:13<00:00, 2.54it/s, loss=0.2151, acc=1.2202]
val Loss: 0.1914 Acc: 0.8391

```

Epoch 7/10

```

# DeiT Model with Unweighted Focal Loss and High Gamma
model = timm.create_model('deit_base_patch16_224', pretrained=True)
for param in model.parameters():
    param.requires_grad = False
model.head = nn.Linear(model.head.in_features, 4)
for param in model.blocks[-1].parameters():
    param.requires_grad = True

```

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

```

```

# UnWeighted Focal Loss
criterion = FocalLoss(gamma=4)
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

```

```

model, history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10, model_name="DeiT-B UnWeighted Focal Loss High Gamma")

```

```

torch.save(model.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification Models/deit_b_unweightedfllgamma.pth')
with open('/content/drive/MyDrive/Building Damage Assessment/Classification Models/history_deitb_unweightedfllgamma.pkl', 'wb') as f:
    import pickle
    pickle.dump(history, f)

```



Training DeiT-B UnWeighted Focal Loss High Gamma

```

=====
Epoch 1/10
-----

```

```

train: 100%|██████████| 3729/3729 [34:54<00:00, 1.78it/s, loss=0.0712, acc=1.8774]
train Loss: 0.1128 Acc: 0.8215
val: 100%|██████████| 1252/1252 [09:18<00:00, 2.24it/s, loss=0.0942, acc=1.1656]
val Loss: 0.1262 Acc: 0.8015

```

Epoch 2/10

```

-----
train: 100%|██████████| 3729/3729 [34:29<00:00, 1.80it/s, loss=0.0592, acc=1.9150]
train Loss: 0.0975 Acc: 0.8379
val: 100%|██████████| 1252/1252 [09:15<00:00, 2.25it/s, loss=0.0793, acc=1.1962]
val Loss: 0.1055 Acc: 0.8226

```

Epoch 3/10

```

-----
train: 100%|██████████| 3729/3729 [34:18<00:00, 1.81it/s, loss=0.0191, acc=1.9317]
train Loss: 0.0904 Acc: 0.8453
val: 100%|██████████| 1252/1252 [09:10<00:00, 2.27it/s, loss=0.1235, acc=1.2154]
val Loss: 0.0984 Acc: 0.8358

```

Epoch 4/10

```

-----
train: 100%|██████████| 3729/3729 [33:42<00:00, 1.84it/s, loss=0.1202, acc=1.9379]
train Loss: 0.0869 Acc: 0.8480
val: 100%|██████████| 1252/1252 [09:08<00:00, 2.28it/s, loss=0.1888, acc=1.1994]
val Loss: 0.1047 Acc: 0.8248

```

Epoch 5/10

```

-----
train: 100%|██████████| 3729/3729 [34:25<00:00, 1.81it/s, loss=0.0535, acc=1.9455]
train Loss: 0.0840 Acc: 0.8513
val: 100%|██████████| 1252/1252 [09:13<00:00, 2.26it/s, loss=0.1653, acc=1.2114]
val Loss: 0.1007 Acc: 0.8331

```

Classification Report:

	precision	recall	f1-score	support
no-damage	0.90	0.94	0.92	28852
minor-damage	0.68	0.40	0.50	4715
major-damage	0.56	0.59	0.58	3840
destroyed	0.72	0.83	0.77	2647
accuracy			0.83	40054
macro avg	0.71	0.69	0.69	40054
weighted avg	0.83	0.83	0.82	40054

Epoch 6/10

```

-----
train: 100%|██████████| 3729/3729 [34:12<00:00, 1.82it/s, loss=0.0588, acc=1.9509]
train Loss: 0.0808 Acc: 0.8537
val: 100%|██████████| 1252/1252 [09:31<00:00, 2.19it/s, loss=0.1353, acc=1.2196]
val Loss: 0.0997 Acc: 0.8387

```

Epoch 7/10

```

# DeiT Model with Ordinal Loss (No class weights)
model = timm.create_model('deit_base_patch16_224', pretrained=True)
for param in model.parameters():
    param.requires_grad = False
model.head = nn.Linear(model.head.in_features, 3) # K-1 thresholds
for param in model.blocks[-1].parameters():
    param.requires_grad = True

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

criterion = OrdinalLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)

model, history = train_modelol(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=10,
                                model_name="DeiT-B OL No Weights", use_ordinal=True)

```

```
↩ Training DeiT-B OL No Weights
=====
Epoch 1/10
-----
train: 100%|██████████| 3729/3729 [47:39<00:00, 1.30it/s, loss=0.2597, acc=1.8611]
train Loss: 0.2051 Acc: 0.8144
val: 100%|██████████| 1252/1252 [13:28<00:00, 1.55it/s, loss=0.2706, acc=1.1689]
val Loss: 0.2196 Acc: 0.8038

Epoch 2/10
-----
train: 100%|██████████| 3729/3729 [47:42<00:00, 1.30it/s, loss=0.1335, acc=1.9127]
train Loss: 0.1783 Acc: 0.8369
val: 100%|██████████| 1252/1252 [13:38<00:00, 1.53it/s, loss=0.1718, acc=1.1916]
val Loss: 0.2041 Acc: 0.8194

Epoch 3/10
-----
train: 100%|██████████| 3729/3729 [47:52<00:00, 1.30it/s, loss=0.0959, acc=1.9268]
train Loss: 0.1697 Acc: 0.8431
val: 100%|██████████| 1252/1252 [13:38<00:00, 1.53it/s, loss=0.2126, acc=1.2151]
val Loss: 0.1817 Acc: 0.8356

Epoch 4/10
-----
train: 100%|██████████| 3729/3729 [47:30<00:00, 1.31it/s, loss=0.1315, acc=1.9346]
train Loss: 0.1653 Acc: 0.8465
val: 100%|██████████| 1252/1252 [13:39<00:00, 1.53it/s, loss=0.1841, acc=1.2143]
val Loss: 0.1816 Acc: 0.8350

Epoch 5/10
-----
train: 100%|██████████| 3729/3729 [49:24<00:00, 1.26it/s, loss=0.2739, acc=1.9399]
train Loss: 0.1611 Acc: 0.8488
val: 100%|██████████| 1252/1252 [14:00<00:00, 1.49it/s, loss=0.1757, acc=1.2150]
val Loss: 0.1860 Acc: 0.8355

Classification Report:
      precision    recall  f1-score   support

no-damage      0.89      0.95      0.92     28852
minor-damage    0.57      0.46      0.50      4715
major-damage    0.60      0.52      0.56      3840
destroyed       0.88      0.73      0.80      2647

   accuracy      0.84      0.84      0.84     40054
  macro avg      0.74      0.66      0.70     40054
weighted avg      0.82      0.84      0.83     40054

Epoch 6/10
-----
train: 100%|██████████| 3729/3729 [50:02<00:00, 1.24it/s, loss=0.0853, acc=1.9496]
train Loss: 0.1569 Acc: 0.8531
val: 100%|██████████| 1252/1252 [13:51<00:00, 1.51it/s, loss=0.2285, acc=1.2256]
val Loss: 0.1724 Acc: 0.8428

Epoch 7/10
```

```
# Save
save_dir = r"C:\Users\sajja\Desktop\Dissertation\Classification Models"
```

```

os.makedirs(save_dir, exist_ok=True)
model_path = os.path.join(save_dir, "deit_unweighted_ordinal.pth")
history_path = os.path.join(save_dir, "history_deitunweightedordinal.pkl")

# Save the model weights
torch.save(model.state_dict(), model_path)
# Save the training history
with open(history_path, "wb") as f:
    pickle.dump(history, f)# Save

```

✓ Further Fine tuning

In this section, we attempt to fine tune each model with their best performing loss function observed in the ablation study conducted above. We adjust the training function to accomodate early stopping since we will be increasing the training epochs to 15 as we will use lower learning rate. Early stopping will prevent overfitting and unnecessary computation. The default patience is set to 5 but we will most likely use a custom patience of 3.

```

# Main Training Function with Early Stopping
def train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=15, model_name="Model",
                early_stopping_patience=5):

    model = model.to(device)

    # Track best model
    best_model_wts = copy.deepcopy(model.state_dict())
    best_acc = 0.0

    # Early stopping variables
    epochs_without_improvement = 0
    early_stopped = False

    # Time tracking
    since = time.time()

    # Training metrics history
    history = {
        'train_loss': [],
        'val_loss': [],
        'train_acc': [],
        'val_acc': [],
        'best_epoch': 0,
        'early_stopped': False,
        'stopped_epoch': 0
    }

    print(f"Training {model_name}")
    print(f"Early stopping patience: {early_stopping_patience} epochs")
    print("=" * 50)

    # Main training loop
    for epoch in range(num_epochs):
        print(f"Epoch {epoch+1}/{num_epochs}")

```

```

print('-' * 20)

# Each epoch has training and validation phase
for phase in ['train', 'val']:
    if phase == 'train':
        model.train()
        dataloader = train_loader
    else:
        model.eval()
        dataloader = val_loader

    running_loss = 0.0
    running_corrects = 0
    all_preds = []
    all_labels = []

    # Progress bar for batches
    pbar = tqdm(dataloader, desc=f'{phase}')

    for inputs, labels in pbar:
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer.zero_grad()

        # Forward pass
        with torch.set_grad_enabled(phase == 'train'):
            outputs = model(inputs)
            _, preds = torch.max(outputs, 1)
            loss = criterion(outputs, labels)

            if phase == 'train':
                loss.backward()
                optimizer.step()

        # Statistics
        running_loss += loss.item() * inputs.size(0)
        running_corrects += torch.sum(preds == labels.data)

        # Update progress bar
        pbar.set_postfix({
            'loss': f'{loss.item():.4f}',
            'acc': f'{running_corrects.double() / ((pbar.n + 1) * inputs.size(0)):.4f}'
        })

        # Collect for metrics
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

    # Calculate epoch statistics
    dataset_size = len(dataloader.dataset)
    epoch_loss = running_loss / dataset_size
    epoch_acc = running_corrects.double() / dataset_size

    print(f"{phase} Loss: {epoch_loss:.4f} Acc: {epoch_acc:.4f}")

    # Store history

```

```

if phase == 'train':
    history['train_loss'].append(epoch_loss)
    history['train_acc'].append(epoch_acc.item())
else:
    history['val_loss'].append(epoch_loss)
    history['val_acc'].append(epoch_acc.item())

    # Check if best model and early stopping logic
    if epoch_acc > best_acc:
        best_acc = epoch_acc
        best_model_wts = copy.deepcopy(model.state_dict())
        history['best_epoch'] = epoch
        epochs_without_improvement = 0 # Reset counter
        print(f"New best model with Validation accuracy of {best_acc:.4f}")
    else:
        epochs_without_improvement += 1
        print(f"No improvement for {epochs_without_improvement} epoch(s)")

    # Print classification report every 5 epochs or at the end
    if (epoch + 1) % 5 == 0 or epoch == num_epochs - 1:
        print("\nClassification Report:")
        class_names = ['no-damage', 'minor-damage', 'major-damage', 'destroyed']
        print(classification_report(all_labels, all_preds,
                                   target_names=class_names,
                                   zero_division=0))

# Update scheduler
if scheduler is not None:
    scheduler.step(history['val_loss'][-1]) # Step on validation loss

# Early stopping check
if epochs_without_improvement >= early_stopping_patience:
    print(f"\nEarly stopping triggered after {epoch + 1} epochs")
    print(f"No improvement in validation accuracy for {early_stopping_patience} consecutive epochs")
    early_stopped = True
    history['early_stopped'] = True
    history['stopped_epoch'] = epoch + 1
    break

print()

# Training complete
time_elapsed = time.time() - since
print(f"Training complete in {time_elapsed // 60:.0f}m {time_elapsed % 60:.0f}s")

if early_stopped:
    print(f"Training stopped early at epoch {history['stopped_epoch']}")

print(f"Best val acc: {best_acc:.4f} (epoch {history['best_epoch'] + 1})")

# Load best model weights
model.load_state_dict(best_model_wts)
return model, history

```

✓ ResNET

For the ResNet architecture, the model which utilises Cross Entropy with weights adjustment showed the best performance. We will further fine tune the model in stages until no improvement is observed.

```
from torchvision.models import resnet50, ResNet50_Weights

# ResNet Fine-tuning: Stage 1: Layer4 + Classifier (with Weight Decay and Reduced Learning Rate)
model = resnet50(weights=ResNet50_Weights.DEFAULT) # Load pretrained ResNet-50
# Freeze all parameters
for param in model.parameters():
    param.requires_grad = False
# Replace classifier for your 4-class problem
model.fc = nn.Linear(model.fc.in_features, 4)
# Unfreeze layer4 + classifier
# Initially freeze all parameters
for param in model.parameters():
    param.requires_grad = False

for param in model.layer4.parameters():
    param.requires_grad = True
for param in model.fc.parameters():
    param.requires_grad = True

model = model.to(device) # Set model to gpu

criterion = nn.CrossEntropyLoss(weight=class_weights.to(device)) # Loss function

# Optimizer with weight decay and different learning rates for layer4 and classifier for better flexibility
optimizer = optim.Adam([
    {'params': model.layer4.parameters(), 'lr': 0.0003, 'weight_decay': 1e-4},
    {'params': model.fc.parameters(), 'lr': 0.0005, 'weight_decay': 1e-4}
])
# Learning rate scheduler
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=3)

# Train the model with early stopping
model_stage_1, history_stage_1 = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler,
                                             num_epochs=15, model_name="ResNet-50 Fine-tuned (Layer4+FC)",
                                             early_stopping_patience=3)

# Save
save_dir = "/content/drive/MyDrive/Building Damage Assessment/Classification models/ResNet Fine Tuning"
os.makedirs(save_dir, exist_ok=True)
model_path_stage_1 = os.path.join(save_dir, "resnet_L4.pth")
history_path_stage_1 = os.path.join(save_dir, "history_resnet_L4.pkl")

# Save the model weights
torch.save(model_stage_1.state_dict(), model_path_stage_1)
# Save the training history
with open(history_path_stage_1, "wb") as f:
    pickle.dump(history_stage_1, f)
```

```

Download: "https://download.pytorch.org/models/resnet50-11ad3fa6.pth" to /root/.cache/torch/hub/checkpoints/resnet50-11ad3fa6.pth
100%|██████████| 97.8M/97.8M [00:00<00:00, 136MB/s]
Training ResNet-50 Fine-tuned (Layer4+FC)
Early stopping patience: 3 epochs
=====
Epoch 1/15
-----
train: 100%|██████████| 1865/1865 [22:59<00:00, 1.35it/s, loss=0.6117, acc=3.3943]
train Loss: 0.6901 Acc: 0.7428
val: 100%|██████████| 626/626 [04:53<00:00, 2.14it/s, loss=1.0707, acc=0.9694]
val Loss: 0.5237 Acc: 0.8181
New best model with Validation accuracy of 0.8181

Epoch 2/15
-----
train: 100%|██████████| 1865/1865 [24:58<00:00, 1.24it/s, loss=0.1675, acc=3.6322]
train Loss: 0.5796 Acc: 0.7949
val: 100%|██████████| 626/626 [05:13<00:00, 1.99it/s, loss=1.3259, acc=0.9727]
val Loss: 0.5134 Acc: 0.8209
New best model with Validation accuracy of 0.8209

Epoch 3/15
-----
train: 100%|██████████| 1865/1865 [26:57<00:00, 1.15it/s, loss=0.2560, acc=3.6954]
train Loss: 0.5369 Acc: 0.8087
val: 100%|██████████| 626/626 [05:11<00:00, 2.01it/s, loss=0.7155, acc=0.9777]
val Loss: 0.5230 Acc: 0.8251
New best model with Validation accuracy of 0.8251

Epoch 4/15
-----
train: 100%|██████████| 1865/1865 [26:46<00:00, 1.16it/s, loss=0.6592, acc=3.7237]
train Loss: 0.5142 Acc: 0.8149
val: 100%|██████████| 626/626 [05:00<00:00, 2.08it/s, loss=0.7182, acc=0.9620]
val Loss: 0.5043 Acc: 0.8119
No improvement for 1 epoch(s)

Epoch 5/15
-----
train: 100%|██████████| 1865/1865 [27:00<00:00, 1.15it/s, loss=0.9164, acc=3.7545]
train Loss: 0.4922 Acc: 0.8216
val: 100%|██████████| 626/626 [05:00<00:00, 2.09it/s, loss=0.8236, acc=0.9776]
val Loss: 0.5022 Acc: 0.8251
No improvement for 2 epoch(s)

Classification Report:
      precision    recall  f1-score   support

no-damage         0.96      0.86      0.91      28852
minor-damage       0.49      0.74      0.59       4715
major-damage       0.58      0.69      0.63       3840
destroyed          0.82      0.82      0.82       2647

   accuracy
macro avg       0.71      0.78      0.74      40054
weighted avg    0.86      0.83      0.84      40054

# ResNet Fine-tuning: Stage 2: Layer3 + Layer4 + Classifier
# Load the best model from Stage 1

```



```

stage1_model_path = os.path.join("/content/drive/MyDrive/Building Damage Assessment/Classification models/ResNet Fine Tuning/resnet_L4.pth")
model = resnet50(weights=ResNet50_Weights.DEFAULT)
model.fc = nn.Linear(model.fc.in_features, 4)

# Load Stage 1 weights
model.load_state_dict(torch.load(stage1_model_path))
model = model.to(device)
# Initially freeze all parameters
for param in model.parameters():
    param.requires_grad = False
# Current state: layer4 and fc are already unfrozen from Stage 1
# Now unfreeze layer3 as well
for param in model.layer3.parameters():
    param.requires_grad = True

# Verify which layers are unfrozen
print("Unfrozen layers:")
for name, param in model.named_parameters():
    if param.requires_grad:
        print(f"- {name}")

# Loss function (same as Stage 1)
criterion = nn.CrossEntropyLoss(weight=class_weights.to(device))

# Optimizer with discriminative learning rates
# Lower learning rate for layer3 (earlier layer), higher for layer4 and classifier
optimizer = optim.Adam([
    {'params': model.layer3.parameters(), 'lr': 0.0002, 'weight_decay': 5e-5}, # Lower LR for earlier layer
    {'params': model.layer4.parameters(), 'lr': 0.0003, 'weight_decay': 1e-4}, # Same as Stage 1
    {'params': model.fc.parameters(), 'lr': 0.0005, 'weight_decay': 1e-4}      # Highest LR for classifier
])

# Learning rate scheduler
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=3)

# Train the model with early stopping
model_stage2, history_stage2 = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler,
                                           num_epochs=15, model_name="ResNet-50 Fine-tuned Stage 2 (Layer3+4+FC)",
                                           early_stopping_patience=3)

# Save Stage 2 model and history
save_dir = "/content/drive/MyDrive/Building Damage Assessment/Classification models/ResNet Fine Tuning"
os.makedirs(save_dir, exist_ok=True)
model_path_stage2 = os.path.join(save_dir, "resnet_L3L4.pth")
history_path_stage2 = os.path.join(save_dir, "history_resnet_L3L4.pkl")

# Save the model weights
torch.save(model_stage2.state_dict(), model_path_stage2)
# Save the training history
with open(history_path_stage2, "wb") as f:
    pickle.dump(history_stage2, f)

📄 Downloading: "https://download.pytorch.org/models/resnet50-11ad3fa6.pth" to /root/.cache/torch/hub/checkpoints/resnet50-11ad3fa6.pth
100%|██████████| 97.8M/97.8M [00:00<00:00, 152MB/s]
Unfrozen layers:
- conv1.weight
- bn1.weight

```

- bn1.bias
- layer1.0.conv1.weight
- layer1.0.bn1.weight
- layer1.0.bn1.bias
- layer1.0.conv2.weight
- layer1.0.bn2.weight
- layer1.0.bn2.bias
- layer1.0.conv3.weight
- layer1.0.bn3.weight
- layer1.0.bn3.bias
- layer1.0.downsample.0.weight
- layer1.0.downsample.1.weight
- layer1.0.downsample.1.bias
- layer1.1.conv1.weight
- layer1.1.bn1.weight
- layer1.1.bn1.bias
- layer1.1.conv2.weight
- layer1.1.bn2.weight
- layer1.1.bn2.bias
- layer1.1.conv3.weight
- layer1.1.bn3.weight
- layer1.1.bn3.bias
- layer1.2.conv1.weight
- layer1.2.bn1.weight
- layer1.2.bn1.bias
- layer1.2.conv2.weight
- layer1.2.bn2.weight
- layer1.2.bn2.bias
- layer1.2.conv3.weight
- layer1.2.bn3.weight
- layer1.2.bn3.bias
- layer2.0.conv1.weight
- layer2.0.bn1.weight
- layer2.0.bn1.bias
- layer2.0.conv2.weight
- layer2.0.bn2.weight
- layer2.0.bn2.bias
- layer2.0.conv3.weight
- layer2.0.bn3.weight
- layer2.0.bn3.bias
- layer2.0.downsample.0.weight
- layer2.0.downsample.1.weight
- layer2.0.downsample.1.bias
- layer2.1.conv1.weight
- layer2.1.bn1.weight
- layer2.1.bn1.bias
- layer2.1.conv2.weight
- layer2.1.bn2.weight
- layer2.1.bn2.bias
- layer2.1.conv3.weight
- layer2.1.bn3.weight
- layer2.1.bn3.bias
- layer2.2.conv1.weight

```
# ResNet Fine-tuning: Stage 3: Layer2 + Layer3 + Layer4 + Classifier
```

```
# Load the best model from Stage 2
```

```
stage2_model_path = os.path.join("/content/drive/MyDrive/Building Damage Assessment/Classification models/ResNet Fine Tuning/resnet_L3L4.pth")
```

```
model = resnet50(weights=ResNet50_Weights.DEFAULT)
```

```
model.fc = nn.Linear(model.fc.in_features, 4)
```

```
# Load Stage 2 weights
```

```

model.load_state_dict(torch.load(stage2_model_path))
model = model.to(device)

# Initially freeze all parameters
for param in model.parameters():
    param.requires_grad = False
# Current state: layer3, layer4 and fc are already unfrozen from Stage 2
# We unfreeze layer2 as well
for param in model.layer2.parameters():
    param.requires_grad = True

# Verify which layers are unfrozen
print("Unfrozen layers:")
for name, param in model.named_parameters():
    if param.requires_grad:
        print(f"- {name}")

# Loss function (same as previous stages)
criterion = nn.CrossEntropyLoss(weight=class_weights.to(device))

# Optimizer with discriminative learning rates
# Lower learning rate for layer2 (earliest layer), progressively higher for later layers
optimizer = optim.Adam([
    {'params': model.layer2.parameters(), 'lr': 0.0001, 'weight_decay': 1e-5}, # Lowest LR for earliest layer
    {'params': model.layer3.parameters(), 'lr': 0.0002, 'weight_decay': 5e-5}, # Same as Stage 2
    {'params': model.layer4.parameters(), 'lr': 0.0003, 'weight_decay': 1e-4}, # Same as Stage 2
    {'params': model.fc.parameters(), 'lr': 0.0005, 'weight_decay': 1e-4}      # Highest LR for classifier
])

# Learning rate scheduler
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=3)

# Train the model with early stopping (with increased patience)
model_stage3, history_stage3 = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler,
                                           num_epochs=15, model_name="ResNet-50 Fine-tuned Stage 3 (Layer2+3+4+FC)",
                                           early_stopping_patience=5)

# Save Stage 3 model and history
save_dir = "/content/drive/MyDrive/Building Damage Assessment/Classification models/ResNet Fine Tuning"
model_path_stage3 = os.path.join(save_dir, "resnet_L2L3L4.pth")
history_path_stage3 = os.path.join(save_dir, "history_resnet_L2L3L4.pkl")

# Save the model weights
torch.save(model_stage3.state_dict(), model_path_stage3)
# Save the training history
with open(history_path_stage3, "wb") as f:
    pickle.dump(history_stage3, f)

📄 Downloading: "https://download.pytorch.org/models/resnet50-11ad3fa6.pth" to /root/.cache/torch/hub/checkpoints/resnet50-11ad3fa6.pth
100%|██████████| 97.8M/97.8M [00:00<00:00, 154MB/s]
Unfrozen layers:
- conv1.weight
- bn1.weight
- bn1.bias
- layer1.0.conv1.weight
- layer1.0.bn1.weight
- layer1.0.bn1.bias

```

```

- layer1.0.conv2.weight
- layer1.0.bn2.weight
- layer1.0.bn2.bias
- layer1.0.conv3.weight
- layer1.0.bn3.weight
- layer1.0.bn3.bias
- layer1.0.downsample.0.weight
- layer1.0.downsample.1.weight
- layer1.0.downsample.1.bias
- layer1.1.conv1.weight
- layer1.1.bn1.weight
- layer1.1.bn1.bias
- layer1.1.conv2.weight
- layer1.1.bn2.weight
- layer1.1.bn2.bias
- layer1.1.conv3.weight
- layer1.1.bn3.weight
- layer1.1.bn3.bias
- layer1.2.conv1.weight
- layer1.2.bn1.weight
- layer1.2.bn1.bias
- layer1.2.conv2.weight
- layer1.2.bn2.weight
- layer1.2.bn2.bias
- layer1.2.conv3.weight
- layer1.2.bn3.weight
- layer1.2.bn3.bias
- layer2.0.conv1.weight
- layer2.0.bn1.weight
- layer2.0.bn1.bias
- layer2.0.conv2.weight
- layer2.0.bn2.weight
- layer2.0.bn2.bias
- layer2.0.conv3.weight
- layer2.0.bn3.weight
- layer2.0.bn3.bias
- layer2.0.downsample.0.weight
- layer2.0.downsample.1.weight
- layer2.0.downsample.1.bias
- layer2.1.conv1.weight
- layer2.1.bn1.weight
- layer2.1.bn1.bias
- layer2.1.conv2.weight
- layer2.1.bn2.weight
- layer2.1.bn2.bias
- layer2.1.conv3.weight
- layer2.1.bn3.weight
- layer2.1.bn3.bias
- layer2.2.conv1.weight

```

We observe overfitting. Further fine tuning can be explored with more severe weight ddecay and drop out but we decide to stop here as there are other models to optimise. Moreover, validation loss seems to have plateaued.

✓ EfficientNET

```
from torchvision.models import efficientnet_b3, EfficientNet_B3_Weights
```

```

# EfficientNet Fine-tuning: Stage 1: Last 2 MBConv Blocks + Classifier
# Load pretrained EfficientNet-B0
model = efficientnet_b3(weights=EfficientNet_B3_Weights.IMAGENET1K_V1)
# Freeze all parameters
for param in model.parameters():
    param.requires_grad = False
# Replace classifier for 4 classes
model.classifier[1] = nn.Linear(model.classifier[1].in_features, 4)

# EfficientNet structure: features[0-7] are different stages
# 'features[7]' contains the last few MBConv blocks

# Unfreeze last 2 blocks from features[7] (final stage)
for param in model.features[7][-2:].parameters(): # Last 2 MBConv blocks
    param.requires_grad = True

# Unfreeze classifier
for param in model.classifier.parameters():
    param.requires_grad = True

model = model.to(device)

# Verify unfrozen parameters
print("Stage 1 - Unfrozen layers:")
for name, param in model.named_parameters():
    if param.requires_grad:
        print(f"- {name}")

# Loss function
criterion = nn.CrossEntropyLoss(weight=class_weights.to(device))

# Optimizer with weight decay and discriminative learning rates
optimizer = optim.Adam([
    {'params': model.features[7][-2:].parameters(), 'lr': 0.0003, 'weight_decay': 1e-4}, # Last 2 MBConv blocks
    {'params': model.classifier.parameters(), 'lr': 0.0005, 'weight_decay': 1e-4}      # Classifier
])

# Learning rate scheduler
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=3)

# Train Stage 1
model_stage1, history_stage1 = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler,
                                           num_epochs=15, model_name="EfficientNet-B3 Stage 1 (Last 2 Blocks+Classifier)",
                                           early_stopping_patience=3)

# Save Stage 1
save_dir = r"C:\Users\sajja\Desktop\Dissertation\Classification Models\Fine Tuning\EfficientNet"
os.makedirs(save_dir, exist_ok=True)
model_path_stage1 = os.path.join(save_dir, "efficientnet_stage1.pth")
history_path_stage1 = os.path.join(save_dir, "history_efficientnet_stage1.pkl")

torch.save(model_stage1.state_dict(), model_path_stage1)
with open(history_path_stage1, "wb") as f:
    pickle.dump(history_stage1, f)

```

```

"""Output of EfficientNet Stage 1:
Stage 1 - Unfrozen layers:
- features.7.0.block.0.0.weight
- features.7.0.block.0.1.weight
- features.7.0.block.0.1.bias
- features.7.0.block.1.0.weight
- features.7.0.block.1.1.weight
- features.7.0.block.1.1.bias
- features.7.0.block.2.fc1.weight
- features.7.0.block.2.fc1.bias
- features.7.0.block.2.fc2.weight
- features.7.0.block.2.fc2.bias
- features.7.0.block.3.0.weight
- features.7.0.block.3.1.weight
- features.7.0.block.3.1.bias
- features.7.1.block.0.0.weight
- features.7.1.block.0.1.weight
- features.7.1.block.0.1.bias
- features.7.1.block.1.0.weight
- features.7.1.block.1.1.weight
- features.7.1.block.1.1.bias
- features.7.1.block.2.fc1.weight
- features.7.1.block.2.fc1.bias
- features.7.1.block.2.fc2.weight
- features.7.1.block.2.fc2.bias
- features.7.1.block.3.0.weight
- features.7.1.block.3.1.weight
- features.7.1.block.3.1.bias
- classifier.1.weight
- classifier.1.bias
Training EfficientNet-B3 Stage 1 (Last 2 Blocks+Classifier)
Early stopping patience: 3 epochs
=====
Epoch 1/15
-----
train: 100%|██████████| 1865/1865 [31:05<00:00, 1.00s/it, loss=0.9590, acc=3.0865]
train Loss: 0.8145 Acc: 0.6755
val: 100%|██████████| 626/626 [08:26<00:00, 1.24it/s, loss=1.3811, acc=0.8802]
val Loss: 0.6461 Acc: 0.7428
New best model with Validation accuracy of 0.7428

Epoch 2/15
-----
train: 100%|██████████| 1865/1865 [30:12<00:00, 1.03it/s, loss=0.9738, acc=3.3634]
train Loss: 0.6800 Acc: 0.7360
val: 100%|██████████| 626/626 [08:20<00:00, 1.25it/s, loss=1.2174, acc=0.9088]
val Loss: 0.6008 Acc: 0.7670
New best model with Validation accuracy of 0.7670

Epoch 3/15
-----
train: 100%|██████████| 1865/1865 [29:44<00:00, 1.05it/s, loss=0.6914, acc=3.4409]
train Loss: 0.6391 Acc: 0.7530
val: 100%|██████████| 626/626 [08:15<00:00, 1.26it/s, loss=0.8660, acc=0.9404]
val Loss: 0.5389 Acc: 0.7937
New best model with Validation accuracy of 0.7937

```

Epoch 4/15

```

-----
train: 100%|██████████| 1865/1865 [30:13<00:00, 1.03it/s, loss=0.9349, acc=3.4985]
train Loss: 0.6113 Acc: 0.7656
val: 100%|██████████| 626/626 [08:23<00:00, 1.24it/s, loss=1.0257, acc=0.9368]
val Loss: 0.5560 Acc: 0.7907
No improvement for 1 epoch(s)

```

Epoch 5/15

```

-----
train: 100%|██████████| 1865/1865 [29:38<00:00, 1.05it/s, loss=0.5067, acc=3.5398]
train Loss: 0.5923 Acc: 0.7747
val: 100%|██████████| 626/626 [08:17<00:00, 1.26it/s, loss=0.9692, acc=0.9239]
val Loss: 0.5608 Acc: 0.7797
No improvement for 2 epoch(s)

```

Classification Report:

	precision	recall	f1-score	support
no-damage	0.97	0.78	0.87	28852
minor-damage	0.45	0.75	0.56	4715
major-damage	0.49	0.76	0.59	3840
destroyed	0.73	0.82	0.77	2647
accuracy			0.78	40054
macro avg	0.66	0.78	0.70	40054
weighted avg	0.85	0.78	0.80	40054

Epoch 6/15

```

-----
train: 100%|██████████| 1865/1865 [29:39<00:00, 1.05it/s, loss=0.3616, acc=3.5497]
train Loss: 0.5806 Acc: 0.7768
val: 100%|██████████| 626/626 [08:17<00:00, 1.26it/s, loss=0.8689, acc=0.9250]val Loss: 0.5543 Acc: 0.7807
No improvement for 3 epoch(s)

```

Early stopping triggered after 6 epochs

No improvement in validation accuracy for 3 consecutive epochs

Training complete in 230m 35s

Training stopped early at epoch 6

Best val acc: 0.7937 (epoch 3)

"""

EfficientNet Fine-tuning: Stage 2: Last Stage (features[7]) + Features[6] + Classifier

Load Stage 1 model

model = efficientnet_b3(weights=EfficientNet_B3_Weights.IMAGENET1K_V1)

model.classifier[1] = nn.Linear(model.classifier[1].in_features, 4)

model.load_state_dict(torch.load("/content/drive/MyDrive/Building Damage Assessment/Classification models/EfficientNet Fine Tuning/efficientnet_stage1.pth"))

model = model.to(device)

Unfreeze entire features[7] (last stage) + features[6]

for param in model.features[6].parameters():

param.requires_grad = True

for param in model.features[7].parameters(): # Last stage (already unfrozen from Stage 1)

param.requires_grad = True

```

for param in model.classifier.parameters(): # Classifier (already unfrozen)
    param.requires_grad = True

# Verify unfrozen parameters
print("Unfrozen layers:")
unfrozen_layers = []
for name, param in model.named_parameters():
    if param.requires_grad:
        unfrozen_layers.append(name)
print(f"Total unfrozen layers: {len(unfrozen_layers)}")

# Loss function
criterion = nn.CrossEntropyLoss(weight=class_weights.to(device))

# Optimizer with discriminative learning rates
optimizer = optim.Adam([
    {'params': model.features[6].parameters(), 'lr': 0.0002, 'weight_decay': 5e-5}, # Lower LR for earlier stage
    {'params': model.features[7].parameters(), 'lr': 0.0003, 'weight_decay': 1e-4}, # Same as Stage 1
    {'params': model.classifier.parameters(), 'lr': 0.0005, 'weight_decay': 1e-4} # Highest LR for classifier
])

# Learning rate scheduler
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=3)

# Train Stage 2
model_stage2, history_stage2 = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler,
                                           num_epochs=15, model_name="EfficientNet-B3 Stage 2 (Stage 6+7+Classifier)",
                                           early_stopping_patience=3)

# Save Stage 2
save_dir = "/content/drive/MyDrive/Building Damage Assessment/Classification models/EfficientNet Fine Tuning"
model_path_stage2 = os.path.join(save_dir, "efficientnet_stage2.pth")
history_path_stage2 = os.path.join(save_dir, "history_efficientnet_stage2.pkl")

torch.save(model_stage2.state_dict(), model_path_stage2)
with open(history_path_stage2, "wb") as f:
    pickle.dump(history_stage2, f)

""" Further Fine Tuning: EfficientNet Stage 2 Output Results
Downloading: "https://download.pytorch.org/models/efficientnet_b3_rwightman-b3899882.pth" to /root/.cache/torch/hub/checkpoints/efficientnet_b3_rwightman-b3899882.pth
100%|██████████| 47.2M/47.2M [00:00<00:00, 134MB/s]
Unfrozen layers:
Total unfrozen layers: 340
Training EfficientNet-B3 Stage 2 (Stage 6+7+Classifier)
Early stopping patience: 3 epochs
=====
Epoch 1/15
-----
train: 100%|██████████| 1865/1865 [26:51<00:00, 1.16it/s, loss=0.3934, acc=3.5274]
train Loss: 0.6010 Acc: 0.7719
val: 100%|██████████| 626/626 [05:05<00:00, 2.05it/s, loss=0.7601, acc=0.9129]
val Loss: 0.5418 Acc: 0.7704
New best model with Validation accuracy of 0.7704

Epoch 2/15
-----

```



```
train: 100%|██████████| 1865/1865 [27:36<00:00, 1.13it/s, loss=1.3992, acc=3.6459]
train Loss: 0.5462 Acc: 0.7979
val: 100%|██████████| 626/626 [05:12<00:00, 2.00it/s, loss=0.9509, acc=0.9123]
val Loss: 0.5436 Acc: 0.7699
No improvement for 1 epoch(s)
```

Epoch 3/15

```
train: 100%|██████████| 1865/1865 [29:17<00:00, 1.06it/s, loss=0.3564, acc=3.6962]
train Loss: 0.5094 Acc: 0.8089
val: 100%|██████████| 626/626 [04:58<00:00, 2.10it/s, loss=0.7638, acc=0.9491]
val Loss: 0.5134 Acc: 0.8010
New best model with Validation accuracy of 0.8010
```

Epoch 4/15

```
train: 100%|██████████| 1865/1865 [28:33<00:00, 1.09it/s, loss=0.4920, acc=3.7410]
train Loss: 0.4756 Acc: 0.8187
val: 100%|██████████| 626/626 [04:56<00:00, 2.11it/s, loss=0.8567, acc=0.9316]
val Loss: 0.5288 Acc: 0.7863
No improvement for 1 epoch(s)
```

Epoch 5/15

```
train: 100%|██████████| 1865/1865 [28:41<00:00, 1.08it/s, loss=0.3753, acc=3.7873]
train Loss: 0.4504 Acc: 0.8288
val: 100%|██████████| 626/626 [05:03<00:00, 2.06it/s, loss=0.9123, acc=0.9501]
val Loss: 0.5094 Acc: 0.8018
New best model with Validation accuracy of 0.8018
```

Classification Report:

	precision	recall	f1-score	support
no-damage	0.97	0.82	0.89	28852
minor-damage	0.45	0.74	0.55	4715
major-damage	0.56	0.74	0.64	3840
destroyed	0.75	0.86	0.80	2647
accuracy			0.80	40054
macro avg	0.68	0.79	0.72	40054
weighted avg	0.86	0.80	0.82	40054

Epoch 6/15

```
train: 100%|██████████| 1865/1865 [28:40<00:00, 1.08it/s, loss=0.3875, acc=3.8213]
train Loss: 0.4202 Acc: 0.8363
val: 100%|██████████| 626/626 [05:06<00:00, 2.04it/s, loss=0.8460, acc=0.9443]
val Loss: 0.5313 Acc: 0.7969
No improvement for 1 epoch(s)
```

Epoch 7/15

```
train: 100%|██████████| 1865/1865 [29:09<00:00, 1.07it/s, loss=0.2854, acc=3.8478]
train Loss: 0.3952 Acc: 0.8421
val: 100%|██████████| 626/626 [05:10<00:00, 2.02it/s, loss=0.7100, acc=0.9521]
```

val Loss: 0.5673 Acc: 0.8035
 New best model with Validation accuracy of 0.8035

Epoch 8/15

 train: 100%|██████████| 1865/1865 [29:06<00:00, 1.07it/s, loss=0.0777, acc=3.8817]
 train Loss: 0.3767 Acc: 0.8495
 val: 100%|██████████| 626/626 [05:05<00:00, 2.05it/s, loss=0.8474, acc=0.9545]
 val Loss: 0.5879 Acc: 0.8056
 New best model with Validation accuracy of 0.8056

Epoch 9/15

 train: 100%|██████████| 1865/1865 [29:00<00:00, 1.07it/s, loss=0.4178, acc=3.9150]
 train Loss: 0.3512 Acc: 0.8568
 val: 100%|██████████| 626/626 [05:05<00:00, 2.05it/s, loss=1.3693, acc=0.9740]
 val Loss: 0.5761 Acc: 0.8220
 New best model with Validation accuracy of 0.8220

Epoch 10/15

 train: 100%|██████████| 1865/1865 [28:47<00:00, 1.08it/s, loss=0.3130, acc=4.0090]
 train Loss: 0.2846 Acc: 0.8773
 val: 100%|██████████| 626/626 [05:38<00:00, 1.85it/s, loss=1.3455, acc=0.9750]
 val Loss: 0.5920 Acc: 0.8229
 New best model with Validation accuracy of 0.8229

Classification Report:

	precision	recall	f1-score	support
no-damage	0.97	0.86	0.91	28852
minor-damage	0.50	0.68	0.57	4715
major-damage	0.56	0.73	0.64	3840
destroyed	0.73	0.86	0.79	2647
accuracy			0.82	40054
macro avg	0.69	0.78	0.73	40054
weighted avg	0.86	0.82	0.83	40054

Epoch 11/15

 train: 100%|██████████| 1865/1865 [29:23<00:00, 1.06it/s, loss=0.3471, acc=4.0496]
 train Loss: 0.2556 Acc: 0.8862
 val: 100%|██████████| 626/626 [05:05<00:00, 2.05it/s, loss=1.4395, acc=0.9877]
 val Loss: 0.6082 Acc: 0.8335
 New best model with Validation accuracy of 0.8335

Epoch 12/15

 train: 100%|██████████| 1865/1865 [29:32<00:00, 1.05it/s, loss=0.0283, acc=4.0761]
 train Loss: 0.2425 Acc: 0.8920
 val: 100%|██████████| 626/626 [04:52<00:00, 2.14it/s, loss=1.5257, acc=0.9893]
 val Loss: 0.6259 Acc: 0.8349
 New best model with Validation accuracy of 0.8349

```
Epoch 13/15
-----
train: 100%|██████████| 1865/1865 [29:03<00:00, 1.07it/s, loss=0.3577, acc=4.0829]
train Loss: 0.2330 Acc: 0.8935
val: 100%|██████████| 626/626 [04:55<00:00, 2.12it/s, loss=1.4529, acc=0.9871]
val Loss: 0.6369 Acc: 0.8331
No improvement for 1 epoch(s)
```

```
Epoch 14/15
-----
train: 100%|██████████| 1865/1865 [28:54<00:00, 1.07it/s, loss=0.0509, acc=4.1083]
train Loss: 0.2180 Acc: 0.8991
val: 100%|██████████| 626/626 [04:52<00:00, 2.14it/s, loss=1.5080, acc=0.9907]
val Loss: 0.6476 Acc: 0.8361
New best model with Validation accuracy of 0.8361
```

```
Epoch 15/15
-----
train: 100%|██████████| 1865/1865 [28:54<00:00, 1.08it/s, loss=0.1580, acc=4.1085]
train Loss: 0.2152 Acc: 0.8991
val: 100%|██████████| 626/626 [04:56<00:00, 2.11it/s, loss=1.6010, acc=0.9917]
val Loss: 0.6538 Acc: 0.8369
New best model with Validation accuracy of 0.8369
```

Classification Report:

	precision	recall	f1-score	support
no-damage	0.96	0.88	0.92	28852
minor-damage	0.52	0.67	0.58	4715
major-damage	0.59	0.71	0.65	3840
destroyed	0.77	0.85	0.81	2647
accuracy			0.84	40054
macro avg	0.71	0.78	0.74	40054
weighted avg	0.86	0.84	0.85	40054

Training complete in 507m 42s
Best val acc: 0.8369 (epoch 15)
"""

```
# EfficientNet Fine Tuning: Stage 3: Unfreeze features[5] + features[6] + features[7] + Classifier
# Load Stage 2 model
model = efficientnet_b3(weights=EfficientNet_B3_Weights.IMAGENET1K_V1)
model.classifier[1] = nn.Linear(model.classifier[1].in_features, 4)
model.load_state_dict(torch.load(
    r"C:\Users\sajja\Desktop\Disertation\Classification Models\Fine Tuning\Efficientnet\efficientnet_stage2.pth"))
model = model.to(device)
# Unfreeze features[5] as well
for p in model.parameters():
    p.requires_grad = False
for param in model.features[5].parameters(): # Third-to-last stage
    param.requires_grad = True
for param in model.features[6].parameters(): # Second-to-last stage (already unfrozen)
    param.requires_grad = True
for param in model.features[7].parameters(): # Last stage (already unfrozen)
```

```

    param.requires_grad = True
for param in model.classifier.parameters(): # Classifier (already unfrozen)
    param.requires_grad = True

# Loss function
criterion = nn.CrossEntropyLoss(weight=class_weights.to(device))

# Optimizer with discriminative learning rates
optimizer = optim.Adam([
    {'params': model.features[5].parameters(), 'lr': 0.0001, 'weight_decay': 5e-5}, # Lowest LR for earliest stage
    {'params': model.features[6].parameters(), 'lr': 0.0002, 'weight_decay': 5e-5}, # Medium LR
    {'params': model.features[7].parameters(), 'lr': 0.0003, 'weight_decay': 1e-4}, # Higher LR
    {'params': model.classifier.parameters(), 'lr': 0.0005, 'weight_decay': 1e-4} # Highest LR for classifier
])

# Learning rate scheduler
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=3)

# Train Stage 3
model_stage3, history_stage3 = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler,
                                           num_epochs=15, model_name="EfficientNet-B3 Stage 3 (Stage 5+6+7+Classifier)",
                                           early_stopping_patience=4)

# Save Stage 3
save_dir = r"C:\Users\sajja\Desktop\Dissertation\Classification Models\Fine Tuning\EfficientNet"
os.makedirs(save_dir, exist_ok=True)
model_path_stage3 = os.path.join(save_dir, "efficientnet_stage3.pth")
history_path_stage3 = os.path.join(save_dir, "history_efficientnet_stage3.pkl")

torch.save(model_stage3.state_dict(), model_path_stage3)
with open(history_path_stage3, "wb") as f:
    pickle.dump(history_stage3, f)

""" EfficientNet Stage 3 Output Results:
Training EfficientNet-B3 Stage 3 (Stage 5+6+7+Classifier)
Early stopping patience: 4 epochs
=====
Epoch 1/15
-----
train: 100%|██████████| 1865/1865 [28:54<00:00, 1.07it/s, loss=0.4387, acc=3.9763]
train Loss: 0.3033 Acc: 0.8702
val: 100%|██████████| 626/626 [05:05<00:00, 2.05it/s, loss=0.9687, acc=0.9607]
val Loss: 0.6243 Acc: 0.8108
New best model with Validation accuracy of 0.8108

Epoch 2/15
-----
train: 100%|██████████| 1865/1865 [30:14<00:00, 1.03it/s, loss=0.1230, acc=4.0045]
train Loss: 0.2850 Acc: 0.8763
val: 100%|██████████| 626/626 [05:21<00:00, 1.95it/s, loss=0.8303, acc=0.9741]
val Loss: 0.6033 Acc: 0.8221
New best model with Validation accuracy of 0.8221

Epoch 3/15
-----
train: 100%|██████████| 1865/1865 [29:24<00:00, 1.06it/s, loss=0.0953, acc=4.0241]

```

train Loss: 0.2744 Acc: 0.8806
val: 100%|██████████| 626/626 [05:01<00:00, 2.07it/s, loss=0.8646, acc=0.9742]
val Loss: 0.6652 Acc: 0.8222
New best model with Validation accuracy of 0.8222

Epoch 4/15

train: 100%|██████████| 1865/1865 [29:10<00:00, 1.07it/s, loss=0.3435, acc=4.0599]
train Loss: 0.2507 Acc: 0.8885
val: 100%|██████████| 626/626 [04:59<00:00, 2.09it/s, loss=0.8087, acc=0.9821]
val Loss: 0.6425 Acc: 0.8289
New best model with Validation accuracy of 0.8289

Epoch 5/15

train: 100%|██████████| 1865/1865 [29:10<00:00, 1.07it/s, loss=1.4135, acc=4.0812]
train Loss: 0.2356 Acc: 0.8931
val: 100%|██████████| 626/626 [05:05<00:00, 2.05it/s, loss=1.3316, acc=0.9874]
val Loss: 0.6835 Acc: 0.8333
New best model with Validation accuracy of 0.8333

Classification Report:

	precision	recall	f1-score	support
no-damage	0.96	0.87	0.92	28852
minor-damage	0.52	0.66	0.58	4715
major-damage	0.56	0.75	0.64	3840
destroyed	0.79	0.84	0.81	2647
accuracy			0.83	40054
macro avg	0.71	0.78	0.74	40054
weighted avg	0.86	0.83	0.84	40054

Epoch 6/15

train: 100%|██████████| 1865/1865 [29:55<00:00, 1.04it/s, loss=0.2508, acc=4.1067]
train Loss: 0.2182 Acc: 0.8987
val: 100%|██████████| 626/626 [05:16<00:00, 1.98it/s, loss=1.3192, acc=0.9863]
val Loss: 0.6941 Acc: 0.8324
No improvement for 1 epoch(s)

Epoch 7/15

train: 100%|██████████| 1865/1865 [29:03<00:00, 1.07it/s, loss=0.1025, acc=4.2107]
train Loss: 0.1577 Acc: 0.9215
val: 100%|██████████| 626/626 [04:57<00:00, 2.11it/s, loss=1.2909, acc=0.9945]
val Loss: 0.7571 Acc: 0.8393
New best model with Validation accuracy of 0.8393

Epoch 8/15

train: 100%|██████████| 1865/1865 [29:03<00:00, 1.07it/s, loss=0.4156, acc=4.2544]
train Loss: 0.1330 Acc: 0.9310
val: 100%|██████████| 626/626 [05:02<00:00, 2.07it/s, loss=1.4197, acc=1.0023]
val Loss: 0.8181 Acc: 0.8459

New best model with Validation accuracy of 0.8459

Epoch 9/15

```
-----
train: 100%|██████████| 1865/1865 [29:29<00:00, 1.05it/s, loss=0.1520, acc=4.2835]
train Loss: 0.1186 Acc: 0.9374
val: 100%|██████████| 626/626 [05:00<00:00, 2.08it/s, loss=1.5850, acc=1.0060]
val Loss: 0.8251 Acc: 0.8490
New best model with Validation accuracy of 0.8490
```

Epoch 10/15

```
-----
train: 100%|██████████| 1865/1865 [26:51<00:00, 1.16it/s, loss=0.2493, acc=4.2959]
train Loss: 0.1106 Acc: 0.9401
val: 100%|██████████| 626/626 [04:50<00:00, 2.15it/s, loss=1.5869, acc=1.0046]
val Loss: 0.8536 Acc: 0.8479
No improvement for 1 epoch(s)
```

Classification Report:

	precision	recall	f1-score	support
no-damage	0.95	0.90	0.93	28852
minor-damage	0.54	0.64	0.58	4715
major-damage	0.61	0.72	0.66	3840
destroyed	0.83	0.83	0.83	2647
accuracy			0.85	40054
macro avg	0.73	0.77	0.75	40054
weighted avg	0.86	0.85	0.85	40054

Epoch 11/15

```
-----
train: 100%|██████████| 1865/1865 [26:43<00:00, 1.16it/s, loss=0.0652, acc=4.3070]
train Loss: 0.1045 Acc: 0.9426
val: 100%|██████████| 626/626 [04:50<00:00, 2.16it/s, loss=1.4707, acc=1.0054]
val Loss: 0.8624 Acc: 0.8486
No improvement for 2 epoch(s)
```

Epoch 12/15

```
-----
train: 100%|██████████| 1865/1865 [26:22<00:00, 1.18it/s, loss=0.0676, acc=4.3166]
train Loss: 0.1013 Acc: 0.9447
val: 100%|██████████| 626/626 [04:53<00:00, 2.14it/s, loss=1.6309, acc=1.0120]
val Loss: 0.8981 Acc: 0.8540
New best model with Validation accuracy of 0.8540
```

Epoch 13/15

```
-----
train: 100%|██████████| 1865/1865 [26:48<00:00, 1.16it/s, loss=0.2465, acc=4.3162]
train Loss: 0.1002 Acc: 0.9446
val: 100%|██████████| 626/626 [04:52<00:00, 2.14it/s, loss=1.6366, acc=1.0087]
val Loss: 0.8818 Acc: 0.8513
No improvement for 1 epoch(s)
```

Epoch 14/15

```

-----
train: 100%|██████████| 1865/1865 [27:26<00:00, 1.13it/s, loss=0.0052, acc=4.3162]
train Loss: 0.0999 Acc: 0.9446
val: 100%|██████████| 626/626 [04:59<00:00, 2.09it/s, loss=1.5024, acc=1.0077]
val Loss: 0.8883 Acc: 0.8505
No improvement for 2 epoch(s)

```

Epoch 15/15

```

-----
train: 100%|██████████| 1865/1865 [27:16<00:00, 1.14it/s, loss=0.4197, acc=4.3203]
train Loss: 0.0975 Acc: 0.9455
val: 100%|██████████| 626/626 [04:46<00:00, 2.19it/s, loss=1.6170, acc=1.0089]
val Loss: 0.8829 Acc: 0.8515
No improvement for 3 epoch(s)

```

Classification Report:

	precision	recall	f1-score	support
no-damage	0.95	0.91	0.93	28852
minor-damage	0.55	0.63	0.58	4715
major-damage	0.61	0.71	0.66	3840
destroyed	0.84	0.83	0.84	2647
accuracy			0.85	40054
macro avg	0.74	0.77	0.75	40054
weighted avg	0.86	0.85	0.86	40054

Training complete in 501m 1s
 Best val acc: 0.8540 (epoch 12)"""

✓ ViT-B/16

From the ablation study, we observe that Cross Entropy without class weights adjustment performed the best. Focal Loss (without weight) also performed really well (almost identical performance to Cross Entropy), however a more balanced performance across the classes is observed. We further fine tune the ViT architecture through different stages. We may adjust the attention blocks if overfitting is observed.

```

from torchvision.models import vit_b_16, ViT_B_16_Weights

# ViT Fine-tuning (Stage 1: Last Transformer Block + Head)
# Load pretrained ViT-B/16
model = vit_b_16(weights=ViT_B_16_Weights.IMAGENET1K_V1)

# Freeze all parameters
for param in model.parameters():
    param.requires_grad = False

# Replace classification head for 4 classes
model.heads.head = nn.Linear(model.heads.head.in_features, 4)

# Unfreeze last transformer block + classification head
for param in model.encoder.layers[-1].parameters(): # Last transformer block

```

```

    param.requires_grad = True
for param in model.heads.parameters(): # Classification head
    param.requires_grad = True

model = model.to(device)

# Loss function
criterion = nn.CrossEntropyLoss(weight=class_weights.to(device))# Loss function

# Optimizer with weight decay and discriminative learning rates
optimizer = optim.Adam([
    {'params': model.encoder.layers[-1].parameters(), 'lr': 0.0003, 'weight_decay': 1e-4},
    {'params': model.heads.parameters(), 'lr': 0.0005, 'weight_decay': 1e-4}
])

# Learning rate scheduler
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=3)

# Train Stage 1
model_stage1, history_stage1 = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler,
                                           num_epochs=15, model_name="ViT-B/16 Stage 1 (Last Block+Head)",
                                           early_stopping_patience=3)

# Save Stage 1
save_dir = "/content/drive/MyDrive/Building Damage Assessment/Classification models/ViT Fine Tuning"
os.makedirs(save_dir, exist_ok=True)
model_path_stage1 = os.path.join(save_dir, "vit_stage1.pth")
history_path_stage1 = os.path.join(save_dir, "history_vit_stage1.pkl")

torch.save(model_stage1.state_dict(), model_path_stage1)
with open(history_path_stage1, "wb") as f:
    pickle.dump(history_stage1, f)

"""Output of ViT Stage 1:
Training ViT-B/16 Stage 1 (Last Block+Head)
Early stopping patience: 3 epochs
=====
Epoch 1/15
-----
train: 100%|██████████| 3729/3729 [1:02:20<00:00, 1.00s/it, loss=1.0661, acc=1.9309]
train Loss: 0.4183 Acc: 0.8449
val: 100%|██████████| 1252/1252 [14:13<00:00, 1.47it/s, loss=0.2719, acc=1.2394]
val Loss: 0.4005 Acc: 0.8523
New best model with Validation accuracy of 0.8523

Epoch 2/15
-----
train: 100%|██████████| 3729/3729 [1:01:33<00:00, 1.01it/s, loss=0.4131, acc=1.9889]
train Loss: 0.3464 Acc: 0.8703
val: 100%|██████████| 1252/1252 [14:27<00:00, 1.44it/s, loss=0.3717, acc=1.2486]
val Loss: 0.3808 Acc: 0.8586
New best model with Validation accuracy of 0.8586

Epoch 3/15
-----
train: 100%|██████████| 3729/3729 [1:01:43<00:00, 1.01it/s, loss=0.2602, acc=2.0066]

```



```
train Loss: 0.3247 Acc: 0.8780
val: 100%|██████████| 1252/1252 [14:10<00:00, 1.47it/s, loss=0.2812, acc=1.2461]
val Loss: 0.3914 Acc: 0.8569
No improvement for 1 epoch(s)

Epoch 4/15
-----
train: 100%|██████████| 3729/3729 [1:01:52<00:00, 1.00it/s, loss=0.2770, acc=2.0144]
train Loss: 0.3127 Acc: 0.8814
val: 100%|██████████| 1252/1252 [14:15<00:00, 1.46it/s, loss=0.2448, acc=1.2601]
val Loss: 0.3682 Acc: 0.8666
New best model with Validation accuracy of 0.8666

Epoch 5/15
-----
train: 100%|██████████| 3729/3729 [1:02:08<00:00, 1.00it/s, loss=0.5372, acc=2.0243]
train Loss: 0.3007 Acc: 0.8858
val: 100%|██████████| 1252/1252 [14:18<00:00, 1.46it/s, loss=0.3086, acc=1.2618]
val Loss: 0.3514 Acc: 0.8677
New best model with Validation accuracy of 0.8677

Classification Report:
      precision    recall  f1-score   support

no-damage      0.91      0.96      0.93      28852
minor-damage    0.67      0.51      0.58       4715
major-damage    0.68      0.67      0.68       3840
destroyed       0.90      0.80      0.85       2647

   accuracy
macro avg    0.79      0.74      0.76      40054
weighted avg 0.86      0.87      0.86      40054

Epoch 6/15
-----
train: 100%|██████████| 3729/3729 [1:01:52<00:00, 1.00it/s, loss=0.4146, acc=2.0308]
train Loss: 0.2922 Acc: 0.8886
val: 100%|██████████| 1252/1252 [14:07<00:00, 1.48it/s, loss=0.3258, acc=1.2509]
val Loss: 0.3692 Acc: 0.8602
No improvement for 1 epoch(s)

Epoch 7/15
-----
train: 100%|██████████| 3729/3729 [1:01:44<00:00, 1.01it/s, loss=0.3972, acc=2.0357]
train Loss: 0.2857 Acc: 0.8908
val: 100%|██████████| 1252/1252 [14:09<00:00, 1.47it/s, loss=0.2708, acc=1.2591]
val Loss: 0.3670 Acc: 0.8659
No improvement for 2 epoch(s)

Epoch 8/15
-----
train: 100%|██████████| 3729/3729 [1:01:38<00:00, 1.01it/s, loss=0.0376, acc=2.0406]
train Loss: 0.2780 Acc: 0.8929
val: 100%|██████████| 1252/1252 [14:07<00:00, 1.48it/s, loss=0.3996, acc=1.2646]
val Loss: 0.3631 Acc: 0.8697
```

New best model with Validation accuracy of 0.8697

Epoch 9/15

```
-----
train: 100%|██████████| 3729/3729 [1:01:32<00:00, 1.01it/s, loss=0.6623, acc=2.0472]
train Loss: 0.2727 Acc: 0.8958
val: 100%|██████████| 1252/1252 [14:02<00:00, 1.49it/s, loss=0.3109, acc=1.2621]
val Loss: 0.3648 Acc: 0.8679
No improvement for 1 epoch(s)
```

Epoch 10/15

```
-----
train: 100%|██████████| 3729/3729 [1:03:41<00:00, 1.02s/it, loss=0.2534, acc=2.0830]
train Loss: 0.2288 Acc: 0.9115
val: 100%|██████████| 1252/1252 [15:01<00:00, 1.39it/s, loss=0.3438, acc=1.2631]
val Loss: 0.3661 Acc: 0.8686
No improvement for 2 epoch(s)
```

Classification Report:

	precision	recall	f1-score	support
no-damage	0.93	0.95	0.94	28852
minor-damage	0.66	0.55	0.60	4715
major-damage	0.65	0.71	0.68	3840
destroyed	0.89	0.80	0.84	2647
accuracy			0.87	40054
macro avg	0.78	0.75	0.77	40054
weighted avg	0.87	0.87	0.87	40054

Epoch 11/15

```
-----
train: 100%|██████████| 3729/3729 [1:06:10<00:00, 1.06s/it, loss=0.3818, acc=2.0933]
train Loss: 0.2163 Acc: 0.9160
val: 100%|██████████| 1252/1252 [14:07<00:00, 1.48it/s, loss=0.3309, acc=1.2633]
val Loss: 0.3760 Acc: 0.8688
No improvement for 3 epoch(s)
```

Early stopping triggered after 11 epochs

No improvement in validation accuracy for 3 consecutive epochs

Training complete in 843m 19s

Training stopped early at epoch 11

Best val acc: 0.8697 (epoch 8)"""

ViT Fine-tuning (Stage 2: Last 3 Transformer Blocks + Head)

Load Stage 1 model

model = vit_b_16(weights=ViT_B_16_Weights.IMAGENET1K_V1)

model.heads.head = nn.Linear(model.heads.head.in_features, 4)

model.load_state_dict(torch.load("/content/drive/MyDrive/Building Damage Assessment/Classification models/ViT Fine Tuning/vit_stage1.pth"))

model = model.to(device)

Unfreeze last 3 transformer blocks (building on Stage 1)

for param in model.encoder.layers[-3:].parameters(): # Last 3 blocks

param.requires_grad = True

for param in model.heads.parameters(): # Keep head unfrozen

```

param.requires_grad = True

# Verify unfrozen layers
print("Unfrozen layers in Stage 2:")
for name, param in model.named_parameters():
    if param.requires_grad:
        print(f"- {name}")

# Loss function
criterion = nn.CrossEntropyLoss(weight=class_weights.to(device))

# Optimizer with discriminative learning rates
optimizer = optim.Adam([
    {'params': model.encoder.layers[-3].parameters(), 'lr': 0.0001, 'weight_decay': 5e-5}, # Lowest LR for earliest block
    {'params': model.encoder.layers[-2].parameters(), 'lr': 0.0002, 'weight_decay': 1e-4}, # Medium LR
    {'params': model.encoder.layers[-1].parameters(), 'lr': 0.0003, 'weight_decay': 1e-4}, # Same as Stage 1
    {'params': model.heads.parameters(), 'lr': 0.0005, 'weight_decay': 1e-4} # Highest LR for head
])

# Learning rate scheduler
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=3)

# Train Stage 2
model_stage2, history_stage2 = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler,
                                           num_epochs=8, model_name="ViT-B/16 Stage 2 (Last 3 Blocks+Head)",
                                           early_stopping_patience=3)

# Save Stage 2
save_dir = "/content/drive/MyDrive/Building Damage Assessment/Classification models/ViT Fine Tuning"
model_path_stage2 = os.path.join(save_dir, "vit_stage2.pth")
history_path_stage2 = os.path.join(save_dir, "history_vit_stage2.pkl")

torch.save(model_stage2.state_dict(), model_path_stage2)
with open(history_path_stage2, "wb") as f:
    pickle.dump(history_stage2, f)


```

 Downloading: "https://download.pytorch.org/models/vit_b_16-c867db91.pth" to /root/.cache/torch/hub/checkpoints/vit_b_16-c867db91.pth
 100%|██████████| 330M/330M [00:01<00:00, 178MB/s]

Unfrozen layers in Stage 2:

- class_token
- conv_proj.weight
- conv_proj.bias
- encoder.pos_embedding
- encoder.layers.encoder_layer_0.ln_1.weight
- encoder.layers.encoder_layer_0.ln_1.bias
- encoder.layers.encoder_layer_0.self_attention.in_proj_weight
- encoder.layers.encoder_layer_0.self_attention.in_proj_bias
- encoder.layers.encoder_layer_0.self_attention.out_proj.weight
- encoder.layers.encoder_layer_0.self_attention.out_proj.bias
- encoder.layers.encoder_layer_0.ln_2.weight
- encoder.layers.encoder_layer_0.ln_2.bias
- encoder.layers.encoder_layer_0.mlp.0.weight
- encoder.layers.encoder_layer_0.mlp.0.bias
- encoder.layers.encoder_layer_0.mlp.3.weight
- encoder.layers.encoder_layer_0.mlp.3.bias
- encoder.layers.encoder_layer_1.ln_1.weight
- encoder.layers.encoder_layer_1.ln_1.bias
- encoder.layers.encoder_layer_1.self_attention.in_proj_weight

```

- encoder.layers.encoder_layer_1.self_attention.in_proj_bias
- encoder.layers.encoder_layer_1.self_attention.out_proj.weight
- encoder.layers.encoder_layer_1.self_attention.out_proj.bias
- encoder.layers.encoder_layer_1.ln_2.weight
- encoder.layers.encoder_layer_1.ln_2.bias
- encoder.layers.encoder_layer_1.mlp.0.weight
- encoder.layers.encoder_layer_1.mlp.0.bias
- encoder.layers.encoder_layer_1.mlp.3.weight
- encoder.layers.encoder_layer_1.mlp.3.bias
- encoder.layers.encoder_layer_2.ln_1.weight
- encoder.layers.encoder_layer_2.ln_1.bias
- encoder.layers.encoder_layer_2.self_attention.in_proj_weight
- encoder.layers.encoder_layer_2.self_attention.in_proj_bias
- encoder.layers.encoder_layer_2.self_attention.out_proj.weight
- encoder.layers.encoder_layer_2.self_attention.out_proj.bias
- encoder.layers.encoder_layer_2.ln_2.weight
- encoder.layers.encoder_layer_2.ln_2.bias
- encoder.layers.encoder_layer_2.mlp.0.weight
- encoder.layers.encoder_layer_2.mlp.0.bias
- encoder.layers.encoder_layer_2.mlp.3.weight
- encoder.layers.encoder_layer_2.mlp.3.bias
- encoder.layers.encoder_layer_3.ln_1.weight
- encoder.layers.encoder_layer_3.ln_1.bias
- encoder.layers.encoder_layer_3.self_attention.in_proj_weight
- encoder.layers.encoder_layer_3.self_attention.in_proj_bias
- encoder.layers.encoder_layer_3.self_attention.out_proj.weight
- encoder.layers.encoder_layer_3.self_attention.out_proj.bias
- encoder.layers.encoder_layer_3.ln_2.weight
- encoder.layers.encoder_layer_3.ln_2.bias
- encoder.layers.encoder_layer_3.mlp.0.weight
- encoder.layers.encoder_layer_3.mlp.0.bias
- encoder.layers.encoder_layer_3.mlp.3.weight
- encoder.layers.encoder_layer_3.mlp.3.bias
- encoder.layers.encoder_layer_4.ln_1.weight
- encoder.layers.encoder_layer_4.ln_1.bias
- encoder.layers.encoder_layer_4.self_attention.in_proj_weight

```

In Stage 3, we unfreeze the patch embedding layer along with the last 6 transformer blocks to address the significant domain gap between ImageNet's natural images and xBD satellite imagery. The patch embedding layer is crucial for adapting to satellite-specific visual characteristics such as different spectral properties, lighting conditions, and textural patterns that are fundamentally different from natural images. By extending from 3 to 6 transformer blocks, we provide the model with greater capacity to learn hierarchical damage patterns, allowing earlier blocks to capture low-level satellite-specific features while deeper blocks integrate these into complex damage assessments. We employ very conservative learning rates ($5e-5$) for the patch embedding to preserve pretrained knowledge while allowing gradual adaptation to the satellite domain. This stage aims to achieve optimal feature extraction for building damage classification by enabling the model to learn both domain-specific low-level representations and task-specific high-level damage patterns.

We maintain early stopping patience at 3 epochs for Stage 3, despite the increased model complexity, based on the progressive fine-tuning approach employed. Since the model builds upon well-adapted representations from Stage 2, it starts from a strong initialization that should converge more efficiently than training from scratch. The significantly increased computational cost per epoch (due to unfreezing the patch embedding and 6 transformer blocks) makes longer patience periods prohibitively expensive while still allowing sufficient opportunity for convergence. Additionally, the risk of overfitting increases with more trainable parameters, making more aggressive early stopping a beneficial regularization strategy for this final fine-tuning stage.

```

# ViT Fine-tuning (Stage 3: Patch Embedding + Last 6 Transformer Blocks + Head)
# Load Stage 2 model
model = vit_b_16(weights=ViT_B_16_Weights.IMAGENET1K_V1)
model.heads.head = nn.Linear(model.heads.head.in_features, 4)
model.load_state_dict(torch.load("/content/drive/MyDrive/Building Damage Assessment/Classification models/ViT Fine Tuning/vit_stage2.pth"))
model = model.to(device)

# Freeze all parameters first
for param in model.parameters():
    param.requires_grad = False

# Unfreeze patch embedding layer (important for satellite imagery adaptation)
for param in model.conv_proj.parameters(): # Patch embedding
    param.requires_grad = True

# Unfreeze last 6 transformer blocks (building on Stage 2)
for param in model.encoder.layers[-6:].parameters(): # Last 6 blocks
    param.requires_grad = True

# Keep head unfrozen
for param in model.heads.parameters(): # Classification head
    param.requires_grad = True

# Verify unfrozen layers
print("Unfrozen layers in Stage 3:")
for name, param in model.named_parameters():
    if param.requires_grad:
        print(f"- {name}")

# Loss function
criterion = nn.CrossEntropyLoss(weight=class_weights.to(device))

# Optimizer with discriminative learning rates
optimizer = optim.Adam([
    {'params': model.conv_proj.parameters(), 'lr': 5e-5, 'weight_decay': 1e-4}, # Patch embedding - lowest LR
    {'params': model.encoder.layers[-6:-3].parameters(), 'lr': 8e-5, 'weight_decay': 1e-4}, # Blocks -6 to -4
    {'params': model.encoder.layers[-3].parameters(), 'lr': 0.0001, 'weight_decay': 5e-5}, # Block -3 (same as Stage 2)
    {'params': model.encoder.layers[-2].parameters(), 'lr': 0.0002, 'weight_decay': 1e-4}, # Block -2 (same as Stage 2)
    {'params': model.encoder.layers[-1].parameters(), 'lr': 0.0003, 'weight_decay': 1e-4}, # Block -1 (same as Stage 2)
    {'params': model.heads.parameters(), 'lr': 0.0005, 'weight_decay': 1e-4} # Head (same as Stage 2)
])

# Learning rate scheduler
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=3)

# Train Stage 3
model_stage3, history_stage3 = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler,
                                           num_epochs=8, model_name="ViT-B/16 Stage 3 (Patch Embedding + Last 6 Blocks + Head)",
                                           early_stopping_patience=3)

# Save Stage 3
model_path_stage3 = os.path.join(save_dir, "vit_stage3.pth")
history_path_stage3 = os.path.join(save_dir, "history_vit_stage3.pkl")

torch.save(model_stage3.state_dict(), model_path_stage3)

```

```
with open(history_path_stage3, "wb") as f:
    pickle.dump(history_stage3, f)
```

Downloading: "https://download.pytorch.org/models/vit_b_16-c867db91.pth" to /root/.cache/torch/hub/checkpoints/vit_b_16-c867db91.pth
100%|██████████| 330M/330M [00:02<00:00, 140MB/s]

Unfrozen layers in Stage 3:

- conv_proj.weight
- conv_proj.bias
- encoder.layers.encoder_layer_6.ln_1.weight
- encoder.layers.encoder_layer_6.ln_1.bias
- encoder.layers.encoder_layer_6.self_attention.in_proj_weight
- encoder.layers.encoder_layer_6.self_attention.in_proj_bias
- encoder.layers.encoder_layer_6.self_attention.out_proj.weight
- encoder.layers.encoder_layer_6.self_attention.out_proj.bias
- encoder.layers.encoder_layer_6.ln_2.weight
- encoder.layers.encoder_layer_6.ln_2.bias
- encoder.layers.encoder_layer_6.mlp.0.weight
- encoder.layers.encoder_layer_6.mlp.0.bias
- encoder.layers.encoder_layer_6.mlp.3.weight
- encoder.layers.encoder_layer_6.mlp.3.bias
- encoder.layers.encoder_layer_7.ln_1.weight
- encoder.layers.encoder_layer_7.ln_1.bias
- encoder.layers.encoder_layer_7.self_attention.in_proj_weight
- encoder.layers.encoder_layer_7.self_attention.in_proj_bias
- encoder.layers.encoder_layer_7.self_attention.out_proj.weight
- encoder.layers.encoder_layer_7.self_attention.out_proj.bias
- encoder.layers.encoder_layer_7.ln_2.weight
- encoder.layers.encoder_layer_7.ln_2.bias
- encoder.layers.encoder_layer_7.mlp.0.weight
- encoder.layers.encoder_layer_7.mlp.0.bias
- encoder.layers.encoder_layer_7.mlp.3.weight
- encoder.layers.encoder_layer_7.mlp.3.bias
- encoder.layers.encoder_layer_8.ln_1.weight
- encoder.layers.encoder_layer_8.ln_1.bias
- encoder.layers.encoder_layer_8.self_attention.in_proj_weight
- encoder.layers.encoder_layer_8.self_attention.in_proj_bias
- encoder.layers.encoder_layer_8.self_attention.out_proj.weight
- encoder.layers.encoder_layer_8.self_attention.out_proj.bias
- encoder.layers.encoder_layer_8.ln_2.weight
- encoder.layers.encoder_layer_8.ln_2.bias
- encoder.layers.encoder_layer_8.mlp.0.weight
- encoder.layers.encoder_layer_8.mlp.0.bias
- encoder.layers.encoder_layer_8.mlp.3.weight
- encoder.layers.encoder_layer_8.mlp.3.bias
- encoder.layers.encoder_layer_9.ln_1.weight
- encoder.layers.encoder_layer_9.ln_1.bias
- encoder.layers.encoder_layer_9.self_attention.in_proj_weight
- encoder.layers.encoder_layer_9.self_attention.in_proj_bias
- encoder.layers.encoder_layer_9.self_attention.out_proj.weight
- encoder.layers.encoder_layer_9.self_attention.out_proj.bias
- encoder.layers.encoder_layer_9.ln_2.weight
- encoder.layers.encoder_layer_9.ln_2.bias
- encoder.layers.encoder_layer_9.mlp.0.weight
- encoder.layers.encoder_layer_9.mlp.0.bias
- encoder.layers.encoder_layer_9.mlp.3.weight
- encoder.layers.encoder_layer_9.mlp.3.bias
- encoder.layers.encoder_layer_10.ln_1.weight
- encoder.layers.encoder_layer_10.ln_1.bias
- encoder.layers.encoder_layer_10.self_attention.in_proj_weight
- encoder.layers.encoder_layer_10.self_attention.in_proj_bias

```

# Save Stage 3
save_dir = "/content/drive/MyDrive/Building Damage Assessment/Classification models/ViT Fine Tuning"

model_path_stage3 = os.path.join(save_dir, "vit_stage3.pth")
history_path_stage3 = os.path.join(save_dir, "history_vit_stage3.pkl")

torch.save(model_stage3.state_dict(), model_path_stage3)
with open(history_path_stage3, "wb") as f:
    pickle.dump(history_stage3, f)
    - encoder.layers.encoder_layer_11.self_attention.out_proj.weight

# ATTENTION MECHANISM EXPLORATION (Considered to mitigate overfitting)
# Custom ViT with Attention Dropout for better regularization
class ViTWithAttentionDropout(nn.Module):
    def __init__(self, base_model, attention_dropout=0.1):
        super().__init__()
        self.base_model = base_model

        # Add attention dropout to last few blocks
        for layer in self.base_model.encoder.layers[-3:]:
            if hasattr(layer, 'self_attention'):
                layer.self_attention.dropout = nn.Dropout(attention_dropout)

    def forward(self, x):
        return self.base_model(x)

"""
# Load best model from Stage 1 or 2
best_model = model_stage1 if max(history_stage1['val_acc']) > max(history_stage2['val_acc']) else model_stage2

# Wrap with attention dropout
model_with_att_dropout = ViTWithAttentionDropout(best_model, attention_dropout=0.1)
model_with_att_dropout = model_with_att_dropout.to(device)

# Fine-tune with attention dropout
optimizer = optim.Adam(model_with_att_dropout.parameters(), lr=0.0001, weight_decay=1e-4)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=3)

model_att_dropout, history_att_dropout = train_model(model_with_att_dropout, train_loader, val_loader,
                                                    criterion, optimizer, scheduler, num_epochs=10,
                                                    model_name="ViT with Attention Dropout",
                                                    early_stopping_patience=3)
"""

Epoch 4/8
-----
✓ DeiT: 100%|██████████| 1865/1865 [55:35<00:00, 1.79s/it, loss=0.2125, acc=3.9046]
train Loss: 0.3654 Acc: 0.8545
val: 100%|██████████| 626/626 [07:54<00:00, 1.32it/s, loss=1.0330, acc=0.9762]

import timm
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

epoch > 8

# Load pretrained DeiT-B/16
model_stage1 = timm.create_model('deit_base_patch16_224', pretrained=True)

# Freeze all parameters

```

```

for param in model_stage1.parameters():
    param.requires_grad = False

# Replace classification head for ordinal regression (3 thresholds for 4 classes)
model_stage1.head = nn.Linear(model_stage1.head.in_features, 4)

# Unfreeze last transformer block + classification head
for param in model_stage1.blocks[-1].parameters(): # Last transformer block
    param.requires_grad = True
for param in model_stage1.head.parameters(): # Classification head
    param.requires_grad = True

model_stage1 = model_stage1.to(device)

# Loss function
criterion = nn.CrossEntropyLoss(weight=class_weights.to(device))

# Optimizer with discriminative learning rates
optimizer_stage1 = optim.Adam([
    {'params': model_stage1.blocks[-1].parameters(), 'lr': 0.0003, 'weight_decay': 1e-4},
    {'params': model_stage1.head.parameters(), 'lr': 0.0005, 'weight_decay': 1e-4}
])

# Learning rate scheduler
scheduler_stage1 = optim.lr_scheduler.ReduceLROnPlateau(optimizer_stage1, mode='min', factor=0.1, patience=3)

# Train Stage 1
model_stage1, history_stage1 = train_model(
    model_stage1, train_loader, val_loader, criterion, optimizer_stage1, scheduler_stage1,
    num_epochs=15, model_name="DeiT-B Stage 1 (Last Block+Head)",
    early_stopping_patience=3, use_ordinal=True
)

save_dir = r"C:\Users\sajja\Desktop\Dissertation\Classification Models\Fine Tuning\DeiT"
os.makedirs(save_dir, exist_ok=True)
model_path_stage1 = os.path.join(save_dir, "deit_ordinal_stage1.pth")
history_path_stage1 = os.path.join(save_dir, "history_deit_stage1.pkl")

torch.save(model_stage1.state_dict(), model_path_stage1)
with open(history_path_stage1, "wb") as f:
    pickle.dump(history_stage1, f)

""" Output of DeiT Stage 1:
Training DeiT-B Stage 1 (Last Block+Head)
Early stopping patience: 3 epochs
=====
Epoch 1/10
train Loss: 0.7531 Acc: 0.7116
val Loss: 0.6417 Acc: 0.7604

Epoch 2/10
train Loss: 0.6695 Acc: 0.7460
val Loss: 0.6761 Acc: 0.7137

Epoch 3/10
train Loss: 0.6456 Acc: 0.7561

```


val Loss: 0.6939 Acc: 0.7245

Epoch 4/10
train Loss: 0.6302 Acc: 0.7622
val Loss: 0.5862 Acc: 0.7791

Epoch 5/10
train Loss: 0.6191 Acc: 0.7661
val Loss: 0.6245 Acc: 0.7489

Epoch 6/10
train Loss: 0.6049 Acc: 0.7709
val Loss: 0.5717 Acc: 0.7957

Epoch 7/10
train Loss: 0.6003 Acc: 0.7759
val Loss: 0.6599 Acc: 0.7295

Epoch 8/10
train Loss: 0.5930 Acc: 0.7754
val Loss: 0.6140 Acc: 0.7523

Epoch 9/10
train Loss: 0.5834 Acc: 0.7798
val Loss: 0.6047 Acc: 0.7581

Epoch 10/10
train Loss: 0.5543 Acc: 0.7887
val Loss: 0.5870 Acc: 0.7556

Classification Report:

	precision	recall	f1-score	support
no-damage	0.97	0.76	0.85	28852
minor-damage	0.41	0.74	0.53	4715
major-damage	0.49	0.70	0.57	3840
destroyed	0.62	0.85	0.72	2647
accuracy			0.76	40054
macro avg	0.62	0.76	0.67	40054
weighted avg	0.84	0.76	0.78	40054

Training complete in 397m 7s
Best val acc: 0.7957 (epoch 6)
"""

```
# STAGE 2: Last 3 Transformer Blocks + Head
# Load Stage 1 model
model_stage2 = timm.create_model('deit_base_patch16_224', pretrained=True)
model_stage2.head = nn.Linear(model_stage2.head.in_features, 4)
model_stage2.load_state_dict(torch.load("/content/drive/MyDrive/Building Damage Assessment/Classification models/DeiT Fine Tuning/deit_b_cel.pth"))
model_stage2 = model_stage2.to(device)

# Freeze all parameters first
for param in model_stage2.parameters():
    param.requires_grad = False
```

```

# Unfreeze last 3 transformer blocks + classification head
for param in model_stage2.blocks[-3:].parameters(): # Last 3 transformer blocks
    param.requires_grad = True
for param in model_stage2.head.parameters(): # Classification head
    param.requires_grad = True

criterion = nn.CrossEntropyLoss(weight=class_weights.to(device))
# Optimizer with more discriminative learning rates
optimizer_stage2 = optim.Adam([
    {'params': model_stage2.blocks[-1].parameters(), 'lr': 0.0002, 'weight_decay': 1e-4},
    {'params': model_stage2.blocks[-3:-1].parameters(), 'lr': 0.0001, 'weight_decay': 1e-4},
    {'params': model_stage2.head.parameters(), 'lr': 0.0003, 'weight_decay': 1e-4}
])

# Learning rate scheduler with more patience
scheduler_stage2 = optim.lr_scheduler.ReduceLROnPlateau(optimizer_stage2, mode='min', factor=0.2, patience=4)

# Train Stage 2
model_stage2, history_stage2 = train_model(
    model_stage2, train_loader, val_loader, criterion, optimizer_stage2, scheduler_stage2,
    num_epochs=12, model_name="DeiT-B Stage 2 (Last 3 Blocks+Head)",
    early_stopping_patience=3
)

#save_dir = r"C:\Users\sajja\Desktop\Dissertation\Classification Models\Fine Tuning\DeiT"
#os.makedirs(save_dir, exist_ok=True)
#model_path_stage2 = os.path.join(save_dir, "deit_ordinal_stage2.pth")
#history_path_stage2 = os.path.join(save_dir, "history_deit_stage2.pkl")
#torch.save(model_stage2.state_dict(), model_path_stage2)
#with open(history_path_stage2, "wb") as f:
#    pickle.dump(history_stage2, f)
torch.save(model_stage2.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/DeiT Fine Tuning/deit_stage2.pth')
with open('/content/drive/MyDrive/Building Damage Assessment/DeiT Fine Tuning/history_deit_stage2.pkl', 'wb') as f:
    pickle.dump(history_stage2, f)

```

```

/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(

model.safetensors: 100% 346M/346M [00:01<00:00, 270MB/s]

Training DeiT-B Stage 2 (Last 3 Blocks+Head)
Early stopping patience: 3 epochs
=====
Epoch 1/12
-----
train: 100%|██████████| 1865/1865 [32:14<00:00, 1.04s/it, loss=0.0493, acc=3.5223]
train Loss: 0.5960 Acc: 0.7708
val: 100%|██████████| 626/626 [08:40<00:00, 1.20it/s, loss=0.8966, acc=0.9002]
val Loss: 0.5772 Acc: 0.7597
New best model with Validation accuracy of 0.7597

Epoch 2/12
-----
train: 100%|██████████| 1865/1865 [32:40<00:00, 1.05s/it, loss=0.3899, acc=3.5853]
train Loss: 0.5675 Acc: 0.7846
val: 100%|██████████| 626/626 [08:45<00:00, 1.19it/s, loss=0.9870, acc=0.9064]
val Loss: 0.5734 Acc: 0.7650
New best model with Validation accuracy of 0.7650

Epoch 3/12
-----
train: 100%|██████████| 1865/1865 [32:50<00:00, 1.06s/it, loss=0.6274, acc=3.6202]
train Loss: 0.5485 Acc: 0.7923
val: 100%|██████████| 626/626 [08:43<00:00, 1.20it/s, loss=1.1512, acc=0.9531]
val Loss: 0.5301 Acc: 0.8043
New best model with Validation accuracy of 0.8043

Epoch 4/12
-----
train: 100%|██████████| 1865/1865 [33:07<00:00, 1.07s/it, loss=0.5221, acc=3.6533]
train Loss: 0.5283 Acc: 0.7995
val: 100%|██████████| 626/626 [08:50<00:00, 1.18it/s, loss=0.8410, acc=0.9296]
val Loss: 0.5488 Acc: 0.7845
No improvement for 1 epoch(s)

Epoch 5/12
-----
train: 100%|██████████| 1865/1865 [33:00<00:00, 1.06s/it, loss=0.2943, acc=3.7068]
train Loss: 0.4997 Acc: 0.8112
val: 100%|██████████| 626/626 [08:46<00:00, 1.19it/s, loss=0.9442, acc=0.9459]
val Loss: 0.5337 Acc: 0.7983
No improvement for 2 epoch(s)

Classification Report:

```

	precision	recall	f1-score	support
no-damage	0.97	0.81	0.89	28852
minor-damage	0.49	0.70	0.57	4715
major-damage	0.47	0.78	0.59	3840

```

torch.save(model_stage2.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification models/deit_stage2.pth')
with open('/content/drive/MyDrive/Building Damage Assessment/Classification models/DeiT Fine Tuning/deit_stage2.pth/history_deit_stage2.pkl', 'wb') as f:
    pickle.dump(history_stage2, f)

```

```

# STAGE 3: Last 6 Transformer Blocks + Head + Advanced Techniques
# Load Stage 2 model
model_stage3 = timm.create_model('deit_base_patch16_224', pretrained=True)

# Enhanced head with dropout for regularization
class EnhancedOrdinalHead(nn.Module):
    def __init__(self, in_features, num_thresholds=3, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(dropout)
        self.head = nn.Linear(in_features, num_thresholds)

    def forward(self, x):
        return self.head(self.dropout(x))

model_stage3.head = EnhancedOrdinalHead(model_stage3.head.in_features, 4, dropout=0.1) # Changed to 4 to match Stage 2
stage2_state_dict = torch.load('/content/drive/MyDrive/Building Damage Assessment/Classification models/DeiT Fine Tuning/deit_stage2.pth')

new_state_dict = {}
for key, value in stage2_state_dict.items():
    if key.startswith('head.'):
        # Map old head weights to new enhanced head structure
        if key == 'head.weight':
            new_state_dict['head.head.weight'] = value
        elif key == 'head.bias':
            new_state_dict['head.head.bias'] = value
    else:
        new_state_dict[key] = value

model_stage3.load_state_dict(new_state_dict, strict=False)
model_stage3 = model_stage3.to(device)

# Freeze all parameters first
for param in model_stage3.parameters():
    param.requires_grad = False

# Unfreeze last 6 transformer blocks + classification head
for param in model_stage3.blocks[-6:].parameters(): # Last 6 transformer blocks (half the model)
    param.requires_grad = True
for param in model_stage3.head.parameters(): # Enhanced classification head
    param.requires_grad = True

# Use weighted loss if class imbalance exists, otherwise regular CrossEntropy
criterion = nn.CrossEntropyLoss(weight=class_weights.to(device) if 'class_weights' in globals() else None)

# Optimizer with very discriminative learning rates
optimizer_stage3 = optim.AdamW([ # Using AdamW for better weight decay
    {'params': model_stage3.blocks[-2:].parameters(), 'lr': 0.0001, 'weight_decay': 1e-4},
    {'params': model_stage3.blocks[-4:-2].parameters(), 'lr': 0.00005, 'weight_decay': 1e-4},
    {'params': model_stage3.blocks[-6:-4].parameters(), 'lr': 0.00002, 'weight_decay': 1e-4},
    {'params': model_stage3.head.parameters(), 'lr': 0.0002, 'weight_decay': 1e-4}
])

# Fix scheduler variable name - should be scheduler_stage3, not scheduler
scheduler_stage3 = optim.lr_scheduler.ReduceLROnPlateau(optimizer_stage3, mode='min', factor=0.1, patience=3)

```

```
# Train Stage 3
model_stage3, history_stage3 = train_model(
    model_stage3, train_loader, val_loader, criterion, optimizer_stage3, scheduler_stage3,
    num_epochs=12, model_name="DeiT-B Stage 3 (Last 6 Blocks+Enhanced Head)",
    early_stopping_patience=3
)

# Save the model and history
torch.save(model_stage3.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification Models/DeiT Fine Tuning/deit_stage3.pth')
with open('/content/drive/MyDrive/Building Damage Assessment/Classification Models/DeiT Fine Tuning/history_deit_stage3.pkl', 'wb') as f:
    pickle.dump(history_stage3, f)
```

```

/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(

model.safetensors: 100% 346M/346M [00:01<00:00, 266MB/s]

Training DeiT-B Stage 3 (Last 6 Blocks+Enhanced Head)
Early stopping patience: 3 epochs
=====
Epoch 1/12
-----
train: 100%|██████████| 1865/1865 [44:45<00:00, 1.44s/it, loss=0.9262, acc=3.6632]
train Loss: 0.5311 Acc: 0.8017
val: 100%|██████████| 626/626 [09:11<00:00, 1.14it/s, loss=0.7833, acc=0.9673]
val Loss: 0.4964 Acc: 0.8163
New best model with Validation accuracy of 0.8163

Epoch 2/12
-----
train: 100%|██████████| 1865/1865 [44:58<00:00, 1.45s/it, loss=0.1403, acc=3.7437]
train Loss: 0.4836 Acc: 0.8193
val: 100%|██████████| 626/626 [09:16<00:00, 1.12it/s, loss=0.9611, acc=0.9301]
val Loss: 0.5058 Acc: 0.7849
No improvement for 1 epoch(s)

Epoch 3/12
-----
train: 100%|██████████| 1865/1865 [45:00<00:00, 1.45s/it, loss=0.2464, acc=3.7948]
train Loss: 0.4504 Acc: 0.8305
val: 100%|██████████| 626/626 [09:21<00:00, 1.11it/s, loss=1.1861, acc=0.9637]
val Loss: 0.4862 Acc: 0.8133
No improvement for 2 epoch(s)

Epoch 4/12
-----
train: 100%|██████████| 1865/1865 [44:59<00:00, 1.45s/it, loss=0.4291, acc=3.8517]
train Loss: 0.4142 Acc: 0.8429
val: 100%|██████████| 626/626 [09:15<00:00, 1.13it/s, loss=0.9356, acc=0.9914]
val Loss: 0.4693 Acc: 0.8367
New best model with Validation accuracy of 0.8367

Epoch 5/12
-----
train: 100%|██████████| 1865/1865 [45:00<00:00, 1.45s/it, loss=0.3473, acc=3.8842]
train Loss: 0.3854 Acc: 0.8500
val: 100%|██████████| 626/626 [09:17<00:00, 1.12it/s, loss=1.2904, acc=0.9801]
val Loss: 0.4937 Acc: 0.8272
No improvement for 1 epoch(s)

Classification Report:

```

	precision	recall	f1-score	support
no-damage	0.97	0.85	0.91	28852
minor-damage	0.47	0.77	0.58	4715
major-damage	0.62	0.73	0.67	3840

```

import os
torch.save(model_stage3.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification models/deit_stage3.pth')

```

```
# Ensure the directory exists before saving the history file
history_dir = '/content/drive/MyDrive/Building Damage Assessment/Classification models/DeiT Fine Tuning'
os.makedirs(history_dir, exist_ok=True)
```

```
history_path_stage3 = os.path.join(history_dir, 'history_deit_stage3.pkl')
with open(history_path_stage3, 'wb') as f:
    pickle.dump(history_stage3, f)

    No improvement for 2 epoch(s)
```

✓ Test 7/12

```
train: 100%|██████████| 1865/1865 [45:01<00:00, 1.45s/it, loss=0.0591, acc=3.9772]
```

```
import pickle
import os
import numpy as np
from skimage import io
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as transforms
from tqdm import tqdm
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Save the model and history
--> 64 torch.save(model_stage3.state_dict(), '/content/drive/MyDrive/Building Damage Assessment/Classification Models/DeiT Fine Tuning/deit_stage3.pth')
65 with open('/content/drive/MyDrive/Building Damage Assessment/Classification Models/DeiT Fine Tuning/history_deit_stage3.pkl', 'wb') as f:
66     pickle.dump(history_stage3, f)
```

✓ Loading and Creating Test Data

```
# Load test metadata
with open(r'Building_Damage_Metadata\test_building_metadata.pkl', 'rb') as f:
    test_metadata = pickle.load(f)

# Replace Colab-style image paths with local path
for entry in test_metadata:
    entry['image_path'] = entry['image_path'].replace('/content/test/images/', 'geotiffs/test/images/')

# Check sample
print(test_metadata[0]['image_path'])
```

📄 geotiffs/test/images/hurricane-florence_00000119_post_disaster.tif

```
test_files = os.listdir('geotiffs/test/images')
missing_test = [os.path.basename(entry['image_path']) for entry in test_metadata if os.path.basename(entry['image_path']) not in test_files]
print(f"Missing test files: {len(missing_test)}")
```

📄 Missing test files: 0

```
class BuildingDamageDataset(Dataset):
    def __init__(self, metadata_list, target_size=(224, 224), transform=None):
        self.metadata = metadata_list
        self.target_size = target_size # Kept for compatibility, though resizing is now in transforms
        self.transform = transform
```

```

# Label mapping
self.label_map = {
    'no-damage': 0,
    'minor-damage': 1,
    'major-damage': 2,
    'destroyed': 3
}

def __len__(self):
    return len(self.metadata)

def __getitem__(self, idx):
    try:
        meta = self.metadata[idx]

        # Load satellite image
        image = io.imread(meta['image_path'])

        # Handle different image formats
        if len(image.shape) == 2:
            image = np.stack([image, image, image], axis=-1) # Convert grayscale to RGB
        elif len(image.shape) == 3 and image.shape[2] > 3:
            image = image[:, :, :3] # Take first 3 channels if multi-channel

        # Extract crop using stored bbox
        min_x, min_y, max_x, max_y = meta['bbox']
        crop = image[min_y:max_y, min_x:max_x]

        # Convert to float32 and ensure range [0, 255] for transforms
        crop = crop.astype(np.float32)
        if crop.max() > 1.0: # If image is in [0, 255] range, keep it; otherwise, scale
            crop = crop
        else:
            crop = crop * 255.0 # Scale [0, 1] to [0, 255] if needed

        # Apply transforms (resizing and normalization now on GPU)
        if self.transform:
            crop = self.transform(crop)

        # Get label
        label = self.label_map[meta['damage_label']]

        return crop, label

    except Exception as e:
        print(f"Error loading sample {idx}: {e}")
        # Return a dummy sample in case of error
        dummy_crop = torch.zeros(3, self.target_size[0], self.target_size[1])
        return dummy_crop, 0

val_transforms = transforms.Compose([
    transforms.ToPILImage(), # Convert numpy array to PIL Image
    transforms.Resize((224, 224), interpolation=transforms.InterpolationMode.BICUBIC), # Resize to 224x224
    transforms.ToTensor(), # Convert to tensor
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])) # ImageNet normalization

```



```
# Create test dataset
test_dataset = BuildingDamageDataset(test_metadata, target_size=(224, 224), transform=val_transforms)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False, num_workers=0, pin_memory=True if torch.cuda.is_available() else False)
```

✓ Best ResNet Model

Stage 3 is optimal for this critical application because it achieves the best recall rates for high-severity damage classes - 73% recall for major-damage and 85% recall for destroyed buildings, which is crucial for disaster response where missing severely damaged structures could have life-safety implications. The model also maintains strong F1-scores across all damage categories (0.66 for major-damage and 0.82 for destroyed), indicating reliable performance in the second stage of the pipeline. The deeper fine-tuning approach allows the model to learn subtle visual patterns in satellite imagery that distinguish between damage levels, which is essential when the stakes are high and false negatives for severe damage could delay critical aid deployment.

```
from torchvision.models import resnet50, ResNet50_Weights
import torch.nn as nn
import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(device)

# Load ResNet and adjust final layer
model = resnet50(weights=ResNet50_Weights.DEFAULT)
model.fc = nn.Linear(model.fc.in_features, 4) # 4 classes

# Load saved weights
model_path = r"C:\Users\sajja\Desktop\Dissertation\Classification Models\Fine Tuning\Resnet\resnet_L2L3L4.pth"
model.load_state_dict(torch.load(model_path))
model = model.to(device)
model.eval()
```



```
cuda
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
  (layer1): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
  )
  (1): Bottleneck(
    (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
```

```

        (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (relu): ReLU(inplace=True)
    )
    (2): Bottleneck(
      (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
  )
  (layer2): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (1): Bottleneck(
      (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    )
  )

from tqdm import tqdm
import numpy as np

all_preds = []
all_labels = []

with torch.no_grad():
    for inputs, labels in tqdm(test_loader, desc="Testing"):
        inputs, labels = inputs.to(device), labels.to(device)

        outputs = model(inputs)
        _, preds = torch.max(outputs, 1)

        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

all_preds = np.array(all_preds)
all_labels = np.array(all_labels)

🔄 Testing: 100%|██████████| 1225/1225 [06:54<00:00, 2.96it/s]

from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

```

```
# Define label names in the correct order
class_names = ['no-damage', 'minor-damage', 'major-damage', 'destroyed']

# Classification report
print("\nClassification Report:")
print(classification_report(all_labels, all_preds, target_names=class_names, digits=3))

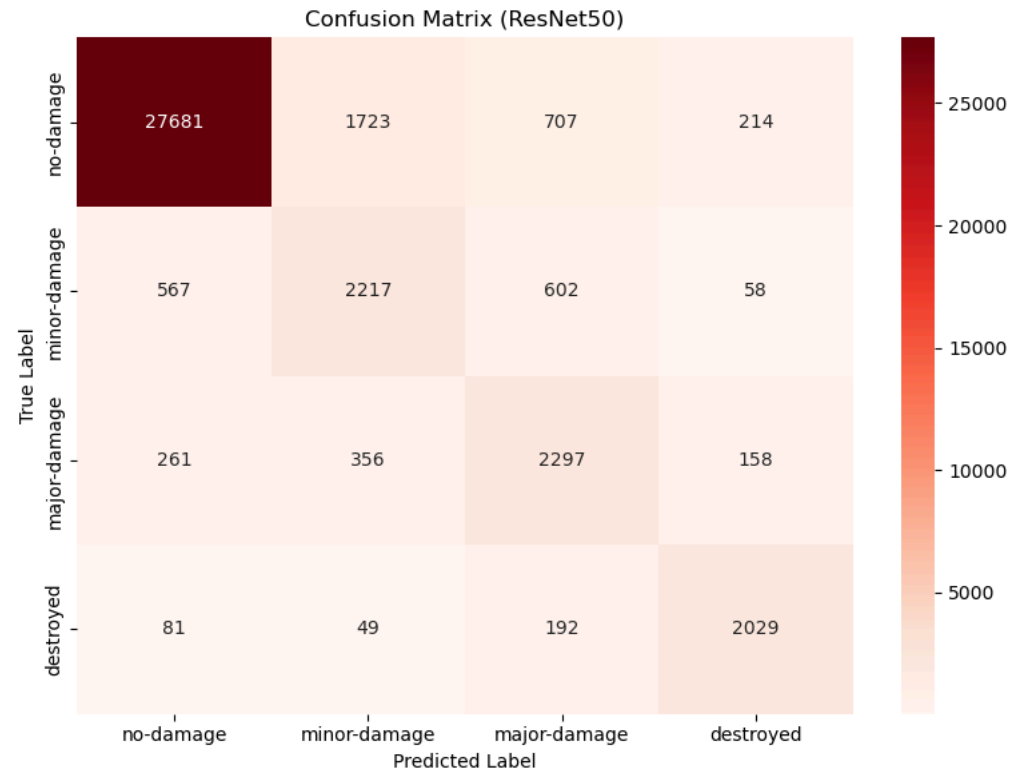
# Confusion matrix
cm = confusion_matrix(all_labels, all_preds)

# Plot confusion matrix
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Reds', xticklabels=class_names, yticklabels=class_names)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix (ResNet50)")
plt.tight_layout()
plt.show()
```



Classification Report:

	precision	recall	f1-score	support
no-damage	0.968	0.913	0.940	30325
minor-damage	0.510	0.644	0.569	3444
major-damage	0.605	0.748	0.669	3072
destroyed	0.825	0.863	0.844	2351
accuracy			0.873	39192
macro avg	0.727	0.792	0.755	39192
weighted avg	0.891	0.873	0.880	39192



```
# Enhanced evaluation with additional metrics and analysis
import torch
import numpy as np
from sklearn.metrics import (
    classification_report, confusion_matrix,
    accuracy_score, balanced_accuracy_score,
    precision_recall_fscore_support,
    cohen_kappa_score, matthews_corrcoef
)
import matplotlib.pyplot as plt
import seaborn as sns
```

```

from tqdm import tqdm
import pandas as pd

def evaluate_model_comprehensive(model, test_loader, device, class_names):
    """
    Comprehensive model evaluation with multiple metrics and visualizations
    """
    model.eval()

    all_preds = []
    all_labels = []
    all_probs = [] # Store probabilities for additional analysis

    with torch.no_grad():
        for inputs, labels in tqdm(test_loader, desc="Testing"):
            inputs, labels = inputs.to(device), labels.to(device)

            outputs = model(inputs)
            probs = torch.softmax(outputs, dim=1) # Get probabilities
            _, preds = torch.max(outputs, 1)

            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())
            all_probs.extend(probs.cpu().numpy())

    all_preds = np.array(all_preds)
    all_labels = np.array(all_labels)
    all_probs = np.array(all_probs)

    return all_preds, all_labels, all_probs

def plot_comprehensive_results(all_labels, all_preds, all_probs, class_names):
    """
    Create comprehensive visualization of results
    """
    fig, axes = plt.subplots(2, 2, figsize=(15, 12))

    # 1. Confusion Matrix
    cm = confusion_matrix(all_labels, all_preds)
    cm_normalized = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]

    sns.heatmap(cm, annot=True, fmt='d', cmap='OrRd',
                xticklabels=class_names, yticklabels=class_names, ax=axes[0,0])
    axes[0,0].set_xlabel("Predicted Label")
    axes[0,0].set_ylabel("True Label")
    axes[0,0].set_title("Confusion Matrix (Counts)")

    # 2. Normalized Confusion Matrix
    sns.heatmap(cm_normalized, annot=True, fmt='.3f', cmap='OrRd',
                xticklabels=class_names, yticklabels=class_names, ax=axes[0,1])
    axes[0,1].set_xlabel("Predicted Label")
    axes[0,1].set_ylabel("True Label")
    axes[0,1].set_title("Confusion Matrix (Normalized)")

    # 3. Class Distribution
    unique_labels, label_counts = np.unique(all_labels, return_counts=True)

```

```

unique_preds, pred_counts = np.unique(all_preds, return_counts=True)

x = np.arange(len(class_names))
width = 0.35

# Ensure all classes are represented
true_counts = np.zeros(len(class_names))
pred_counts_full = np.zeros(len(class_names))

for i, count in zip(unique_labels, label_counts):
    true_counts[i] = count
for i, count in zip(unique_preds, pred_counts):
    pred_counts_full[i] = count

axes[1,0].bar(x - width/2, true_counts, width, label='True', alpha=0.7)
axes[1,0].bar(x + width/2, pred_counts_full, width, label='Predicted', alpha=0.7)
axes[1,0].set_xlabel('Classes')
axes[1,0].set_ylabel('Count')
axes[1,0].set_title('Class Distribution Comparison')
axes[1,0].set_xticks(x)
axes[1,0].set_xticklabels(class_names, rotation=45)
axes[1,0].legend()

# 4. Prediction Confidence Distribution
max_probs = np.max(all_probs, axis=1)
correct_mask = (all_labels == all_preds)

axes[1,1].hist(max_probs[correct_mask], bins=30, alpha=0.7,
               label='Correct Predictions', density=True)
axes[1,1].hist(max_probs[~correct_mask], bins=30, alpha=0.7,
               label='Incorrect Predictions', density=True)
axes[1,1].set_xlabel('Maximum Probability')
axes[1,1].set_ylabel('Density')
axes[1,1].set_title('Prediction Confidence Distribution')
axes[1,1].legend()

plt.tight_layout()
plt.show()

def calculate_detailed_metrics(all_labels, all_preds, all_probs, class_names):
    """
    Calculate comprehensive metrics
    """
    # Basic metrics
    accuracy = accuracy_score(all_labels, all_preds)
    balanced_acc = balanced_accuracy_score(all_labels, all_preds)
    kappa = cohen_kappa_score(all_labels, all_preds)
    mcc = matthews_corrcoef(all_labels, all_preds)

    # Per-class metrics
    precision, recall, f1, support = precision_recall_fscore_support(
        all_labels, all_preds, average=None, labels=range(len(class_names))
    )

    # Macro and weighted averages
    precision_macro, recall_macro, f1_macro, _ = precision_recall_fscore_support(

```

```

        all_labels, all_preds, average='macro'
    )
precision_weighted, recall_weighted, f1_weighted, _ = precision_recall_fscore_support(
    all_labels, all_preds, average='weighted'
)

# Create detailed results dataframe
results_df = pd.DataFrame({
    'Class': class_names,
    'Precision': precision,
    'Recall': recall,
    'F1-Score': f1,
    'Support': support
})

print("="*60)
print("COMPREHENSIVE EVALUATION RESULTS")
print("="*60)
print(f"Overall Accuracy: {accuracy:.4f}")
print(f"Balanced Accuracy: {balanced_acc:.4f}")
print(f"Cohen's Kappa: {kappa:.4f}")
print(f"Matthews Correlation Coefficient: {mcc:.4f}")
print()
print("Macro Averages:")
print(f" Precision: {precision_macro:.4f}")
print(f" Recall: {recall_macro:.4f}")
print(f" F1-Score: {f1_macro:.4f}")
print()
print("Weighted Averages:")
print(f" Precision: {precision_weighted:.4f}")
print(f" Recall: {recall_weighted:.4f}")
print(f" F1-Score: {f1_weighted:.4f}")
print()
print("Per-Class Results:")
print(results_df.to_string(index=False, float_format='%.4f'))

return results_df

def analyze_misclassifications(all_labels, all_preds, all_probs, class_names, top_k=10):
    """
    Analyze the most confident misclassifications
    """
    incorrect_mask = (all_labels != all_preds)
    if not np.any(incorrect_mask):
        print("No misclassifications found!")
        return

    incorrect_indices = np.where(incorrect_mask)[0]
    incorrect_confidences = np.max(all_probs[incorrect_mask], axis=1)

    # Get top-k most confident misclassifications
    top_incorrect_indices = incorrect_indices[np.argsort(-incorrect_confidences)[:top_k]]

    print(f"\nTop {min(top_k, len(top_incorrect_indices))} Most Confident Misclassifications:")
    print("-" * 70)

```

```
for i, idx in enumerate(top_incorrect_indices):
    true_label = class_names[all_labels[idx]]
    pred_label = class_names[all_preds[idx]]
    confidence = np.max(all_probs[idx])

    print(f"{i+1:2d}. Sample {idx:4d}: {true_label:12s} → {pred_label:12s} "
          f"(confidence: {confidence:.3f})")

# Usage example:
# Assuming you have your model, test_loader, device, and class_names defined

class_names = ['no-damage', 'minor-damage', 'major-damage', 'destroyed']

# Run comprehensive evaluation
all_preds, all_labels, all_probs = evaluate_model_comprehensive(
    model, test_loader, device, class_names
)

# Calculate detailed metrics
results_df = calculate_detailed_metrics(all_labels, all_preds, all_probs, class_names)

# Create comprehensive visualizations
plot_comprehensive_results(all_labels, all_preds, all_probs, class_names)

# Analyze misclassifications
analyze_misclassifications(all_labels, all_preds, all_probs, class_names)

# Original classification report for comparison
print("\n" + "="*60)
print("SKLEARN CLASSIFICATION REPORT (for comparison)")
print("="*60)
print(classification_report(all_labels, all_preds, target_names=class_names, digits=4))
```



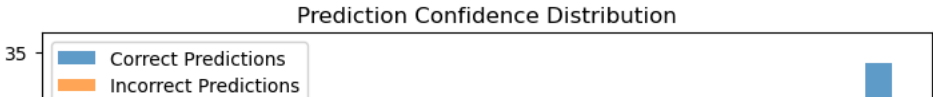
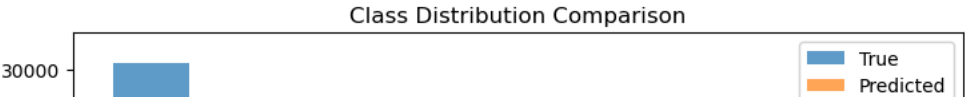
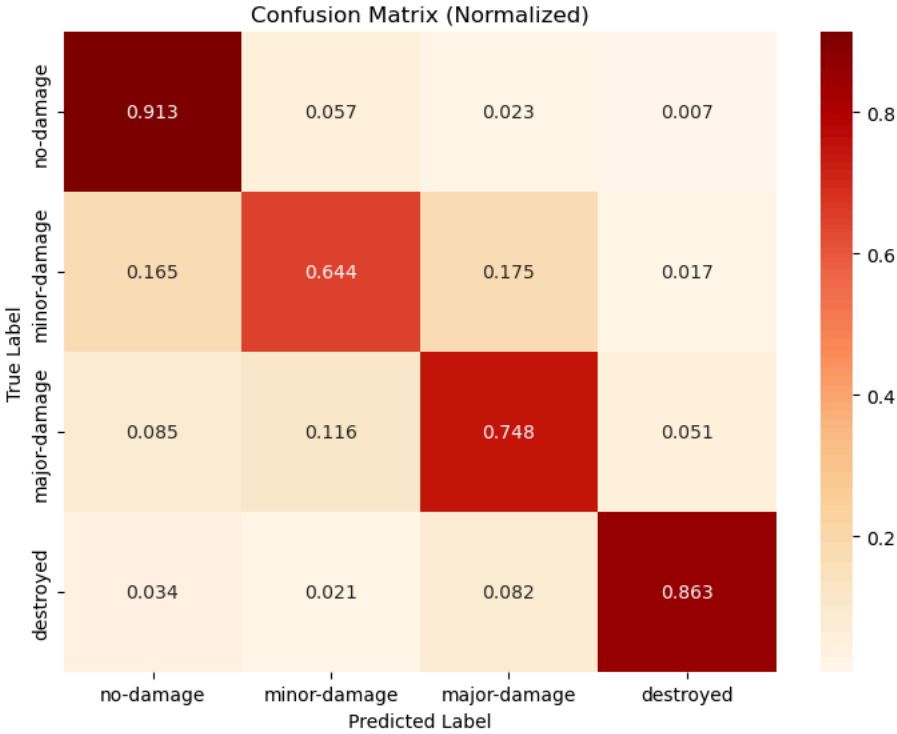
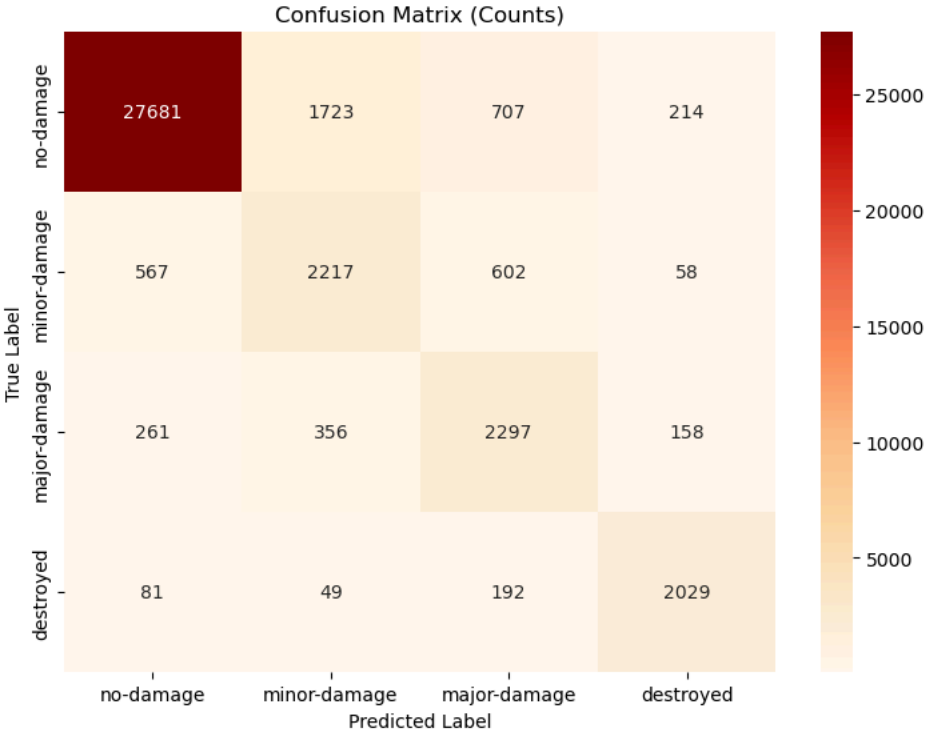
```
Testing: 100%|██████████| 1225/1225 [07:57<00:00, 2.57it/s]
=====
COMPREHENSIVE EVALUATION RESULTS
=====
Overall Accuracy: 0.8732
Balanced Accuracy: 0.7918
Cohen's Kappa: 0.6942
Matthews Correlation Coefficient: 0.6983

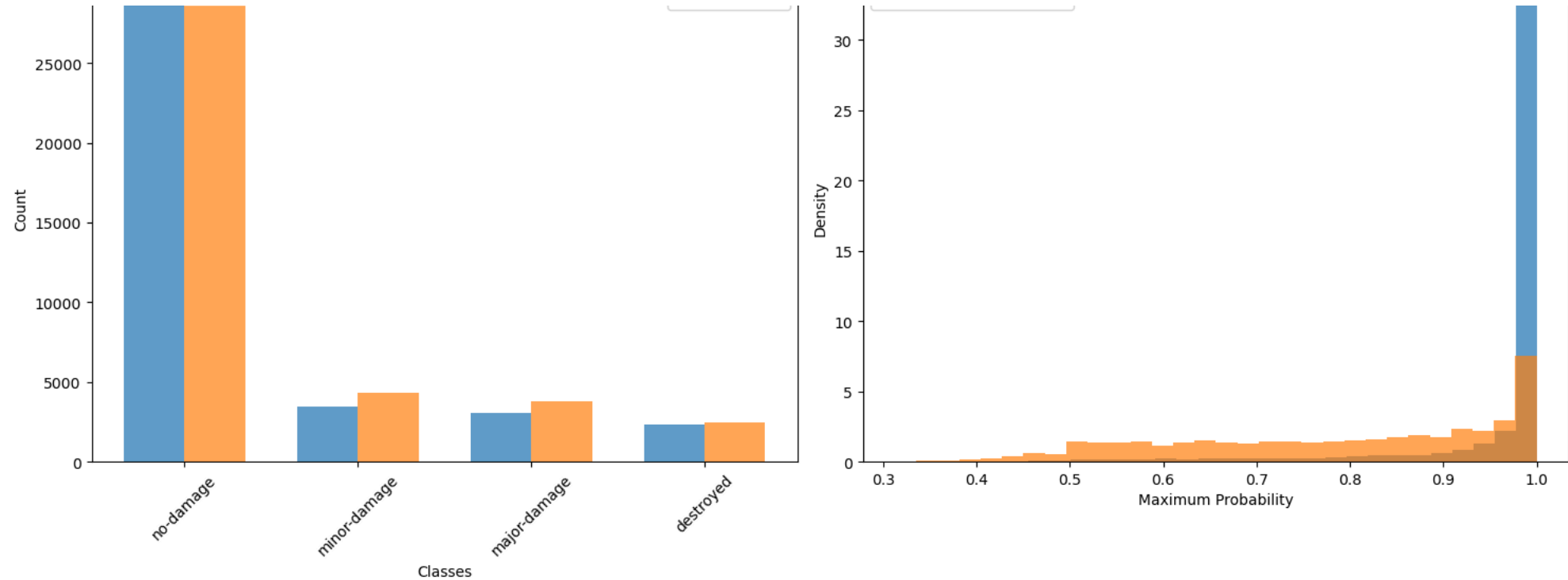
Macro Averages:
Precision: 0.7271
Recall: 0.7918
F1-Score: 0.7553

Weighted Averages:
Precision: 0.8909
Recall: 0.8732
F1-Score: 0.8801
```

Per-Class Results:

Class	Precision	Recall	F1-Score	Support
no-damage	0.9682	0.9128	0.9397	30325
minor-damage	0.5102	0.6437	0.5693	3444
major-damage	0.6048	0.7477	0.6687	3072
destroyed	0.8251	0.8630	0.8437	2351





Top 10 Most Confident Misclassifications:

- 1. Sample 23305: minor-damage → no-damage (confidence: 1.000)
- 2. Sample 17536: minor-damage → no-damage (confidence: 1.000)
- 3. Sample 20851: minor-damage → no-damage (confidence: 1.000)
- 4. Sample 23483: no-damage → major-damage (confidence: 1.000)
- 5. Sample 34407: major-damage → destroyed (confidence: 1.000)
- 6. Sample 23470: no-damage → major-damage (confidence: 1.000)
- 7. Sample 7718: major-damage → destroyed (confidence: 1.000)
- 8. Sample 32670: minor-damage → no-damage (confidence: 1.000)
- 9. Sample 26690: no-damage → destroyed (confidence: 1.000)
- 10. Sample 23471: no-damage → major-damage (confidence: 1.000)

=====

SKLEARN CLASSIFICATION REPORT (for comparison)

=====

	precision	recall	f1-score	support
no-damage	0.9682	0.9128	0.9397	30325
minor-damage	0.5102	0.6437	0.5693	3444
major-damage	0.6048	0.7477	0.6687	3072
destroyed	0.8251	0.8630	0.8437	2351
accuracy			0.8732	39192
macro avg	0.7271	0.7918	0.7553	39192
weighted avg	0.8909	0.8732	0.8801	39192

```

import matplotlib.pyplot as plt
import numpy as np
import torch
import random

def denormalize_image(tensor, mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]):
    """Denormalize ImageNet normalized tensor for visualization"""
    tensor = tensor.clone()
    for t, m, s in zip(tensor, mean, std):
        t.mul_(s).add_(m)
    return torch.clamp(tensor, 0, 1)

def visualize_random_predictions(model, test_loader, device, class_names, n_images=20):
    """Simple visualization of random test images with predictions"""

    model.eval()
    selected_samples = []

    with torch.no_grad():
        for inputs, labels in test_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            _, preds = torch.max(outputs, 1)

            for i in range(inputs.size(0)):
                selected_samples.append({
                    'image': inputs[i].cpu(),
                    'true_label': labels[i].item(),
                    'pred_label': preds[i].item()
                })

            # Stop when we have enough samples
            if len(selected_samples) >= n_images:
                break

    # Stop when we have enough samples
    if len(selected_samples) >= n_images:
        break

    # Create grid layout
    cols = 10
    rows = (n_images + cols - 1) // cols # Ceiling division

    fig, axes = plt.subplots(rows, cols, figsize=(15, 3*rows))
    axes = axes.flatten() if n_images > cols else [axes] if n_images == 1 else axes

    for i in range(n_images):
        if i < len(selected_samples):
            sample = selected_samples[i]

            # Denormalize and display image
            img = denormalize_image(sample['image'])
            img_np = img.permute(1, 2, 0).numpy()

            axes[i].imshow(img_np)

```

```

axes[i].set_title(f"Label: {sample['true_label']} Pred: {sample['pred_label']}",
                  fontsize=10)

axes[i].axis('off')

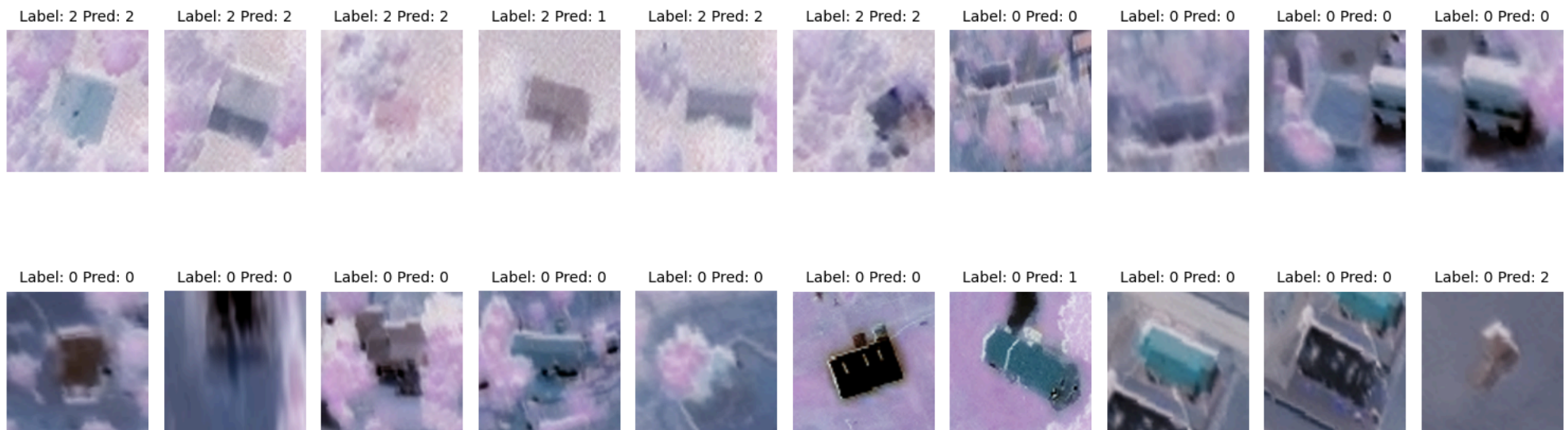
# Hide extra subplots if any
for i in range(n_images, len(axes)):
    axes[i].axis('off')

plt.tight_layout()
plt.show()

class_names = ['no-damage', 'minor-damage', 'major-damage', 'destroyed']

# Show 20 random images
visualize_random_predictions(model, test_loader, device, class_names, n_images=20)

```



✓ Best EfficientNet Model

Stage 3 represents the optimal EfficientNet-B3 configuration, achieving the highest validation accuracy (85.40%) and delivering superior performance across all damage severity levels. Most critically for disaster response, it achieves 71% recall for major-damage and 83% recall for destroyed buildings, with strong F1-scores of 0.66 and 0.84 respectively, ensuring reliable detection of high-severity cases where false negatives could have life-safety implications. The model maintains excellent performance on the dominant no-damage class (91% recall, 0.93 F1-score) while achieving balanced 63% recall for minor-damage, demonstrating robust classification across the full damage spectrum. The deeper fine-tuning strategy (unfreezing stages 5+6+7+classifier) enables the model to learn sophisticated feature hierarchies from satellite imagery, capturing subtle visual patterns that distinguish between damage levels.

```

from torchvision.models import efficientnet_b3, EfficientNet_B3_Weights
import torch.nn as nn

```

```

import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = efficientnet_b3(weights=EfficientNet_B3_Weights.IMAGENET1K_V1)

# Load EfficientNet-B3 model and adjust final layer
model = efficientnet_b3(weights=EfficientNet_B3_Weights.IMAGENET1K_V1)
model.classifier[1] = nn.Linear(model.classifier[1].in_features, 4) # 4 classes

# Load best EfficientNet model weights (Stage 2)
model_path = r"C:\Users\sajja\Desktop\Dissertation\Classification Models\Fine Tuning\Efficientnet\efficientnet_stage3.pth"
model.load_state_dict(torch.load(model_path, map_location=device))
model = model.to(device)
model.eval()

# Evaluation
all_preds = []
all_labels = []

with torch.no_grad():
    for inputs, labels in tqdm(test_loader, desc="Testing EfficientNet"):
        inputs, labels = inputs.to(device), labels.to(device)

        outputs = model(inputs)
        _, preds = torch.max(outputs, 1)

        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

all_preds = np.array(all_preds)
all_labels = np.array(all_labels)

# Calculate and display metrics
# Define label names in the correct order
class_names = ['no-damage', 'minor-damage', 'major-damage', 'destroyed']

# Classification report
print("\nClassification Report (EfficientNet-B3):")
print(classification_report(all_labels, all_preds, target_names=class_names, digits=3))

# Confusion matrix
cm = confusion_matrix(all_labels, all_preds)

# Plot confusion matrix
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=class_names, yticklabels=class_names)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix (EfficientNet-B3)")
plt.tight_layout()
plt.show()

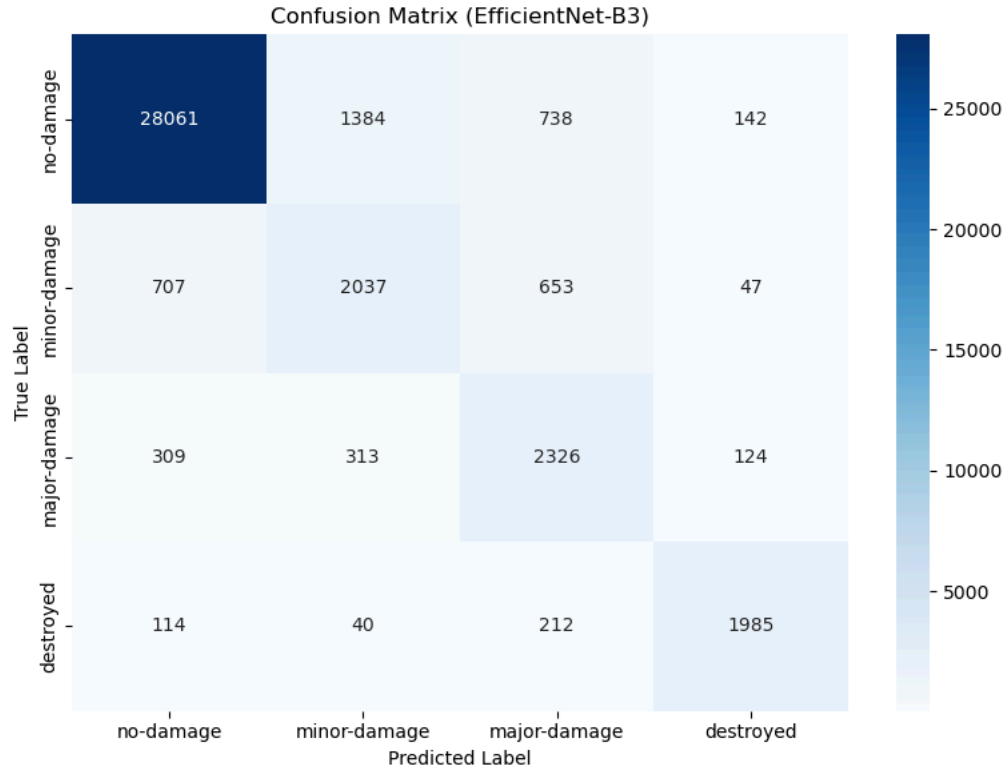
print(f"Total test samples: {len(all_labels)}")
print(f"Overall accuracy: {(all_preds == all_labels).mean():.4f}")

```

Testing EfficientNet: 100% | 1225/1225 [06:55<00:00, 2.95it/s]

Classification Report (EfficientNet-B3):

	precision	recall	f1-score	support
no-damage	0.961	0.925	0.943	30325
minor-damage	0.540	0.591	0.564	3444
major-damage	0.592	0.757	0.664	3072
destroyed	0.864	0.844	0.854	2351
accuracy			0.878	39192
macro avg	0.739	0.780	0.756	39192
weighted avg	0.889	0.878	0.883	39192



Total test samples: 39192
Overall accuracy: 0.8780

Best ViT Model

Contrary to expectations, ViT-B/16 achieved peak overall accuracy (86.97%) in Stage 1 with minimal fine-tuning, suggesting that Vision Transformer's pre-trained self-attention mechanisms are remarkably well-suited for satellite damage classification. However, we selected Stage 3 (84.23% accuracy) as the optimal configuration despite the 2.74% accuracy decrease, because it delivers superior recall for mission-critical damage classes: 86% recall for destroyed buildings versus 80% in Stage 1, and 74% recall for major-damage versus 71% in

Stage 1. In disaster response applications, the operational cost of missing severely damaged buildings that may contain survivors or pose safety hazards far outweighs the cost of lower overall accuracy, making Stage 3's enhanced sensitivity to high-severity damage the optimal choice for our emergency response pipeline where false negatives could have life-safety implications.

```
from torchvision.models import vit_b_16, ViT_B_16_Weights
import torch.nn as nn
import torch
```

```
model = vit_b_16(weights=ViT_B_16_Weights.IMAGENET1K_V1)
model.heads.head = nn.Linear(model.heads.head.in_features, 4) # 4 classes
```

```
# Load best ViT model weights (Stage 3)
model_path = r"C:\Users\sajja\Desktop\Dissertation\Classification Models\Fine Tuning\ViT\vit_stage3.pth" # Update this path to your actual model path
model.load_state_dict(torch.load(model_path, map_location=device))
model = model.to(device)
model.eval()
```

```
→ VisionTransformer(
  (conv_proj): Conv2d(3, 768, kernel_size=(16, 16), stride=(16, 16))
  (encoder): Encoder(
    (dropout): Dropout(p=0.0, inplace=False)
    (layers): Sequential(
      (encoder_layer_0): EncoderBlock(
        (ln_1): LayerNorm((768,), eps=1e-06, elementwise_affine=True)
        (self_attention): MultiheadAttention(
          (out_proj): NonDynamicallyQuantizableLinear(in_features=768, out_features=768, bias=True)
        )
        (dropout): Dropout(p=0.0, inplace=False)
        (ln_2): LayerNorm((768,), eps=1e-06, elementwise_affine=True)
        (mlp): MLPBlock(
          (0): Linear(in_features=768, out_features=3072, bias=True)
          (1): GELU(approximate='none')
          (2): Dropout(p=0.0, inplace=False)
          (3): Linear(in_features=3072, out_features=768, bias=True)
          (4): Dropout(p=0.0, inplace=False)
        )
      )
      (encoder_layer_1): EncoderBlock(
        (ln_1): LayerNorm((768,), eps=1e-06, elementwise_affine=True)
        (self_attention): MultiheadAttention(
          (out_proj): NonDynamicallyQuantizableLinear(in_features=768, out_features=768, bias=True)
        )
        (dropout): Dropout(p=0.0, inplace=False)
        (ln_2): LayerNorm((768,), eps=1e-06, elementwise_affine=True)
        (mlp): MLPBlock(
          (0): Linear(in_features=768, out_features=3072, bias=True)
          (1): GELU(approximate='none')
          (2): Dropout(p=0.0, inplace=False)
          (3): Linear(in_features=3072, out_features=768, bias=True)
          (4): Dropout(p=0.0, inplace=False)
        )
      )
      (encoder_layer_2): EncoderBlock(
        (ln_1): LayerNorm((768,), eps=1e-06, elementwise_affine=True)
        (self_attention): MultiheadAttention(
          (out_proj): NonDynamicallyQuantizableLinear(in_features=768, out_features=768, bias=True)
        )
        (dropout): Dropout(p=0.0, inplace=False)
        (ln_2): LayerNorm((768,), eps=1e-06, elementwise_affine=True)
```

```

        (mlp): MLPBlock(
          (0): Linear(in_features=768, out_features=3072, bias=True)
          (1): GELU(approximate='none')
          (2): Dropout(p=0.0, inplace=False)
          (3): Linear(in_features=3072, out_features=768, bias=True)
          (4): Dropout(p=0.0, inplace=False)
        )
      )
    (encoder_layer_3): EncoderBlock(
      (ln_1): LayerNorm((768,), eps=1e-06, elementwise_affine=True)
      (self_attention): MultiheadAttention(
        (out_proj): NonDynamicallyQuantizableLinear(in_features=768, out_features=768, bias=True)
      )
      (dropout): Dropout(p=0.0, inplace=False)
      (ln_2): LayerNorm((768,), eps=1e-06, elementwise_affine=True)
      (mlp): MLPBlock(

# Evaluation
all_preds = []
all_labels = []

with torch.no_grad():
    for inputs, labels in tqdm(test_loader, desc="Testing ViT-B/16"):
        inputs, labels = inputs.to(device), labels.to(device)

        outputs = model(inputs)
        _, preds = torch.max(outputs, 1)

        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

all_preds = np.array(all_preds)
all_labels = np.array(all_labels)

# Calculate and display metrics
# Define label names in the correct order
class_names = ['no-damage', 'minor-damage', 'major-damage', 'destroyed']

# Classification report
print("\nClassification Report (ViT-B/16):")
print(classification_report(all_labels, all_preds, target_names=class_names, digits=3))

# Confusion matrix
cm = confusion_matrix(all_labels, all_preds)

# Plot confusion matrix
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens', xticklabels=class_names, yticklabels=class_names)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix (ViT-B/16)")
plt.tight_layout()
plt.show()

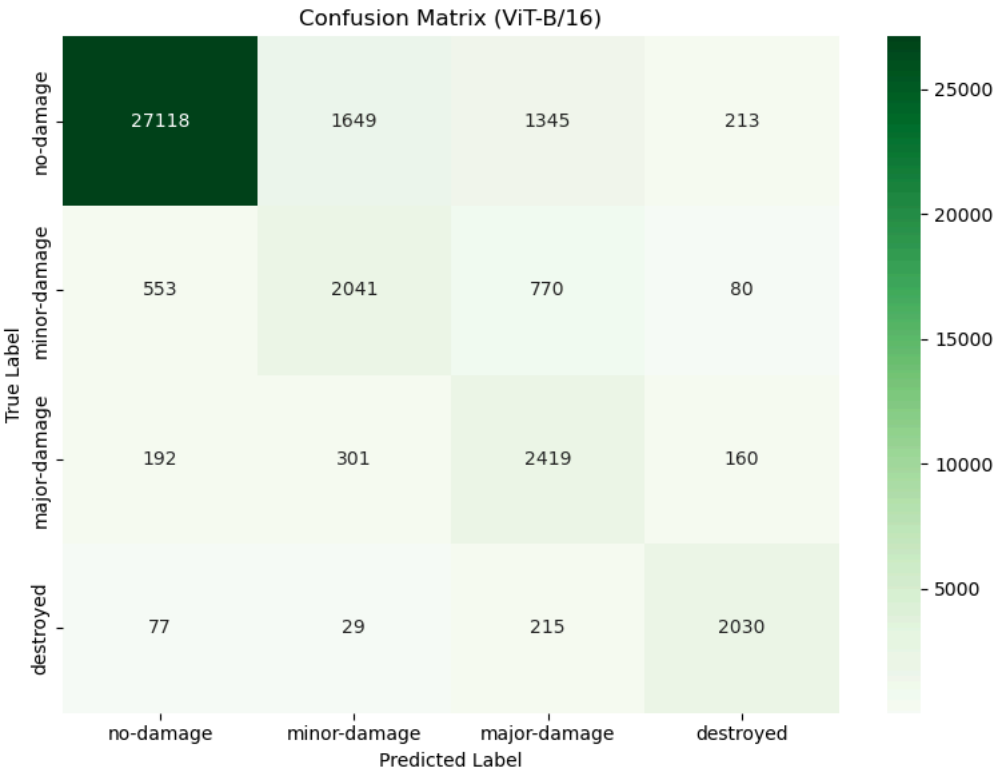
print(f"Total test samples: {len(all_labels)}")
print(f"Overall accuracy: {(all_preds == all_labels).mean():.4f}")

```


Testing ViT-B/16: 100%1225/1225 [12:03<00:00, 1.69it/s]

Classification Report (ViT-B/16):

	precision	recall	f1-score	support
no-damage	0.971	0.894	0.931	30325
minor-damage	0.508	0.593	0.547	3444
major-damage	0.509	0.787	0.619	3072
destroyed	0.818	0.863	0.840	2351
accuracy			0.858	39192
macro avg	0.701	0.784	0.734	39192
weighted avg	0.885	0.858	0.867	39192



Total test samples: 39192
Overall accuracy: 0.8575